

Vysoká škola ekonomická v Praze

Fakulta informatiky a statistiky



Vývoj webových aplikací pomocí umělé inteligence

BAKALÁŘSKÁ PRÁCE

Studijní program: Informační média a služby

Autor: Volha Rachkouskaya

Vedoucí bakalářské práce: Ing. Richard Antonín Novák, Ph.D.

Praha, červen 2025

Poděkování

Chtěla bych poděkovat panu Ing. Richardu Antonínu Novákovi, Ph.D. za věnovaný čas, trpělivost a odborné rady, poskytnuté při zpracování této práce.

Abstrakt

Tato práce se zabývá využitím systémů založených na umělé inteligenci (AI) při vývoji webových aplikací. Cílem práce je porovnání dvou přístupů k vývoji – tradičního vývoje a programování s podporou umělé inteligence. Při hodnocení obou přístupů se autor zaměří na předem definovaná kritéria, která zahrnují kvalitu kódu, uživatelskou přívětivost výsledného řešení, časovou náročnost, míru potřebné odbornosti a zkušenost vývojáře.

Klíčová slova

Webový design, vývoj webových stránek, umělá inteligence (AI), generativní AI, efektivita.

Abstract

This thesis deals with the use of artificial intelligence (AI) systems in web application development. The aim of the thesis is to compare two approaches to development – traditional development and programming with artificial intelligence support. When evaluating both approaches, the author focuses on predefined criteria, which include code quality, user-friendliness of the final solution, time consumption, and the level of expertise and experience required of the developer.

Keywords

Web design, web development, artificial intelligence (AI), generative AI, effectivity

Obsah

Úvod	11
1 Literární rešerše	12
1.1 Webové aplikace	12
1.2 Typy webových aplikací.....	12
1.3 Specifikace požadavků.....	14
1.4 Životní cyklus vývoje softwaru	14
1.5 Implementační procesy	15
1.6 Trendy v moderních webových aplikacích	17
1.7 Využité technologie	19
1.7.1 Javascript	19
1.7.2 Node.js	20
1.7.3 React.js.....	20
1.7.4 Next.js	21
1.7.5 MongoDB.....	22
1.8 Úvod do AI	22
1.9 Úvod do LLM.....	24
1.9.1 Definice LLM.....	25
1.9.2 Historie LLM	25
1.9.3 Generative Pre-trained Transformer.....	26
1.10 Vybrané nástroje založené na velkých jazykových modelech.....	27
1.10.1 ChatGPT	28
1.10.2 GitHub Copilot.....	29
1.10.3 Cursor.....	30
1.10.4 Tabnine	31
1.10.5 Windsurf	32
1.11 Prompt inženýrství.....	33
1.12 Praktické využití LLM ve vývoji.....	34
1.13 Výhody a omezení využití umělé inteligence v programování	37
1.14 Metody multi-kriteriálního hodnocení.....	39
1.14.1 Metoda AHP Saatyho	39
2 Návrh projektu	41
2.1 Představení firmy	41
2.2 Současný stav používaného webu	41

2.3 Návrh nového řešení	41
2.4 Stanovení požadavků	42
2.4.1 Funkční požadavky pro administrátorský panel	42
2.4.2 Funkční požadavky pro stránky e-shopu	42
2.4.3 Dodatečné požadavky.....	43
3 Implementace řešení bez použití AI	44
3.1 Panel administrátora.....	44
3.1.1 Struktura projektu.....	44
3.1.2 Připojení k databázi	45
3.1.3 Funkce Autentizace.....	47
3.1.4 Správa kategorií	50
3.1.5 Správa produktů	54
3.1.6 Správa uživatelů.....	56
3.1.7 Správa objednávek.....	58
3.2 Stránky E-shopu	60
3.2.1 Struktura projektu	60
3.2.2 Domovská stránka.....	61
3.2.3 Stránka „Produkty“	61
3.2.4 Osobní účet.....	62
3.2.5 Seznam oblíbených produktů.....	68
3.2.6 Nákupní košík	71
3.2.7 Objednání zboží.....	72
3.2.8 Rezervace termínu pro konzultaci	74
3.2.9 Stránka „Služby“	77
3.2.10 Stránka „Kontakt“	77
4 Implementace řešení pomocí AI	78
4.1 Postup řešení	78
4.2 Výběr nástrojů.....	78
4.3 Panel administrátora	79
4.3.1 Struktura souboru.....	79
4.3.2 Připojení k databázi	80
4.3.3 Funkce Autentizace	81
4.3.4 Správa dat pomocí CRUD operací	82
4.3.5 Vytváření komponent uživatelského rozhraní	85
4.4 Stránky e-shopu	87

4.4.1 Použití LocalStorage	87
4.4.2 Vytvoření prvků uživatelského rozhraní	88
5 Porovnání odlišných metod vývoje konkrétního webu	91
5.1 Definice kritérií.....	91
5.2 Výpočet vah kritérií.....	92
5.3 Hodnocení přístupů v rámci vybraných kritérií	93
5.3.1 Kvalita kódu	93
5.3.2 Uživatelská přívětivost	94
5.3.3 Časová náročnost.....	95
5.3.4 Míra potřebné odbornosti	95
5.3.5 Vývojářská zkušenost	96
5.4 Výpočet výsledného hodnocení.....	96
Závěr	98
Diskuse	99
Použitá literatura	101

Seznam obrázků

Obrázek 1: životní cyklus vývoje softwaru zdroj:(Aytemiz, 2023).....	15
--	----

Obrázek 2: vodopádový model zdroj: (<i>What Is the Waterfall Model in Software Engineering?</i> , 2014).....	16
Obrázek 3: ukázka použití asynchronní funkce (zdroj: Parody, 2025)	20
Obrázek 4: hierarchicky znázorněné různé typy umělé inteligence (zdroj: Taulli, 2024) .	24
Obrázek 5: uživatelské rozhraní aplikace ChatGPT pro výběr specializovaných GPT modelů určených k asistenci při programování (zdroj: autor).....	29
Obrázek 6: zjednodušené schéma složkové struktury aplikace (zdroj: autor).....	45
Obrázek 7: přihlašovací stránka (zdroj: autor)	50
Obrázek 8: stránka "Kategorie" (zdroj: autor)	50
Obrázek 9: tlačítko pro úpravu a odstranění kategorie (zdroj: autor)	51
Obrázek 10: přidání nové kategorie (zdroj: autor)	51
Obrázek 11: volba nadřazené kategorie (zdroj: autor).....	51
Obrázek 12: volba nadřazené kategorie (zdroj: autor)	53
Obrázek 13: stránka "Produkty", (zdroj: autor)	54
Obrázek 14: přidání nového produktu (zdroj: autor)	55
Obrázek 15: stránka "Správci" (zdroj: autor)	56
Obrázek 16: formulář pro přidání nového správce (zdroj: autor)	57
Obrázek 17: stránka "Objednávky" (zdroj: autor)	59
Obrázek 18: zjednodušené schéma složkové struktury webových stránek pro E-shop (zdroj: autor).....	60
Obrázek 19: uživatelské rozhraní zobrazující hlavní kategorie (zdroj: autor)	61
Obrázek 20: stránka s přehledem produktů vybrané kategorie (zdroj: autor)	62
Obrázek 21: registrační formulář (zdroj: autor)	63
Obrázek 22: dodatečná pole pro registraci firmy (zdroj: autor)	64
Obrázek 23: zpráva informující o nedostatečné délce hesla (zdroj: autor).....	64
Obrázek 24: zobrazení uložených osobních údajů (zdroj: autor).....	66
Obrázek 25: zobrazení předchozích objednávek v osobním účtu (zdroj: autor).....	68
Obrázek 26: stránka pro přidání oblíbených produktů (zdroj: autor)	69
Obrázek 27: zobrazení produktů na stránce "Nákupní košík"	71
Obrázek 28: stránka pro zadání podrobností objednávky (zdroj: autor).....	73
Obrázek 29: stránka potvrzující úspěšné dokončení nákupu (zdroj: autor).....	74
Obrázek 30: UI komponenta sloužící k prezentaci konzultačních služeb (zdroj: autor)	75
Obrázek 31: uživatelské rozhraní pro výběr preferovaného způsobu konzultace (zdroj: autor).....	76
Obrázek 32: uživatelské rozhraní pro výběr preferovaného času konzultace	76
Obrázek 33: formulář pro zadání klientských údajů (zdroj: autor)	76
Obrázek 34: struktura souborů navržená nástrojem GitHub Copilot (zdroj: autor)	79
Obrázek 35: stránka pro přihlášení vytvořená pomocí GitHub Copilot (zdroj: autor)	86
Obrázek 36: stránka "Objednávky" vytvořená pomocí GitHub Copilot (zdroj: autor)	86
Obrázek 37: stránka pro přidání nového produktu vytvořená pomocí GitHub Copilot (zdroj: autor)	87
Obrázek 38: ukázka kódu vlastního React hooku <code>useLocalStorage</code> vygenerovaného nástrojem ChatGPT.....	88
Obrázek 39: obrázek poskytnutý nástroji ChatGPT jako vzor (zdroj: autor).....	89
Obrázek 40: vzhled prvku Header vytvořeného pomocí ChatGPT (zdroj: autor).....	89
Obrázek 41: stránka "Nákupní košík" vytvořená pomocí ChatGPT (zdroj: autor)	89

Obrázek 42: stránka s přehledem produktů vybrané kategorie vytvořená pomocí ChatGPT (zdroj: autor)	90
---	----

Seznam výpisů programového kódu

Výpis 1: ukázka připojení k databázi	46
Výpis 2: připojení k MongoDB pomocí knihovny mongoose	47
Výpis 3: ukázka nastavení knihovny NextAuth.js	48

Výpis 4: ukázka funkce bcrypt.hash	48
Výpis 5:použití komponenty <SessionProvider>	49
Výpis 6: získání dat relace pomocí funkce useSession	49
Výpis 7: ukázka použití metody signIn.....	50
Výpis 8: Ukázka kódu pro připojení k službě Amazon S3 a odeslání souboru	52
Výpis 9: model "Category"	54
Výpis 10: definování modelu "Product"	56
Výpis 11: funkce pro kontrolu unikátnosti zadaného hesla	58
Výpis 12: definování modelu User	58
Výpis 13 : funkce GET v API endpointu Orders	59
Výpis 14: stránka "Osobní účet" (zdroj: autor).....	63
Výpis 15: volání funkce pro uložení nového uživatele na straně klienta	65
Výpis 16: ukázka funkce POST v souboru Clients	66
Výpis 17: volání funkce pro úpravu uživatelských údajů	66
Výpis 18: ukázka funkce PUT v souboru Clients	67
Výpis 19: ukázka funkce GET v souboru Order	68
Výpis 20:ukázka kódu vlastního React hooku useLocalStorage	70
Výpis 21: ukázka kódu ze souboru wishlistContext	70
Výpis 22: ukázka kódu vlastního React hooku useCartProducts	72
Výpis 23 : funkce pro vytvoření platební relace pomoci služby Stripe.....	74
Výpis 24: ukázka kódu pro připojení k databázi vygenerovaného nástrojem GitHub Copilot	81
Výpis 25: ukázka kódu pro konfiguraci autentizace pomocí NextAuth.js vygenerovaného nástrojem GitHub Copilot	82
Výpis 26: rozšíření kódu o diagnostické funkce pomocí nástroje GitHub Copilot	84

Úvod

Webové aplikace představují dynamicky se rozvíjející oblast softwarového inženýrství, v níž dochází k neustálému vývoji a prosazování nových trendů. Objevují se inovativní technologie, vývojová prostředí, softwarové nástroje, vývojové a designové přístupy.

Jedním z významných trendů v oblasti webových aplikací je programování s podporou umělé inteligence. Tento přístup má potenciál zásadním způsobem transformovat proces tvorby webových aplikací ve srovnání s tradičními metodami, jako je ruční psaní kódu či přístupy typu low-code a no-code. Vývoj pomocí systémů založených na umělé inteligenci slibuje zvýšení efektivity vývojového procesu, zlepšení kvality výsledného kódu a současně zpřístupnění tvorby webových aplikací i uživatelům s nižší úrovní technických znalostí.

Cíl práce

Tato práce si klade za cíl na základě vícekritériálního hodnocení porovnat dva přístupy k vývoji webových aplikací – tradiční vývoj a programování s podporou umělé inteligence. Při hodnocení obou přístupů se autor zaměří na předem definovaná kritéria, která zahrnují kvalitu kódu, uživatelskou přívětivost výsledného řešení, časovou náročnost, míru potřebné odbornosti a zkušenost vývojáře.

Metodologie

V rámci zpracování této práce byly využity následující metody:

- Rešerše: autor provedl analýzu odborné literatury zaměřené na problematiku vývoje webových aplikací a umělé inteligence.
- Experiment: autor byl definoval praktický úkol, který byl následně implementován pomocí dvou odlišných přístupů k vývoji – vývoje typu self-code a vývoje pomocí systémů založených na umělé inteligence.
- Komparace: Dva odlišné přístupy k vývoji webových aplikací byly následně porovnány pomocí metody AHP.

1 Literární rešerše

1.1 Webové aplikace

V rámci literárního průzkumu bylo nalezeno několik definic pojmu „webová aplikace“.

Společnost IBM popisuje pojem jako aplikaci zpřístupněnou prostřednictvím webového prohlížeče, která uživateli nabízí funkce přesahující pouhé zobrazení statického obsahu – například umožňuje pracovat s databází prostřednictvím dotazů. Mezi běžné komponenty této aplikace IBM řadí například HTML stránky.¹

Další definice uvádějí:

„Webová aplikace je softwarový program, ke kterému se přistupuje prostřednictvím webového prohlížeče a který poskytuje uživatelům interaktivní funkce.“ (Jain & Dony, 2023, s.2)

„Webová aplikace je podobná desktopové aplikaci, která je dodávána přes internet a spouští se v prohlížeči namísto v hostitelském operačním systému. Od tradičních webových stránek se liší tím, že mají několik úrovní oprávnění, ukládají uživatelské vstupy do databází a často umožňují uživatelům sdílet obsah mezi sebou.“ (Hoffman, 2020, s. 25)

Earle, Eftos a Wheeler charakterizují webové aplikace jako weby, poskytující funkcionalitu odpovídající klasickým aplikacím. (Earle, Castledine et al., 2013)

Z výše uvedených definic vyplývá, že webové aplikace jsou přístupné prostřednictvím internetového připojení a nabízí určité interaktivní funkce, podobné těm, které jsou běžně dostupné v nativních aplikacích.

1.2 Typy webových aplikací

Na základě předchozího vymezení pojmu webová aplikace je zřejmé, že tento pojem zahrnuje různé spektrum řešení lišících se svou strukturou, účelem i technickým provedením. Níže jsou popsány typy webových aplikací, které byly nejčastěji identifikovány v rámci literární rešerše, ačkoliv existují i další typy webových aplikací.

¹ Autor čerpal ze <https://cloud.ibm.com/docs/overview?topic=overview-glossary#x2116500>, rok zveřejnění nebyl uveden.

Statické webové aplikace

Nejjednodušší typ webových aplikací představují statické webové aplikace, které uživatelům poskytují pevný obsah přímo, bez potřeby načítání dat ze serveru. Jsou vytvářeny pomocí základních technologií, jako jsou HTML, CSS a JavaScript, což usnadňuje jejich vývoj i následnou údržbu. Typickým příkladem jsou jednoduché webové aplikace s neměnným obsahem, který se aktualizuje pouze ručně. Jejich zásadní nevýhodou je, že nedokážou efektivně pracovat s dynamickým nebo interaktivním obsahem. Proto se nejlépe hodí pro malé a jednoduché webové projekty. (Rafalski, 2025a)

Dynamické webové aplikace

Dynamické webové aplikace generují a mění obsah podle uživatelských akcí nebo dat z databáze. Na rozdíl od statických stránek obsahují tzv. back-end, který zpracovává uživatelské vstupy (např. přihlášení), komunikuje s databází a sestavuje výslednou stránku. Tyto aplikace podporují interaktivitu, jako je přehrávání videí, personalizace obsahu nebo práce s formuláři. Běžně se používají u spousty moderních služeb. Vývoj dynamických aplikací je však oproti statickým náročnější a vyžaduje víc času, zdrojů i technologií – včetně serverů, databází a API. Na druhou stranu, ale nabízejí pokročilé funkce a umožňují vytvářet celkově modernější a flexibilnější webová řešení. (Tarek, 2023)

Jednostránkové webové aplikace (SPA)

Jednostránkové aplikace jsou takové aplikace, které načítají pouze jednu stránku aplikace a při interakci s uživatelem dynamicky mění obsah stránky bez opětovného načítání z webového serveru. Jejich výhodou je tedy zejména rychlá reakce, plynulé přechody a aktualizace v reálném čase – například při kliknutí na tlačítko nebo odeslání formuláře. Tento typ je často využíván v oblastech jako e-commerce. Jednostránkové webové aplikace využívají jak klientskou, tak serverovou část, často pomocí API a umožňují oddělený vývoj front-endu a back-endu s možností opakovaného využití kódu. (2023)

Vícestránkové webové aplikace (MPA)

MPA jsou webové aplikace, kde obvykle každá změna nebo přechod na jinou stránku vyžaduje nové načtení dat ze serveru. Každá stránka funguje samostatně, což usnadňuje škálování, správu obsahu i optimalizaci pro vyhledávače. Tyto aplikace jsou ideální pro rozsáhlé weby s velkým množstvím obsahu, protože umožňují snadné přidávání nových stránek a poskytují přesnější analytická data. Přestože se dnes často upřednostňují jednostránkové aplikace, vícestránkové řešení nadále představuje osvědčenou volbu pro rozsáhlé a velké webové projekty. (Osypenko, b.r.)

Progresivní webové aplikace (PWA)

Tyto webové aplikace kombinují výhody webových i mobilních aplikací a poskytují rychlý a spolehlivý uživatelský zážitek, za předpokladu kvalitní implementace. Využívají responzivní design, který se přizpůsobí různým velikostem obrazovek. Některé funkce mohou být přístupné i offline, pomocí ukládání obsahu do mezipaměti. PWA jsou navrženy tak, aby základní funkce fungovaly i na starších zařízeních, ale zároveň uměly využít moderní funkce novějších prohlížečů. Tyto aplikace podporuje Google, Microsoft i Apple, protože

představují moderní a efektivní přístup k vývoji multiplatformních webových aplikací. (2022)

1.3 Specifikace požadavků

Sběr požadavků je jednou z nejdůležitějších fází při vytváření jakéhokoliv softwarového projektu. Tento proces pomáhá předejít selhání systémů a vzniku komplikací v závěrečné fázi projektu. Sběr požadavků se někdy také označuje jako specifikace systému. Přístup ke specifikaci se liší v závislosti na konkrétním projektu, nicméně obvykle se rozlišují několik hlavních kategorií požadavků. (Šilhavý et al., 2013)

První skupinou jsou funkční požadavky, které definují, jaké činnosti má daná webová aplikace vykonávat. Specifikace určují, jaké akce má systém provádět v reakci na určité události, jaké operace jsou dostupné uživateli, jaké interaktivní komponenty mají být k dispozici a jak jednotlivé stránky a komponenty jsou propojeny mezi sebou. (Casteleyn et al., 2009)

Další skupinu jsou nefunkční požadavky neboli požadavky na kvalitu. To jsou požadavky, které nepopisují funkce a způsob fungování budoucího systému, ale pomáhají pochopit pravidla, procesy a podmínky řešené domény, na které je třeba se zaměřit v procesu vývoje. (Šilhavý et al., 2013)

Dále se při specifikaci požadavků často určují systémové charakteristiky. Autoři Šilhavý et al. (2013) je popisují jako nežádoucí scénáře chování systému, ke kterým by v žádném případě nemělo dojít.

Poslední kategorií jsou požadavky na omezení. Tyto požadavky zahrnují vnější podmínky, které ovlivňují realizaci a nasazení projektu. Do této skupiny patří například technologická, časová a finanční omezení. (Robertson & Robertson, 1995)

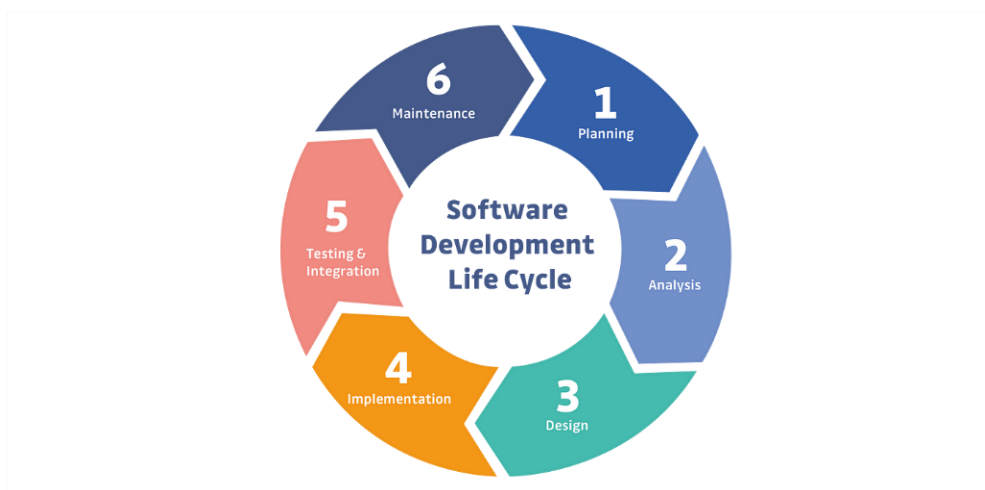
1.4 Životní cyklus vývoje softwaru

Životní cyklus vývoje systému (z angl. the software development lifecycle, dále jen SDLC) je model používaný v oblasti projektového řízení, který definuje jednotlivé fáze, kterými prochází projekt vývoje informačního systému.

Autoři Šilhavý et al. (2013) následovně charakterizují jednotlivé etapy životního cyklu vývoje systému:

- **Plánování:** V této fázi se rozhoduje o vytvoření nového systému a stanovují se jeho hlavní cíle a požadované výstupy.
- **Analýza požadavků:** Cílem této etapy je specifikovat, jaké funkce bude systém nabízet a jaké bude mít kvalitativní charakteristiky.
- **Dělení a výběr:** Během této fáze se systém dělí na logické module.

- Návrh architektury: vytváří se návrh toho, jakou strukturu bude mít výsledná webová aplikace a jak jednotlivé části budou propojeny mezi sebou.
- Rozpracování komponent: V dané fázi se detailně popisuje, jak každá komponenta bude fungovat, jaké bude přijímat vstupy a v jaké formě bude vracet výsledky jejích zpracování.
- Kódování.
- Integrace a testování: V této fázi se komponenty propojí do celku a následně se ověřuje správnost implementace a splnění dříve stanovených požadavků.
- Nasazení a instalace.
- Školení a používání.
- Údržba: Přidávání nových funkcí a řešení technických problémů, které se vyskytují během používání aplikace.



Obrázek 1: životní cyklus vývoje softwaru zdroj:(Aytemiz, 2023)

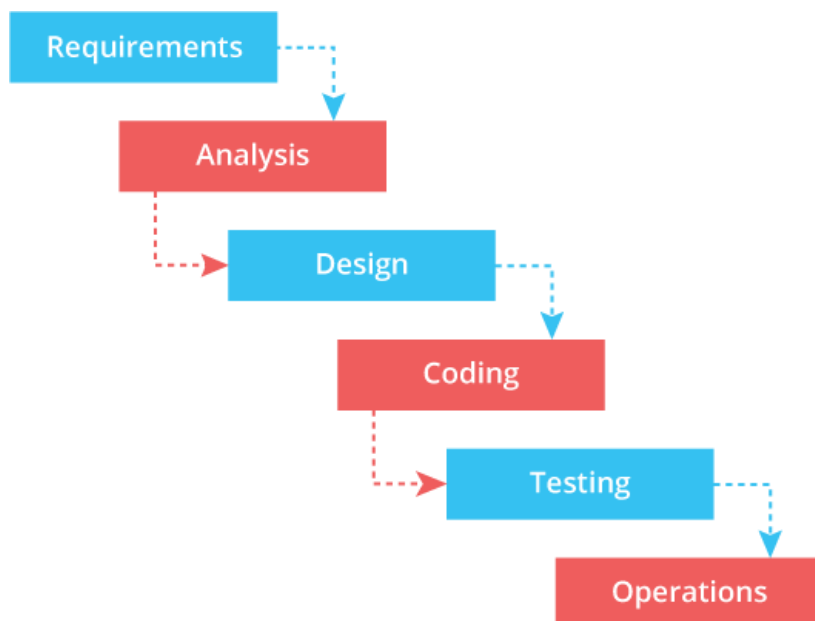
1.5 Implementační procesy

Vodopádový model

Vodopádový model se rozšířil v 70. a 80. letech 20. století a je jedním z prvních modelů životního cyklu určených pro softwarový vývoj. Model zahrnuje následující fáze: specifikace požadavků, analýza, návrh, implementace, testování a zavedení. Specifika tohoto přístupu spočívá v tom, že fáze vývoje probíhají postupně jedna po druhé a každá jednotlivá fáze se provádí pouze jednou během životního cyklu projektu. (Bruckner et al., 2012)

V současné době je tento model často kritizován. Hlavním důvodem je to, že na začátku projektu může být obtížné správně určit všechny požadavky a strukturu systému. Před dokončením projektu se tak mohou objevit problémy, které budou vyžadovat návrat k předchozím fázím a přepracování již implementovaných částí, což vede k posunutí termínu dodání projektu. Za nevýhodu přístupu se také považuje to, že model nepočítá se zpětnou vazbu od klienta v průběhu vývoje a konzultace probíhají převážně na začátku a na

konci projektu. Navzdory tomu se tento model stále používá a může přinášet dobré výsledky v případě malých projektů a projektů, u nichž se požadavky během vývoje nemění. (Bruckner et al., 2012)



Obrázek 2: vodopádový model zdroj: (*What Is the Waterfall Model in Software Engineering?*, 2014)

V-model

Tento model je rozšířenou verzí vodopádového modelu. Model zahrnuje stejné fáze, ale po každé etapě projektu se provádí její testování, což umožňuje identifikovat případné problémové oblasti již v průběhu vývoje. (Laplante, 2007)

Spirálový model

Při daném přístupu vývoj probíhá v několika spirálách, každá představuje jednu iteraci zahrnující specifikaci požadavků, přípravu návrhu, kódování a testování. (O'Regan, 2008) Během každé iterace dochází k identifikaci a analýze rizik a následnému vytvoření prototypu. Ve výsledku každé spirály tak vzniká prototyp, na základě kterého zákazník může poskytnout zpětnou vazbu vývojářům. Iterace se provádí, dokud systém dokončený. (Laplante, 2007)

Iterativní model

Iterativní vývoj znamená, že se vývojový cyklus opakuje – každá iterace přináší vylepšenou verzi produktu. Cílem je postupně zlepšovat funkčnost i kvalitu softwaru na základě zpětné vazby, testování a učení se z předchozích kroků. Díky opakování a vyhodnocování může tým rychle reagovat na chyby nebo změny požadavků. (Gurnov, 2025)

Inkrementální model

Inkrementální vývoj znamená, že systém se buduje po částech – jednotlivé části se postupně přidávají, čímž se budovaný systém zvětšuje a získává nové funkce. Každá část navazuje na předchozí a přináší konkrétní hodnotu. (Gurnov, 2025)

Agilní model

Model představuje iterativní přístup k vývoji softwaru, jenž klade důraz na úzkou spolupráci mezi členy týmu, pravidelnou zpětnou vazbu od zákazníků a rychlé dodávání funkčních částí systému. Vývojový proces je v rámci agilního přístupu rozdělen do krátkých opakujících se cyklů, které zpravidla zahrnují fáze plánování, implementace, testování a následného zhodnocení dosažených výsledků (Asana, 2025). Mezi nejčastěji využívané agilní metodiky patří například Scrum, Kanban či extrémní programování (XP).

1.6 Trendy v moderních webových aplikacích

Pro poskytování kvalitních zákaznických služeb pro podniky a vývojáře je nezbytné držet krok s novými trendy v oblasti vývoje. Cílem této kapitoly je představit vybrané technologické přístupy, které se v posledních letech staly součástí běžné praxe při vývoji webových aplikací.

Personalizace obsahu pomocí umělé inteligence

V posledních letech používání strojového učení výrazně mění přístup, kterým podniky přistupují k personalizaci uživatelského zážitku. Dříve byla personalizace obsahu implementována hlavně na základě plavidlových a segmentačních technologií, které však oproti systémům umělé inteligence poskytují menší flexibilitu a přesnost. Mezi hlavní metody pro doporučení obsahu založené na strojovém učení patří kolaborativní filtrování, filtrování založené na obsahu a hybridní filtrování. (Cherukuri, 2024)

Kolaborativní filtrování je založeno především na interakcích uživatelů se systémem. Metoda umožňuje buď identifikovat uživatele s podobným chováním a následně jim doporučovat obdobný obsah, nebo najít podobnosti mezi jednotlivými položkami, se kterými uživatelé interagovali, a na jejich základě nabízet obsah odpovídající předchozím preferencím. (Cherukuri, 2024)

Filtrování na základě obsahu, na rozdíl od kolaborativního filtrování, se zaměřuje pouze na vlastnosti každého příspěvku a nezohledňuje chování ostatních uživatelů. Mechanismus analyzuje klíčová slova, kategorie a metadata položek, které uživatel vybral v minulosti, a navrhuje obsah s podobným popisem. (Cherukuri, 2024)

Hybridní filtrování kombinuje v sobě prvky dvou předchozích metod, čímž umožňuje překonat nedostatky spojené s každým z nich. (Cherukuri, 2024)

Mobile-First Design

V posledních 10 letech výrazně vzrostl podíl přístupů na webové stránky pomocí mobilních zařízení. Podle statistiky (Ceci, 2025) v roce 2015 mobilní zařízení tvořila 31 % celkové webové aktivity a v roce 2024 tento podíl vzrostl na 62 %. (Rafalski, 2025b)

V souvislosti s tímto trendem vznikl tzv. Mobile-first Design, což je přístup k návrhu webových aplikací, který klade důraz na optimalizaci rozhraní pro mobilní zařízení. Při

tomto přístupu se vývoj aplikace často nejdřív dělá pro nejmenší obrazovky a následně se mění pro ostatní typy rozlišení. Rozdíl mezi návrhem zaměřeným na mobilní zařízení a responzivním designem spočívá v tom, že se Mobile-first design nejen přizpůsobuje rozhraní pro chytré mobily a tablety, ale také se cíleně snaží vylepšit zážitek pro uživatele mobilních zařízení. (Tomasis, 2025)

Progresivní webové aplikace

Progresivní webové aplikace (z angl. progressive web applications, dále jen PWA) představují přístup k vývoji aplikací, který spojuje výhody nativních aplikací a tradičních webových aplikací. PWA jsou přístupné prostřednictvím webového prohlížeče, ale zároveň umožňují instalaci na zařízení a následné spouštění aplikace bez aktivního připojení k internetu. (2025)

PWA nabízejí některé funkce typické pro nativní aplikace jako je automatická synchronizace dat na pozadí, přijímání oznámení ze serveru, zobrazení aplikace v režimu celé obrazovky a využití vybraných funkcí zařízení. (2025)

Headless CMS Architecture

Headless CMS jsou moderní systémy pro správu digitálního obsahu. Jejich hlavním principem je oddělení frontendu, který se používá pro prezentaci dat, od backendu, kde se data ukládají. Komunikace mezi těmito částmi probíhá prostřednictvím API. (Teselko, 2025)

Výhodou tohoto přístupu je nezávislost frontendu na serverové části, kde je uložen obsah. Díky tomu vývojáři mohou implementovat uživatelská rozhraní pro různé platformy, jako jsou například webové aplikace, mobilní aplikace nebo zařízení IoT, a to pomocí libovolných technologií. Obsah, jako jsou články, fotografie a jiné multimediální komponenty, lze následně prezentovat prostřednictvím všech uvedených platform bez nutnosti duplikace. (Teselko, 2025)

Bezserverová architektura

V posledních letech vzrostla popularita aplikací s tzv. bezserverovou architekturou (z angl. serverless architecture). Jedná se o přístup, při němž vývojáři nejsou odpovědní za implementaci backendové části aplikace. Namísto toho provoz serverů zajišťují poskytovatelé cloudových služeb. Daný model umožňuje vývojářům soustředit se na tvorbu uživatelského rozhraní a implementaci obchodní logiky, zatímco cloudová platforma poskytuje nezbytnou infrastrukturu pro spuštění kódu a stará se o údržbu, úpravy a škálování. V okamžiku, kdy vzniká požadavek na spuštění části kódu, cloudový poskytovatel přiděluje potřebné výpočetní prostředky a po ukončení operace tyto prostředky znovu uvolňuje. (Smalley & Susnjara, 2024)

V současnosti nabízejí služby bezserverových platform všichni velcí poskytovatelé cloudových služeb, jako jsou Amazon Web Services, Microsoft Azure, Google Cloud a IBM Cloud. (Smalley & Susnjara, 2024)

1.7 Využité technologie

Kapitola se věnuje technologiím, které byly použity k implementaci řešení bez využití umělé inteligence.

1.7.1 Javascript

JavaScript je skriptovací jazyk určený především pro práci s webovým prohlížečem. Jazyk vznikl v roce 1995 za účelem validace vstupních dat na straně klienta, jelikož do té doby tyto kontroly vždy musely být prováděny na straně serveru, obvykle pomocí programovacích jazyků, jako byl například Perl. V roce 1997 Evropskou asociací výrobců počítačů (Ecma) byla vydána specifikace ECMA-262, která popisovala nový skriptovací jazyk ECMAScript. Jazyk byl schválen mezinárodní organizací pro normalizaci (ISO) a Mezinárodní elektrotechnickou komisí (IEC) a od té doby ECMAScript slouží jako základ pro implementaci JavaScriptu v jednotlivých webových prohlížečích. (Frisbie, 2023)

JavaScript se skládá ze tří hlavních komponent:

- jádro (ECMAScript)
- objektový model dokumentu (z angl. Document Object Model, dále jen DOM)
- objektový model prohlížeče (z angl. Browser Object Model, dále jen BOM)

ECMAScript je základem, na kterém jsou postaveny různé skriptovací jazyky, včetně JavaScriptu. Norma ECMA-262 specifikuje pouze základní prvky jazyka, jako jsou syntaxe, datové typy, operátory, příkazy a globální objekty. Je důležité zmínit, že ECMAScript není přímo spojen s webovými prohlížeči. Prohlížeč slouží pouze jako hostitelské prostředí, které poskytuje implementaci jazyka a doplňuje ji o vlastní funkcionalitu. Dalšími příklady hostitelských prostředí jsou Adobe Flash nebo Node.js. (Frisbie, 2023)

DOM je programové rozhraní, které popisuje strukturu webové stránky ve formě objektového stromu. Tento model je následně využíván JavaScriptem pro přístup k obsahu a prvkům na stránce. Díky DOM mohou vývojáři měnit strukturu dokumentu, upravovat jeho styly a reagovat na uživatelské interakce. (Rascia, 2020)

BOM je soubor rozšíření skriptovacích jazyků, zejména jazyka JavaScript, který umožňuje přístup k vlastnostem a funkcím prostředí webového prohlížeče. Na rozdíl od DOM, který manipuluje s obsahem zobrazené webové stránky, BOM se zaměřuje na prvky, které nejsou součástí obsahu, ale souvisejí s prostředím, ve kterém se stránka zobrazuje. Níže jsou uvedeny příklady některých nejčastěji používaných prvků BOM:

- Možnost otevírat nové okno v prohlížeči.
- Objekt Screen, poskytující informaci o rozlišení obrazovky.
- Objekt Navigátor, obsahující informace o prohlížeči, včetně verze operačního systému, nastavení prohlížeče, aktuální polohy uživatele a dalších technických informací.

- Podpora souborů cookie. (Frisbie, 2023)

JavaScript je známý především jako nástroj pro webový vývoj na straně klienta, ale díky různým frameworkům a knihovnám jazyk může být použit i na straně serveru. Kromě webových aplikací se pomocí něj vytváří mobilní aplikace, hry a v posledních letech i modely strojového učení. (O'Grady, 2020)

1.7.2 Node.js

Node.js je open-source serverové běhové prostředí určené pro jazyk JavaScript. Pro zpracování kódu se v Node.js využívá engine V8, který byl původně vyvinut společností Google pro rychlé a efektivní vykonávání JavaScriptu v prohlížeči Google Chrome. (Buna, 2025)

Hlavní vlastností Node.js je asynchronní programování. Node.js využívá speciální tzv. neblokující model řízený událostmi (event-driven non-blocking model), který umožňuje vykonávat více úloh současně. Moderní weby často provádějí časově náročné operace, jako je nahrávání souborů, komunikace s databázemi a odesílání dat pomocí HTTP požadavků. Zatímco v synchronním prostředí by takové požadavky mohly zablokovat hlavní vlákno a zpomalit systém, Node nečeká na jejich dokončení a zpracovává další části kódu. (Parody, 2025)

Tento přístup je realizován pomocí tzv. event loop. Jedná se o mechanismus, který při spuštění úkolu přidá ho na pozadí jinému systému, který úkol zpracuje a oznámí jeho dokončení. Po dokončení úlohy je vyvolán tzv. callback, což je příslušná funkce, která se provede jako reakce na dokončenou operaci. (Parody, 2025)

Na obrázku je uveden příklad použití asynchronní funkce s callbackem.

```
setTimeout(() => {  
    console.log('This is executed later, after 2 seconds');  
}, 2000);
```

Obrázek 3: ukázka použití asynchronní funkce (zdroj: Parody, 2025)

Vznik Node.js byl významným pokrokem ve vývoji webových aplikací, jelikož rozhraní umožnilo vývojářům psát v jazyce JavaScript jak klientskou, tak i serverovou část aplikace.

1.7.3 React.js

React.js je JavaScriptová open-source knihovna určená pro tvorbu uživatelských rozhraní, která byla vyvinuta společností Facebook v roce 2011. (Kumar, 2024)

Jednou z hlavních vlastností Reactu je komponentová architektura. React umožňuje vytvářet samostatné opakovaně použitelné objekty, které lze poté využít v jakékoli části aplikace nebo přenášet mezi různými projekty. Každá komponenta v Reactu může samostatně ukládat a spravovat data a mít vlastní logiku. (2024)

Další užitečnou funkcionalitou Reactu je tzv. virtuální DOM (virtual DOM). Manipulace s aktuálním DOM může být pro systém poměrně náročná, proto React nejprve ukládá veškeré změny do virtuálního DOM. Následně se dvě verze virtuálního DOM porovnávají a vybírají se pouze části, které musí být aktualizovány. Tento princip umožňuje předávat do DOM pouze nezbytné změny, což zrychluje načítání dat a snižuje zatížení systému. (Hutsulyak, 2024)

Za důležitou vlastnost Reactu považuje také rozšíření syntaxe JavaScriptu (dále jen JSX), které umožňuje psát kód se strukturou podobnou HTML. Mezi výhody použití JSX patří snadnější čtení a psaní kódu, podpora komponentové architektury, a zvýšená bezpečnost, jelikož při kompilaci JSX do JavaScriptu dochází k automatické náhradě potenciálně nebezpečných znaků bezpečnými ekvivalenty. (Kumar, 2024)

1.7.4 Next.js

Next.js je framework založený na Reactu, navržený pro vývoj webových aplikací. Framework byl vytvořen s cílem zjednodušit složitější úlohy, které se běžně vyskytují při práci s čistým Reactem, jako například nastavení nástrojů pro zpracování kódu, použití externích knihoven pro směrování a manuální nastavení server-side renderingu. (Rodrigues, 2025)

Next.js nabízí řadu užitečných funkcí, které z něj činí efektivní nástroj pro vývoj webových aplikací:

Generování statických stránek

Tato funkcionalita umožňuje připravit veškerý obsah statických stránek již při sestavování webu. Kopie výsledných souborů se poté ukládají na speciální servery (CDN) po celém světě. Vzhledem k tomu, že servery jsou umístěny na různých místech, při požadavku ze strany klienta se obsah odesílá z nejbližšího dostupného serveru. Tento přístup zvyšuje rychlost načítání stránek a umožňuje webu snadno zvládat velký počet návštěvníků. Kromě toho jsou takové stránky méně zranitelné vůči útokům, protože nemají vlastní živý server, který by generoval stránku při každém požadavku na zobrazení. (Shelby, 2024)

Generování na straně serveru (SSR)

Serverové vykreslování spočívá v přípravě HTML kódu kompletní webové stránky na serveru a následném odeslání kódu na stranu klienta. Tento přístup zrychluje načítání stránek a pozitivně ovlivňuje výkon webové aplikace. (Minjire, 2024)

Inkrementální statická regenerace (ISR)

Tento přístup kombinuje statické a serverové generování stránek. Umožňuje vytvářet statické stránky, které lze po zveřejnění webu průběžně aktualizovat bez nutnosti kompletního nového sestavení. (Shelby, 2024)

Automatické směrování

Next.js automaticky provádí přepisování v rámci systému díky souborové struktuře, ve které každý soubor ve složce app představuje jednu adresu aplikace.

Automatické rozdělení kódu

Framework zahrnuje automatické rozdělení kódu na malé části a použití pouze nezbytných částí pro načtení stránky. (Rodrigues, 2025)

Optimalizace pro vyhledávače (SEO)

Next.js automaticky generuje důležité SEO prvky, jako jsou meta tagy, nadpisy a popisy jednotlivých stránek. Kromě toho framework vytváří soubor sitemap.xml a robots.txt, které poskytují vyhledávačům strukturu webu a určují, které části aplikace mohou být přístupné pro roboty. (Rodrigues, 2025)

1.7.5 MongoDB

MongoDB je open source nerelační databázový systém, který ukládá data pomocí dokumentů ve formátu binárního JSONu (2021).

Databáze je navržena jako distribuovaný systém, který běží současně na velkém počtu serverů. V běžné konfiguraci jsou kopie jedné databáze zároveň replikovány na třech uzlech. Pomocí speciálního algoritmu je jeden z uzlů vybrán jako primární a data se nejprve zapisují na něj. V případě selhání daného uzlu je automaticky vybrán jako primární jiný server. Tento systém pomáhá chránit data v případě možných výpadků a zaručuje jejich stálou dostupnost. (Aleksendrić et al., 2024)

MongoDB je NoSQL databáze a na rozdíl od relačních databází nevyžaduje pevnou strukturu dat. Záznamy mohou obsahovat jiná pole, což může být výhodné při práci s nestrukturovanými nebo polostrukturovanými informacemi a v situacích, kdy není možné předem přesně určit strukturu dat. (Aleksendrić et al., 2024)

1.8 Úvod do AI

Konfederace laboratoří pro výzkum umělé inteligence v Evropě (CAIRNE, z angl. Confederation of Laboratories for Artificial Intelligence Research in Europe) popisuje umělou inteligenci následně:

„Umělá inteligence zahrnuje algoritmy a systémy, které mohou napodobovat, podporovat nebo překonávat lidské vnímání, jazykové a rozumové procesy, učit se, vyvozovat závěry a předpovídat na základě velkého množství dat, napodobovat nebo zlepšovat lidské vnímání, podporovat člověka v diagnostice, plánování, rozvrhování, přidělování zdrojů a

rozhodování a fyzicky a intelektuálně spolupracovat s lidmi a jinými systémy umělé inteligence.“ (Černý et al., 2024)²

Z definice vyplývá, že pod pojmem umělá inteligence se dá rozumět technologie, které napodobují lidské myšlení, vnímání a rozhodování, přičemž své výstupy zakládají na pokročilém zpracování rozsáhlých dat.

Existuje několik hlavních technologií, které patří do oblasti umělé inteligence:

Strojové učení

Strojové učení je oblast umělé inteligence, která umožňuje systémům učit se z dat, rozpoznávat vzorce a zlepšovat své výstupy bez nutnosti konkrétních instrukcí a programování (Geetha & Sendhilkumar, 2023). Klasické neboli „nehluboké“ strojové učení obvykle vyžaduje strukturované vstupy a spoléhá na lidský zásah při výběru charakteristik pomáhající systému porozumět rozdílům ve vstupních datech (IBM, 2023).

Typy strojového učení

Strojové učení se dělí do několika hlavních typů podle způsobu, jakým se modely učí z dat a prostředí:

- **Supervised Learning (učení s učitelem)** - Model se učí z označených dat, kde k jednotlivým vstupům jsou přiřazeny správné výstupy. Tedy pro vstupní data jsou známé hodnoty cílového atributu. Cílem je, aby model na základě těchto dat dokázal přesně předpovídat výsledky u nových vstupů. Používá se například pro klasifikaci (spam / ne-spam) nebo regresi (predikce ceny domu). Používá se například lineární regrese, rozhodovací stromy nebo neuronové sítě.
- **Unsupervised Learning (učení bez učitele)** - Model pracuje s neoznačenými daty, kde nejsou k dispozici správné výstupy. Neboli pro vstupní data nejsou známé hodnoty cílového atributu. Cílem je nalézt skryté vzory, skupiny nebo struktury v datech, například rozdělení zákazníků do segmentů nebo hledání asociací v nákupním košíku. Používají se metody jako clustering nebo detekce anomálií.
- **Reinforcement Learning (zpětnovazební učení)** - Model neboli agent se učí prostřednictvím interakce s prostředím. Na základě akcí získává odměny nebo penalizace a snaží se maximalizovat celkovou odměnu v čase. Učení probíhá metodou pokus-omyl bez předem označených dat. Postupně se agent učí nejlepší strategii k získání co nejvíce odměn. Typické aplikace jsou robotika, autonomní řízení nebo hry.
- **Semi-Supervised Learning (částečné učení s učitelem)** - Kombinuje označená i neoznačená data. Využívá malé množství označených dat spolu s velkým množstvím neoznačených pro dosažení lepší přesnosti, než by bylo možné pouze s

² Autor použil překlad z knihy (Černý et al., 2024), originální znění definice je zveřejněno na webu organizace CAIRNE.

neoznačenými daty. Používá se v případech, kdy označení dat je nákladné nebo časově náročné. (2025)

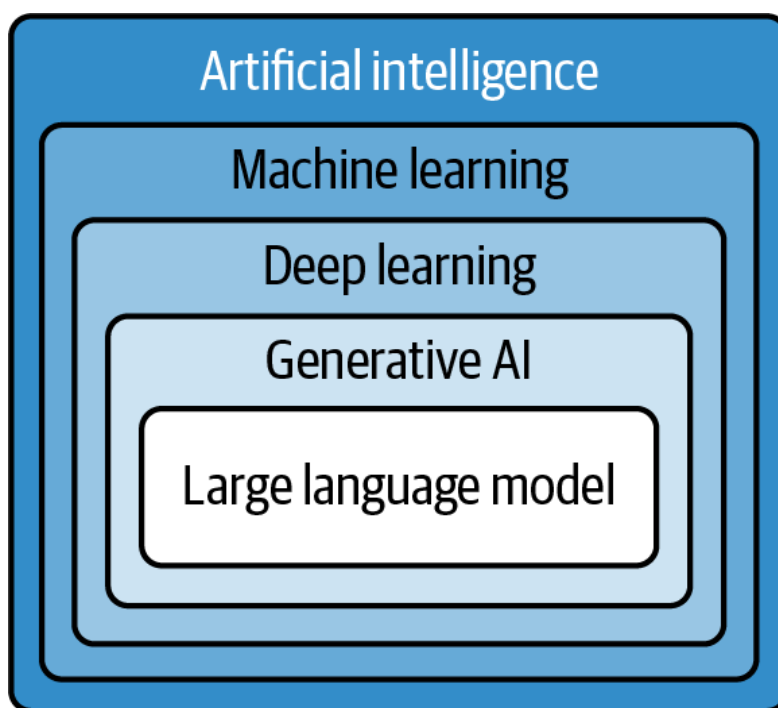
Hluboké učení

Hluboké učení je užší odvětví strojového učení, které využívá neuronové sítě s několika vrstvami a zaměřuje se primárně na nestrukturovaná data jako jsou texty a obrázky (Stöffelbauer, 2023). Oproti strojovému učení automatizuje proces extrakce příznaků a umožňuje práci s velmi rozsáhlými datovými sadami. (IBM, 2023)

Generativní umělá inteligence

Generativní umělá inteligence označuje specifickou skupinu algoritmů v oblasti hlubokého učení, které se na základě analýzy velkého množství dat učí vytvářet nový obsah. Výstupy mohou obsahovat text, fotografie, videa a programovací kód. Na rozdíl od klasických systémů hlubokého učení, které pouze vytvářejí předpovědi na základě poskytnutých dat, generativní inteligence generuje zcela nové výstupy, které v trénovacích datech neexistovaly. (Prabhu, 2025)

Různé typy AI lze představit jako hierarchicky uspořádané podskupiny, jak je znázorněno na následujícím schématu:



Obrázek 4: hierarchicky znázorněné různé typy umělé inteligence (zdroj: Taulli, 2024)

1.9 Úvod do LLM

Tato kapitola se věnuje velkým jazykovým modelům (z angl. large language models, dále jen LLM) – jejich historii, principům fungování a možnostem praktického využití při vývoji softwaru.

1.9.1 Definice LLM

LLM jsou podskupinou generativní umělé inteligence, které se specializují na řešení úloh v oblasti zpracování přirozeného jazyka. Tyto modely jsou založeny na velmi rozsáhlých neuronových sítích a jsou trénovány na velkém množství textových dat s cílem porozumět lidskému jazyku a plnit jazykové a související úkoly. (Campesato, 2024)

1.9.2 Historie LLM

Aktivní rozvoj generativní umělé inteligence začal teprve v posledních patnácti letech, kdy to umožnil vyšší výpočetní výkon techniky a lepší dostupnost dat.

Variabilní autoenkodéry (VAE)

Důležitým krokem na cestě vývoje LLM byl speciální typ umělé inteligence VAE, který byl objeven v roce 2013. Technologie umožňuje převádět velká data, jako jsou například obrázky, do menší verze, která je snadnější pro zpracování. VAE se skládají ze dvou částí – enkodéru a dekodéru. Enkodér převádí data do menšího formátu – tzv. latentního prostoru a dekodér z této zjednodušené podoby znovu sestavuje obrázek nebo jiná složitá data. Od klasických autoenkodérů se VAE liší tím, že nepřidávají pro každý prvek jedno číslo, ale dělají pravděpodobnostní rozdělení, což umožňuje generování nových výstupů, podobných původním datům. (Taulli, 2024)

Generativní adversariální sítě (GANs)

Dalším velkým pokrokem se staly Generativní adversariální sítě (GANs). Technologie je založena na dvou neuronových sítích – generátoru a diskriminátoru, které mezi sebou „soutěží“. Úkolem generátoru je vytvářet nová data a diskriminátor je naprogramován tak, aby se snažil je odlišit od skutečných dat. Ve výsledku tohoto procesu, model se učí vytvářet stále realističtější výstupy. (Taulli, 2024)

GANs se staly základem pro další pokročilé technologie, včetně transformer modelů.

Model sequence-to-sequence

Sequence-to-sequence (dále jen Seq2seq) byl vyvinut společností Google a byl jedním z nejvýkonnějších jazykových modelů v roce 2014. Model byla rekurentní neuronová síť, která zpracovávala text postupně, vždy jeden token najednou, a opakovaně aktualizovala vnitřní paměť. Možnosti této sítě umožňovaly překládat texty, vytvářet shrnutí a hledat klíčová slova. Architektura seq2seq se skládala ze dvou částí – enkodéru a dekodéru. Enkodér přijímal vstupní text a vytvářel myšlenkový vektor (thought vector), který větu shrnul do číselného záznamu. Dekodér z daného záznamu následně vytvářel výsledný text. Omezení modelu spočívalo v tom, že myšlenkový vektor měl pevnou velikost, která mohla být pro zpracování dlouhých vět nedostatečná, což vedlo ke ztrátě důležitých informací. (Berryman & Ziegler, 2024)

Model Transformer

Model Transformer byl vyvinut společností Google v roce 2017 a stal se revolučním pokrokem v jazykových modelech. Na rozdíl od rekurentních neuronových sítí, které zpracovávaly text postupně po jednom slově, Transformer umožňuje paralelní zpracování celého vstupu najednou. Hlavní inovací Transformeru je mechanismus s názvem pozornost (attention), který modelu pomáhá určit, jaká slova jsou v daném kontextu nejdůležitější. Díky tomu model lépe rozumí významům slov a udržuje kontext. (Taulli, 2024)

Objevení modelu Transformer umožnilo vznik velkých jazykových modelů (LLM) včetně modelu GPT.

1.9.3 Generative Pre-trained Transformer

Generative pre-trained Transformers představují skupinu velkých jazykových modelů, které jsou založeny na architektuře hlubokého učení Transformer. Tyto základní modely byly vyvinuty společností OpenAI a současně tvoří základy pro systémy jako je ChatGPT a další generativní nástroje umělé inteligence schopné generovat texty podobné těm, které produkuje člověk. (Belcic & Stryker, 2024)

Název modelu Generative Pre-trained Transformer naznačuje, že se jedná o model, který vytváří text pomocí predikce následujícího slova, je založen na modelu Transformer a je trénován na velkém množství dat. (Stöffelbauer, 2023)

Struktura modelů Transformer

Transformer modely jsou typ neuronových sítí, které jsou určeny pro práci s přirozeným jazykem. Tyto modely pracují se vstupním textem tak, že jej rozdělují na menší tokeny, přičemž jedno slovo může být rozděleno i do několika tokenů. Díky tomu, že zpracovává všechny tokeny najednou, je schopen lépe zachytit dlouhodobé závislosti, což jsou vztahy mezi slovy, která se v textu mohou nacházet daleko od sebe. (Belcic & Stryker, 2024)

Architektura transformer modelů se skládá ze dvou komponent: enkodérů a dekodérů. Obě části využívají mechanismus sebevnímání (z angl. self-attention), který umožňuje modelu zpracovávat celou vstupní sekvenci najednou a identifikovat tokeny, které jsou v daném kontextu nejdůležitější. Tato technologie je klíčovým prvkem transformátorů. (Belcic & Stryker, 2024)

Enkodéry provádí převod jednotlivých tokenů do vektoru v prostoru. Tokeny, které jsou umístěny blízko sebe ve 3D prostoru se považují za významově podobné. Následně enkodérové bloky přidělují jednotlivým vektorům váhy, které vyjadřují relativní význam slova v rámci celého vstupu a pomáhají modelu se zaměřit na důležité části textu. Kromě toho enkodér také využívá mechanismus pozicového kodování (z angl. positional encoding), který zjišťuje pořadí tokenů ve větě. (Belcic & Stryker, 2024)

Dekodér poté vytváří nový obsah na základě informací z enkodérů.

Trénování modelu

Důležitou fází ve vývoji velkého jazykového modelu je fáze předtrénování (pre-training), během níž se model učí předvídat následující slovo na základě velkého množství neoznačených textových dat. K trénování se využívají různé typy textu, jako jsou knihy, články, formuláře, příspěvky na sociálních sítích nebo programovací kód. (Stöffelbauer, 2023) Předtrénování probíhá formou neřízeného učení, jelikož model dostává neoznačená data a snaží se v nich samostatně rozpoznat vzorce a souvislosti (Belcic & Stryker, 2024).

V průběhu této fáze LLM osvojuje jazyková pravidla a obecné znalosti o světě. Natrénovaný model je schopen generovat rozsáhlé texty na různé téma. Je důležité však zmínit, že v této fázi model ještě neumí reagovat na vstupy uživatele, jako jsou otázky a pokyny, protože se interakce typu „pokyn-odpověď“ vyskytují v trénovacích datech pouze minimálně. Aby byl LLM v souladu s lidskými záměry a byl schopen reagovat na konkrétní požadavky, po této fázi následuje etapa „doladění“ modelu. (Stöffelbauer, 2023)

Supervizované doladění instrukcí

Cílem této fáze je naučit model správně reagovat na uživatelské pokyny. Stejně jako při přetrénování se model učí predikovat následující slovo, avšak s rozdílem, že doladění instrukcí probíhá výhradně na základě vysoce kvalitních dvojic instrukcí a odpovědí. Objem těchto dat bývá menší než u přetrénování, jelikož data obvykle připravují odborníci, což dělá proces časově a finančně náročnějším. Ve výsledku této fáze je model schopen nejen generovat souvislé texty, ale také odpovídat na otázky v souladu se záměry uživatele, čímž se přibližuje roli inteligentního asistenta. (Stöffelbauer, 2023)

Posilování učení na základě zpětné vazby od člověka (RLHF)

Cílem této fáze je zarovnat odpovědi jazykového modelu s preferencemi uživatelů. Proces probíhá v několika etapách. Prvním krokem je vytvoření modelu odměn, který hodnotí kvalitu a relevanci odpovědi poskytnutých modelem pomocí číselného skóre. (Lee, b.r.)

K vytvoření modelu odměn tým anotátorů nejprve hodnotí různé odpovědi LLM na stejné zadání a seřazuje je podle kvality. Na základě seřazených dat se model odměn učí samostatně přiřazovat číselný skóre tak, aby reflektoval lidské preference. (Lee, b.r.)

Následně je vytvořena kopie původního modelu, tzv. RL politika, která generuje odpovědi a získává hodnocení z modelu odměn. Na základě této zpětné vazby dochází k opakované optimalizaci parametrů modelu s cílem zvýšení kvality a relevance generovaných výstupů. (Lee, b.r.)

1.10 Vybrané nástroje založené na velkých jazykových modelech

Cílem této kapitoly je popsat dva nástroje postavené na velkých jazykových modelech typu GPT, které byly v rámci práce využity pro vývoj řešení založeného na umělé inteligenci.

1.10.1 ChatGPT

ChatGPT je univerzální konverzační systém založený na AI, který byl vyvinut společností OpenAI. Nástroj je postaven na rodině velkých jazykových modelů GPT a pro zpracování vstupů využívá metody z oblasti zpracování přirozeného jazyka. V době psaní této práce nástroj využívá nejnověji vydaný model – GPT-4o, který kombinuje možnosti generování textu, obrazu a zvuku. (Belcic & Stryker, 2025)

Funkcionalita

Podle Belcic a Stryker (2025) ChatGPT je multifunkční jazykový model, který nabízí široké spektrum využití:

- Tvorba textového obsahu.
- Zpracování velkých textových dokumentů a poskytování shrnutí.
- Překlady textu.
- Vyhledávání informací na webu.
- Psaní a ladění kódu.
- Generování obrázků.
- Hlasové interakce.

Licenční model

Služba ChatGPT je nabízena ve vícero verzích, které se liší úrovní předplatného a rozsahem nabízených funkcí:

Základní verze: při generování odpovědí se využívá model GPT-4.1mini, existuje možnost omezeného přístupu k modelům GPT-4o a OpenAI o4-mini. Verze je dostupná zdarma a nabízí základní funkce pro generování textu.³

ChatGPT Plus: uživatelům se poskytuje přístup k modelům OpenAI o3, o4-mini a o4-mini-high a k testovacím verzím modelů GPT-4.1 a GPT-4.5. Verze zahrnuje doplňkové funkce, jako je „hloubkový výzkum“ (z angl. deep research), hlasový režim a omezený přístup k nástroji Sora, který se využívá ke generování videí. (Belcic & Stryker, 2025) Součástí verze ChatGPT Plus je také plug-in služba, která umožňuje propojení modelu s externími aplikacemi vyvinutými třetími stranami pro rozšíření funkcionality nástroje. (Caelen & Blete, 2024)

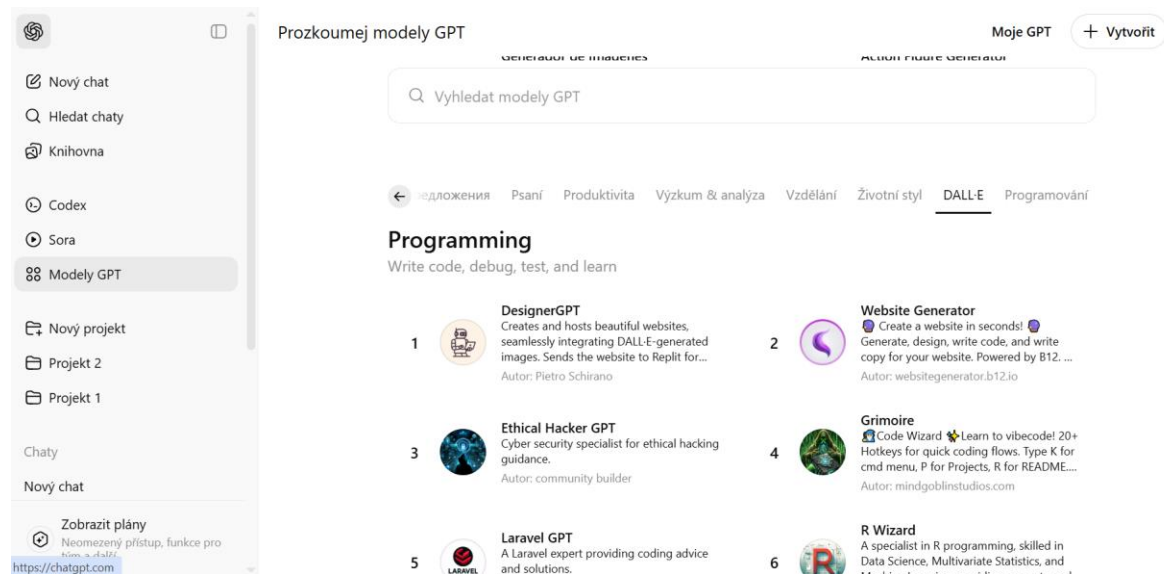
ChatGPT Pro: verze nabízí neomezený přístup ke všem modelům včetně GPT-4o a rozšířené limity funkcí dostupných v předchozích verzích. Verze ChatGPT Pro zahrnuje také model o3-pro, který využívá vyšší výpočetní kapacitu pro řešení složitých dotazů. Součástí této

³ Autor čerpal ze zdroje <https://openai.com/chatgpt/pricing/>, rok zveřejnění nebyl uveden.

verze je také přístup k testování nových funkcí nástroje, jako jsou verze Operator a Codex agent.⁴

Kromě uvedených plánů existují také speciální varianty určené pro týmy, společnosti a akademické instituce. (Belcic & Stryker, 2025)

Zmiňovaná knihovna vlastních modelů GPT, která je dostupná uživatelům verze ChatGPT Plus, obsahuje rozsáhlou sadu plug-in modulů, které byly speciálně navrženy pro podporu programování. Tyto modely plní úlohy, jako je generování a kontrola kódu, vylepšení SEO nebo tvorba datových vizualizací. (Ahmed, 2024)



Obrázek 5: uživatelské rozhraní aplikace ChatGPT pro výběr specializovaných GPT modelů určených k asistenci při programování (zdroj: autor)

1.10.2 GitHub Copilot

GitHub Copilot je nástroj pro asistované programování, který byl vyvinut společností GitHub ve spolupráci se společností OpenAI v roce 2021. Aplikace je založena na jazykovém modelu Codex, který je přímým potomkem modelu GPT-3. (Dowswell, 2024)

Nástroj je dostupný prostřednictvím webového rozhraní nebo jako rozšíření pro vývojová prostředí. Podle (Dowswell, 2024) integrace s vývojovým prostředím je k dispozici v následujících editorech:

- Visual Studio Code
- Visual Studio
- Azure Data Studio

⁴ Viz poznámku pod čarou č. 1

- JetBrains IDEs
- Neovim a Visual Studio.

Funkcionalita

Na rozdíl od ChatGPT byl GitHub Copilot vytvořen speciálně za účelem asistence vývojářům při psaní kódu, proto je jeho funkčnost přímo spojena s programováním. Podle (Dowswell, 2024) nástroj nabízí následující funkce:

- doplnění kódu v reálném čase
- generování kódu na základě uživatelských pokynů
- odpovídání na otázky, vysvětlení technologií a částí kódu
- návrh alternativních přístupů pro již existující kód
- generování testů
- generování dokumentace.

Licenční model

Nástroj nabízí několik verzí v závislosti na potřebách uživatelů a na tom, zda se jedná o jednotlivce nebo organizaci⁵:

- GitHub Copilot Free: základní plán, který je zdarma. Verze nabízí omezený přístup k vybraným funkcím aplikace.
- GitHub Copilot Pro: verze nabízí neomezený počet doplnění kódu, přístup k prémiovým modelům v rámci Copilot chatu a určitý počet prémiových požadavků.
- GitHub Copilot Pro+: kromě funkcí dostupných v předchozích verzích poskytuje plný přístup ke všem modelům v Copilot chat, funkce speciálního kódovacího agenta a větší limit prémiových požadavků.

Stejně jako ChatGPT Copilot nabízí plány určené pro společnosti – GitHub Copilot Business a GitHub Copilot Enterprise, které poskytují možnost centralizovaně spravovat nastavení a pravidla používání nástroje Copilot v rámci organizací.⁶

1.10.3 Cursor

Cursor využívá umělou inteligenci pro podporu programování a zefektivnění vývoje softwaru. Je založený na klasickém editoru Visual Studio Code, avšak obohacuje jej o AI funkce na bázi velkých jazykových modelů. Cursor je navržen tak, aby programátorům pomáhal v reálném čase s analýzou, návrhem, psaním i laděním kódu jako je například rychlá oprava chybného kódu. (Eleks, 2025)

⁵ Autor čerpal ze zdroje <https://docs.github.com/en/copilot/about-github-copilot/plans-for-github-copilot/>, rok zveřejnění nebyl uveden.

⁶ Viz poznámku pod čarou č. 3

Funkcionalita

Podobně jako GitHub Copilot byl nástroj Cursor navržen jako AI asistent pro vývojáře, který zefektivňuje psaní a údržbu kódu. Mezi klíčové funkce nástroje patří například⁷:

- Celková optimalizace kódu.
- Inteligentní automatické doplňování kódu.
- Agent, který dokončí i komplexnější úlohy.
- Automatické opravy a detekce bugů.
- Vysoká kompatibilita s VS Code ekosystémem.

Licenční model

Cursor je nabízen jak pro jednotlivce, tak pro vývojářské týmy. V rámci jednotlivců má cursor tři plány⁸:

- Free: verze, která je zadarmo a zahrnuje omezený počet AI dotazů a omezený přístup k jednotlivým funkčním částem Cursoru.
- Pro: velmi vysoké až neomezené limity pro jednotlivé funkce, pokročilé refaktorační nástroje, přístup k chybovému botu.
- Ultra: až 20 násobně větší limit pro využívání všech podporovaných jazykových modelů, přednostní přístup k novým funkcím.

Podobně jako GitHub Copilot nabízí i Cursor jednotlivé plány pro společnosti.

1.10.4 Tabnine

Tabnine je AI asistent pro vývojáře, který od roku 2018 patří mezi průkopníky generativní AI v oblasti softwarového vývoje. Rozdílem oproti konkurenci může být velká snaha o soukromí. Společnost staví na filozofii „AI, kterou ovládáš ty“ – vývojářům a firmám tak dává plnou kontrolu nad tím, kde a jak je nasazen. Tabnine může běžet jako SaaS, v privátním cloudu nebo on-premise, a nikdy neukládá ani nesdílí žádný kód. Modely jsou trénovány výhradně na kódu s otevřenými licencemi, což eliminuje právní rizika.⁹

⁷ Autor čerpal ze zdroje <https://www.cursor.com/features>, rok zveřejnění nebyl uveden.

⁸ Autor čerpal ze zdroje <https://www.cursor.com/pricing>, rok zveřejnění nebyl uveden.

⁹ Autor čerpal ze zdroje <https://www.tabnine.com/about>, rok zveřejnění nebyl uveden.

Funkcionalita

Funkcionalita Tabnine je velmi podobná ostatním AI asistovaným nástrojům pro kódování, jako jsou GitHub Copilot, avšak Tabnine klade větší důraz na bezpečnost a ochranu soukromí uživatelů. Výhodou je i podpora prostředí IDE.

Licenční model

Tabnine nabízí 3 stupně licencí:

- Basic: Nabízí základní funkce, jako je AI-powered doplňování kódu, generování kódu z přirozeného jazyka, vysvětlení kódu a generování testů. V tomto plánu je však omezený přístup k pokročilým funkcím jako Code Review Agent, implementaci Jira ani soukromé nasazení.
- Dev: Poskytuje plný přístup ke všem AI funkcím, včetně přepínání mezi různými velkými jazykovými modely a automatické implementace a validace úkolů z Jira nebo také podporu na bázi tiketů.
- Enterprise: Umožňuje nasazení na vlastní infrastrukturu nebo v privátním cloudu, nabízí pokročilý Code Review Agent pro kontrolu kvality a bezpečnosti kódu a další pokročilé funkce.¹⁰

1.10.5 Windsurf

Windsurf kombinuje asistenci copilota s autonomním AI agentem Cascade, aby zajistil efektivní a intuitivní vývoj softwaru. Editor je založený na Visual Studio Code a umožňuje automatické generování, refaktoring a opravy kódu, přičemž udržuje kontext celé vývojové relace. Windsurf podporuje integrované workflow pro tvorbu, iteraci a nasazení aplikací a nabízí inteligentní návrhy kódu založené na historii příkazů.¹¹

Funkcionalita

I když jsou funkcionality podobné jako u předchozích, windsurf odlišuje především fungování – kde AI není jen asistent pro doplňování kódu, ale funguje jako autonomní agent schopný samostatně řešit víceukrokové úkoly například kompletní generování funkcí, refaktoring nebo opravy chyb.

Další odlišností je zaměření na workflow – Windsurf nabízí integrované nástroje nejen pro psaní kódu, ale i pro jeho iteraci, testování a nasazení, což zjednodušuje celý životní cyklus vývoje přímo v jednom prostředí.¹²

¹⁰ Autor čerpal ze zdroje <https://www.tabnine.com/pricing>, rok zveřejnění nebyl uveden.

¹¹ Autor čerpal ze zdroje <https://windsurf.com>, rok zveřejnění nebyl uveden.

¹² Viz poznámku pod čarou č. 11

Licenční model

Windsurf nabízí 4 licenční plány:

- Free: základní plán, který je zdarma a nabízí 25 prompt kreditů měsíčně, přístup k AI modelům.
- Pro: 500 prompt kreditů a přístup k lepšímu modelu.
- Teams: přidává mnoho funkcionalit jako je administrátorská správa s dashboardy, priorita v rámci podpory a další zejména firemní funkce.
- Enterprise: nabízí rozšířené možnosti řízení přístupu, větší množství využitelných prompt kreditů, možnost hybridního nasazení a prémiovou podporu.¹³

1.11 Prompt inženýrství

Prompt inženýrství (z angl. prompt engineering) je disciplína strojového učení a přirozeného jazyka, která se zaměřuje na metody navrhování a optimalizace dotazů na umělou inteligenci s cílem umožnit systému rozumět lidské řeči a správně ji interpretovat. (Taulli, 2024) Pojem „prompt“ označuje vstup zadaný do modelu. Může se jednat o text obsahující otázku, pokyny nebo jiný typ požadavku. (Berryman & Ziegler, 2024).

Dvěma základními prvky, které zpravidla obsahuje prompt, jsou instrukce pro model a vložený obsah.

Instrukce určují, jak má model s daným vstupem naložit. Nedoporučuje se používat větší množství instrukcí v rámci jednoho promptu, jelikož to může vést k neefektivnímu zpracování informací – LLM musí zvláště analyzovat každou část dotazu a následně sestavit z výstupů jednu související odpověď. Nejednoznačně formulované nebo vzájemně rozporné pokyny mohou navíc model zmást a vést k nepřesné nebo neúplné odpovědi. (Taulli, 2024)

Vložený obsah může zahrnovat textový nebo datový vstup, s nímž má model dále pracovat. V případě vložení textu se doporučuje používat speciální symboly, jako jsou ### nebo "", které jasně oddělují uživatelské instrukce od obsahu promptu. (Taulli, 2024)

Pro dosažení přesnějších odpovědí lze využít komplexnější dotazy, které dle (Alammar & Grootendorst, 2024) mohou obsahovat následující komponenty:

- Kontext
- Persona: V dané části se uvádí role nebo postava, kterou má model ztvárnit. Příklad uvedení persony: „Jste vývojář s mnohaletými zkušenostmi v oblasti webového vývoje...“ (Taulli, 2024).

¹³ Autor čerpal ze zdroje <https://windsurf.com/pricing>, rok zveřejnění nebyl uveden.

- **Publikum:** Definice cílové skupiny, pro kterou je výstup LLM určen. Toto kritérium má vliv na styl odpovědi, jazykovou úroveň a míru odbornosti.
- **Příklady:** ukázky očekávané odpovědi.
- **Formát:** Očekávaná struktura výstupu, např. tabulka, seznam, programovací kód, graf, markdown nebo multimediální obsah .

Je důležité zdůraznit, že pro získání kvalitní odpovědi prompt nemusí nutně obsahovat všechny zmíněné komponenty. Jejich vhodné použití však zvyšuje pravděpodobnost, že odpověď bude přesně odpovídat požadavkům uživatele.

1.12 Praktické využití LLM ve vývoji

Programování pomocí AI nemusí nutně být samostatnou metodikou vývoje softwarového projektu, týmy jej mohou integrovat do již zavedených agilních nebo DevOps metodik (Osmani, 2025). V praxi se dle (Osmani, 2025) nejčastěji používají tři hlavní modely začlenění AI do vývojového procesu:

- AI vytváří první návrhy kódu, které vývojáři následně upravují, přepracovávají, testují a integrují s jinými částmi.
- AI plní roli pomocného programátora. Vývojář v takovém případě neustále komunikuje s AI: zadává úkoly, kontroluje výstupy a průběžně upravuje na jejich základě kód.
- AI se používá pro validaci kódu. Vývojáři samostatně vytvářejí kód a následně jej pomocí AI kontrolují na chyby a vylepšují.

Proces vývoje moderních softwarových projektů je velmi komplexní, a ne všechny úkoly lze automatizovat pomocí AI. Nicméně existují určité úlohy, které moderní nástroje založené na velkých jazykových modelech zvládají efektivně:

Tvorba nového projektu od nuly

Nástroje AI se dobře hodí pro generování počátečních šablon hlavních částí aplikace, jako jsou uživatelské rozhraní, serverová část, API nebo databázový model. Díky tomu tým může rychle vytvořit prototyp budoucího produktu, který lze použít k ověření vybraného návrhu. (Osmani, 2025)

Generování unit testů

Tvorba jednotkových testů (z angl. unit tests) je důležitá pro kontrolu kvality kódu, ale tento proces může být časově náročný a často spočívá v opakovaném vytváření podobných testovacích případů. AI nástroje jsou schopné na základě zdrojového kódu generovat testovací případy pokrývající různé větve kódu a scénáře. (Whelan, 2025)

Generování opakujících se a standardních částí kódu

AI si dobře poradí s generováním stejných částí kódu, jako jsou například datové modely pro různé typy záznamů nebo řadiče (z angl. controllers). Pro generování podobných souborů stačí ukázat modelu několika příkladů.

Kromě toho se velké jazykové modely dobře zvládají generování standardizovaných komponent, jako jsou funkce CRUD, API koncové body nebo formuláře s validací vstupů. Jelikož tyto komponenty jsou součástí téměř všech moderních webových aplikací, jejich příklady se velmi často vyskytují v trénovacích datech a umožňují modelům je produkovat s vysokou mírou přesnosti. (Osmani, 2025)

Revize kódu

Nástroje založené na LLM jsou schopné automaticky identifikovat syntaktické chyby v kódu a poskytovat zpětnou vazbu. Kromě chyb, AI může také sledovat dodržování v kódu vybraného stylu a formátování, což může být přínosné při týmových projektech, na kterých pracuje více vývojářů. (Kluza & Nowak, 2024)

Generování dokumentace

Nástroje založené na AI umí při každé úpravě kódu automaticky aktualizovat dokumentaci k jednotlivým částem aplikace a jejím rozhraním. Velké jazykové modely lze navíc využít k usnadnění orientace ve velkých softwarových projektech. AI může generovat přehledná shrnutí rozsáhlých částí kódu nebo převádět vývojářské komunikace do textové strukturované podoby, která zachovává důležité body diskuse. (Shrivastava, 2025).

Využití moderních frameworků a integrace služeb

Frameworky a knihovny zahrnují specifické funkce, metody a parametry, takže vývojáři nemusí vždy znát všechny potřebné detaily, zejména při práci s novými technologiemi. Díky tomu, že velké jazykové modely a zejména speciální nástroje pro podporu programování jsou trénovány na velkém množství technické dokumentace a manuálů, mohou vygenerovat šablonu kódu přizpůsobenou konkrétnímu API, jako je například návrh komponenty pracující se stavem v knihovně React. (Osmani, 2025)

Z téhož důvodu LLM lze použít na psaní kódu potřebného pro propojení několika služeb nebo rozhraní API mezi sebou. Příklady takových úloh mohou být například vytvoření správných koncových bodů nebo převod dat do požadovaného formátu. (Osmani, 2025)

Přestože lze AI nástroje efektivně využít v určitých oblastech vývoje, existuje řada úloh, které svou složitostí přesahují současné možnosti LLM:

Velmi komplexní systémy

AI mohou být neúčinné při řešení velmi složitých nebo nových problémů, jako je například vytvoření kompilátoru, návrh vysoce paralelního systému nebo implementace zcela nového algoritmu na základě vědeckého výzkumu. Podobné úkoly vyžadují velmi hluboké pochopení problému a často také nestandardní a kreativní myšlení. V takových případech se LLM může pokusit vygenerovat potenciální řešení, ale s velkou pravděpodobností nebude

takový kód fungovat správně a může zavést vývojáře k neefektivnímu postupu.(Osmani, 2025)

Kreativní UI a UX návrhy

Umělá inteligence je schopná vygenerovat kód na základě už existujících vzorů grafických rozhraní, jako jsou formuláře nebo navigační menu. Model však může být neefektivní v situacích, kdy je třeba vymyslet nový kreativní přístup k uživatelskému rozhraní.

Pro AI bude také obtížným úkolem vytvoření skutečně kvalitního UX návrhu, jelikož daný proces vyžaduje empatii a schopnost dobře porozumět způsobu uvažování a problémům uživatelů, a pouhé využití často používaných principů nemusí být pro konkrétní projekt postačující. (Stevens, 2024)

Vzácné nebo nové technologie

Při použití málo známých frameworků nebo nových technologií, které v době trénování LLM modelu ještě neexistovaly, AI je nemusí znát a může vygenerovat chybný kód, který bude působit důvěryhodně, ale ve skutečnosti bude obsahovat neexistující prvky API. Problém může také nastat v případě aktualizace frameworků, protože model může nabízet řešení vhodné pouze pro staré verze. Problém však lze vyřešit přidáním do promptu příslušné dokumentace, na jejímž základě se model může naučit reprodukovat relevantnější kód. (Osmani, 2025)

Interpretace lidského záměru skrz zadání

AI nemusí vždy správně pochopit důvody a cíle úkolů zadaných uživatelem, zejména pokud jsou instrukce napsány implicitně, nebo řešená doména vyžaduje znalost specifických informací, požadavků nebo pravidel týkajících se dané oblasti. V takovém případě navrhované modelem řešení může být logické a funkční, ale nepomůže s řešením skutečného problému.(Osmani, 2025)

Autor publikace (Osmani, 2025) uvádí, že nástroje založené na AI jsou schopny vytvořit prvotní řešení, které odpovídá přibližně 70% požadovaných kritérií. Zbývajících 30 % tvoří velmi důležité úkoly, které vyžadují lidskou odbornost. Vygenerovaný kód může být technicky funkční a být postačujícím pro základní scénář, ale s velkou pravděpodobností nebude zohledňovat detaily jako jsou neobvykle případy, extrémní provozní podmínky, optimalizaci výkonu, udržitelnost řešení a možnosti budoucího rozšíření. V současném stavu vývoje LLM, jejich použití při kódování může zvýšit produktivitu vývojářů při plnění rutinních činnostech, ale nenabízí spolehlivá řešení komplexnějších problémů, které vyžadují kreativní a analytické myšlení.

1.13 Výhody a omezení využití umělé inteligence v programování

Kapitola se zaměřuje přínosy a výzvy, které využití nástrojů generativní AI přináší do oblasti programování.

Zapojení AI do procesů vývoje softwarového projektu může přinést vývojářům řadu výhod:

Zvýšení produktivity

Využití AI k optimalizaci určitých úkolů obecně vede k navýšení produktivity vývojářů. AI může ušetřit čas strávený hledáním řešení problémů na fórech a v dokumentacích, prováděním opakovacích úkolů a revizí kódu. Ve výsledku tým může využít své kapacity na rozhodování o komplexnějších problémech, jako jsou návrhy udržitelné a výkonné architektury softwaru či zlepšování uživatelské zkušenosti. (Shrivastava, 2025)

Možnosti pro experimentování

Využití AI snižuje časovou a finanční náročnost procesu prototypování, čímž umožňuje týmu více experimentovat. Prostřednictvím různě formulovaných promptů lze během kratší doby vygenerovat a ověřit více možných řešení, z nichž se následně vybere nejvhodnější. (Osmani, 2025)

Dostupnost znalostí

Velké jazykové modely jsou trénovány na obrovském množství programových znalostí, které programátoři potřebují při každodenní práci. AI s velkou pravděpodobností bude znát specifické detaily API nebo řešení určitých chyb a bude moci okamžitě poskytnout odpověď.

Demokratizace softwarového inženýrství

Díky nástrojům pro generování kódu založeným na umělé inteligenci se vývoj softwaru stává dostupnější pro uživatele bez hlubokých technických znalostí nebo zkušeností s programováním. Tak mohou produktové manažeři, obchodní analytici či majitelé malých podniků vytvářet softwarová řešení přizpůsobená jejich konkrétním potřebám. (Finio & Downie, 2024)

Ačkoli nástroje založené na AI mohou zvýšit efektivitu vývojového procesu, jejich implementace je zároveň spojena s určitými omezeními a riziky:

Halucinace

Jedním z velkých problémů spojených s LLM je jev označovaný jako „halucinace“. Jedná se o situace, kdy model generuje nepodložená fakta, která přesto na první pohled působí přesně a důvěryhodně. (Stöffelbauer, 2023) V kontextu generování kódu to vede k vytvoření kódu, který nefunguje správně a obsahuje chyby nebo neexistující prvky API, což značně komplikuje proces vývoje, zejména pro méně zkušené programátory. (Taulli, 2024)

K „halucinacím“ dochází v důsledku toho, že LLM modely negenerují odpovědi na základě skutečného porozumění obsahu, ale na základě pravděpodobnostních vzorců naučených během trénování. (Tam, 2023) . Kromě toho samotná trénovací data mohou obsahovat chyby nebo nepravdivá informace, které model neumí ověřit. (Stöffelbauer, 2023)

V současné době není možné problém halucinací zcela vyřešit, lze mu však částečně předejít uváděním do promptu podrobného kontextu nebo poskytováním přímého textu ze zdrojů, na které se má model opírat při generování odpovědi. (Stöffelbauer, 2023).

Bias v datech

Modely AI mohou přebírat a následně používat při generování výstupů zkreslení, která byla obsažena v trénovacích datech. V kontextu psaní kódu se to může projevit použitím určitých názvů proměnných a funkcí, které nevyjadřují jejich skutečný účel, nebo generováním kódu, vycházejícího z předpokladu určitých nastavení uživatelského prostředí. Tato řešení často vycházejí z četnosti výskytu určitého vzoru v trénovacích datech, ale nemusí představovat nejoptimálnější řešení pro konkrétní projekt. (Osmani, 2025)

Duševní vlastnictví

V souvislosti s využíváním AI k generování obsahu může docházet k vzniku rizik porušení práv třetích stran. Jelikož k trénování modelů se používá data, která pocházejí z veřejně dostupných zdrojů, AI při generování obsahu může využívat části textu chráněných autorským právem. (Taulli, 2024)

Tento problém se týká i programování s podporou AI, protože LLM při tvorbě nového kódu jako základ používá velké množství veřejně dostupného kódu. Mnoho z daných projektů je chráněno různými licencemi typu open-source, jako jsou například GNU GPL, MIT či Apache. Každá z licencí zahrnuje určitá pravidla, například licence GPL vyžaduje, aby byl použitý kód v budoucnu zveřejněn pod stejnou licencí, což nemusí odpovídat účelu projektu a požadavkům společnosti. Rizikem je také to, že nástroje založené na AI neposkytují informace o podkladových materiálech, odkud byl kód převzat, takže uživatel nemusí být informován o použití licencovaných zdrojů. (Osmani, 2025)

Ochrana osobních údajů

Používání nástrojů založených na AI také vyvolává mnoho obav ohledně ochrany osobních údajů uživatelů, zejména v souvislosti s daty předávanými v rámci komunikace s modely. Obavy se týkají způsobu uchování a zabezpečení uživatelských údajů a také rizika jejich následného využití pro účely budoucího trénování modelů. (Taulli, 2024)

V této souvislosti se některé společnosti rozhodují pro vývoj vlastního systému založeného na umělé inteligenci. Tento přístup umožňuje vyhnout se sdílení dat a IP adres třetím

stranám, a zároveň poskytuje možnost přizpůsobit model podle interních potřeb organizace. (Taulli, 2024)

Bezpečnost

Kód napsaný AI může představovat potenciální bezpečnostní rizika, jelikož modely mohou při generování kódu přebírat z trénovacích dat chyby, jako jsou například Cross-Site Scripting (XSS) nebo zranitelnosti vůči SQL injection útokům. Zranitelná místa v kódu mohou také vzniknout v důsledku omezené schopnosti modelu porozumět specifickému bezpečnostnímu kontextu projektu a zaměření na dosažení funkčních požadavků na úkor implementace bezpečnostních opatření. (Kush, 2025)

Využití umělé inteligence v oblasti programování má řadu výhod, jako jsou například zvýšení produktivity a podpora tvůrčího procesu. Je však nezbytné přistupovat k jejímu nasazení s ohledem na možná rizika, a uplatňovat vyvážený přístup, který zahrnuje implementaci kontrolních mechanismů a bezpečnostních opatření.

1.14 Metody multi-kriteriálního hodnocení

1.14.1 Metoda AHP Saatyho

Analytický hierarchický proces (z angl. Analytic Hierarchy Process, dále jen AHP) neboli metoda Saatyho byla vyvinuta Thomasem Saatyem. Metoda slouží k určení relativní váhy několika kritérií nebo několika variant s ohledem na daná kritéria. (Kasperczyk & Knickel, b.r.)

Metodologie AHP

Nejprve je nutné rozdělit řešený problém na jednotlivé části, které lze představit jako prvky hierarchické struktury. Na nejvyšší úrovni takové struktury se nacházejí cíle, kterých chce řešitel dosáhnout. Dále následují kritéria a podkritéria, na jejichž základě se provádí srovnání. Na nejnižší úrovni v dané struktuře se nacházejí jednotlivé možnosti, mezi nimiž se volí. Při definování hierarchie je důležité brát v potaz kontext problému a aspekty, které mohou mít na řešení vliv. (Kasperczyk & Knickel, b.r.)

Dalším krokem je párové srovnání kritérií. Každá dvojice kritérií se porovnává na základě toho, které z nich je pro dosažení hlavního cíle nejdůležitější. Preferovanému kritériu je následně přiděleno hodnocení v rozmezí od 1 do 9, zatímco druhému kritériu bude přidělena opačná hodnota. Pomocí daného přístupu se určuje relativní váha každého kritéria. Následně se provádí normalizace těchto vah a spočítání průměrné hodnoty pro každé kritérium. (Kasperczyk & Knickel, b.r.)

V poslední fázi AHP metody se provádí párové srovnání konkrétních možností. V každé dvojici variant v rámci jednotlivých kritérií, nejlepší variantě je opět přiřazeno hodnocení od 1 do 9 a druhé možnosti se přiděluje opačné hodnocení. Následně se tato skóre opět normalizují a zprůměrují. (Kasperczyk & Knickel, b.r.)

Výhody a omezení metody AHP

Podle (Karthikeyan et al., 2016) za hlavní výhody dané metody se považují:

- Použitelnost.
- Intuitivní a snadno pochopitelný systém.
- Možnost rozložení komplexního problému na jednodušší části.
- Metoda umožňuje pracovat nejen s přesně měřitelnými údaji, ale také se subjektivními odhady.

Autoři (Kasperczyk & Knickel, b.r.) za další výhodu metody AHP považují také podporu skupinového rozhodování, které lze implementovat s využitím geometrického průměru individuálních párových srovnání.

Navzdory přínosům dané metody (Karthikeyan et al., 2016) zdůrazňuje také aspekty, za které je metoda Saatyho kritizována:

- Metoda AHP není vhodná pro situace s vysokou mírou nejistoty, kdy pro řešitele může být náročné jednoznačně číselně vyhodnotit různé možnosti.
- Model je citlivý na změny ve struktuře – v případě přidání nebo odebrání kritérií může být ovlivněno pořadí všech prvků.
- Při použití většího počtu kritérií výrazně stoupá výpočetní a časová náročnost metody.

2 Návrh projektu

2.1 Představení firmy

V rámci dané bakalářské práce byl vytvořen prototyp e-shopu pro malou společnost, která od roku 1993 působí na území České republiky jako specializovaný dodavatel vybavení pro obchody, prodejny, školní jídelny a restaurace.

Na základě individuálních požadavků klientů firma realizuje i rozsáhlé projekty, které zahrnují profesionální poradenství, přípravu 3D návrhu řešení, objednání a dodání zboží, montáž zařízení a případné zaškolení personálu.

Firma spolupracuje především s provozovateli podniků a obchodních řetězců, ale v rámci menších projektů jsou zákazníky i fyzické osoby.

2.2 Současný stav používaného webu

Společnost v současné době již má své webové stránky, které slouží k prezentaci nabídky produktů a služeb firmy. Návštěvníci se mohou seznámit s aktuálním sortimentem prostřednictvím katalogů ve formátu PDF. Na stránkách je také k dispozici formulář pro přímý kontakt se společností. Objednání zboží a dohody ohledně konzultace projektů se v současné době realizují výhradně prostřednictvím e-mailové komunikace.

2.3 Návrh nového řešení

V rámci práce byl implementován prototyp nového řešení, který se skládá ze dvou částí: webových stránek internetového obchodu a administrátorského panelu.

Panel správce bude navržen tak, aby administrátor mohl samostatně přidávat a odstraňovat produkty, spravovat kategorie, sledovat objednávky a poskytovat přístup ostatním zaměstnancům bez nutnosti přistupovat přímo k databázovému rozhraní nebo používat jiné externí webové služby.

Hlavní web bude představovat kombinaci firemní prezentace a e-shopu, který umožní efektivní online nákup nabízených produktů. Web bude obsahovat standardní sekce pro e-shop, jako je stránka s nabídkou produktů, stránka s podrobnostmi o konkrétním produktu, seznam oblíbených produktů a nákupní košík. Dále zákazníci zde také najdou informace o dalších službách společnosti a veškeré důležité kontaktní údaje.

2.4 Stanovení požadavků

2.4.1 Funkční požadavky pro administrátorský panel

Administrátorský panel je webové rozhraní, které umožňuje spravovat e-shop bez nutnosti přímé interakce s databází nebo zdrojovým kódem. Tento nástroj není veřejně dostupný a je určen výhradně pro zaměstnance společnosti, kteří se zabývají správou produktů, kategorií a objednávek. Na základě účelu této aplikace a konzultace s majitelem řešené firmy byly stanoveny následující funkční požadavky:

Funkce autorizace pro uživatele

Přístup ke stránkám administrátorského panelu musí být omezen na uživatele se speciálním přístupem. Informační systém proto musí implementovat funkci autentizace uživatelů pomocí přihlašovacího jména a hesla a poskytovat přístup k dalším stránkám pouze po úspěšném přihlášení do účtu správce.

Funkce pro správu produktů

Vzhledem k tomu, že se sortiment obchodu může v průběhu času měnit, správce musí mít možnost přidávat na web nové produkty, měnit informace o stávajících produktech a také produkty mazat. Při zakládání nového produktu by měla existovat možnost nahrání a mazání souvisejícího multimediálního obsahu.

Funkce pro správu kategorií

Kromě produktů jsou součástí dynamicky spravovaného obsahu e-shopu také kategorie, které obsahují jednotlivé produkty. Administrátorský panel by měl poskytovat nástroje pro jejich přidání, mazání a aktualizaci.

Funkce pro správu objednávek

Pro zajištění plynulého fungování e-shopu je nutné, aby správce systému vždy měl aktuální informace o všech nově přijatých objednávkách.

Funkce pro správu správců

Vzhledem k tomu, že přístup k nástroji je omezen pouze na úzký okruh uživatelů, je nezbytné, aby aplikace umožňovala správu administrátorských účtů. To zahrnuje možnost vytvářet nové účty, upravovat existující záznamy a mazat neaktuální účty.

2.4.2 Funkční požadavky pro stránky e-shopu

Pro stránky internetového obchodu byly stanoveny následující funkční požadavky:

Možnost prohlížení všech dostupných produktů.

Zákazník by měl mít možnost zobrazit si všechny dostupné produkty a také se podrobněji seznámit s vybranou položkou přechodem na stránku konkrétního produktu.

Manipulace s produkty v nákupním košíku.

Zákazník by měl mít možnost před nákupem přidat do nákupního košíku všechny produkty, o jejichž zakoupení má zájem. Tato funkce musí být dostupná i pro neregistrované uživatele. Obsah košíku musí zůstat zachován i po uplynutí určité doby nebo po uzavření webového prohlížeče. Součástí funkcionality košíku by měla být také možnost správy jednotlivých položek, jako je úprava množství konkrétního produktu nebo úplné odstranění položky.

Přidání produktu na seznam oblíbených produktů.

Zákazníkům by mělo být umožněno ukládat si do seznamu oblíbených produkty, které je zaujaly, ale jejichž zakoupení dosud zvažují. Zároveň by měli mít možnost tyto produkty ze seznamu kdykoli odstranit.

Rezervace termínu konzultace

V případě zájmu o konzultaci svého projektu by měl zákazník mít možnost provést rezervaci prostřednictvím volby preferovaného termínu a formy setkání.

Registrace a přihlášení do osobního účtu

Klient by měl mít možnost si v případě zájmu vytvořit vlastní účet a poté se do něj přihlásit pomocí svého uživatelského jména a hesla.

2.4.3 Dodatečné požadavky

Pro obě části systému – e-shop a příslušnou administrátorskou aplikaci – byly definovány společné nefunkční požadavky, jelikož se jedná o univerzální kritéria, která jsou relevantní pro jakoukoli moderní webovou aplikaci.

Zvolená kritéria zahrnují:

- Bezpečnost: v systémech by měla být implementována ochrana proti neoprávněnému přístupu; citlivá data, jako jsou například hesla, by se měla ukládat v hashovací podobě.
- Intuitivní a přehledné uživatelské rozhraní.
- Responzivní design: zobrazení stránek by se mělo přizpůsobovat pro zařízení s různým rozlišením obrazovky.
- Konzistence designu a znouvupoužitelnost komponent.

3 Implementace řešení bez použití AI

Tato kapitola popisuje proces implementace navrženého řešení s využitím programovacího jazyka JavaScript, frameworku Next.js, Javascriptové knihovny React.js a databázového systému MongoDB. Proces vývoje byl realizován na základě vysokoškolského vzdělávacího programu a dostupných online výukových materiálů.

3.1 Panel administrátora

3.1.1 Struktura projektu

Architektura složek projektu odpovídá standardní struktuře projektů ve frameworku Next.js od verze 13, kdy byl zaveden nový způsob organizace aplikací na základě App Routeru. V této architektuře představuje základní strukturální jednotku složka `app/`, která obsahuje jednotlivé stránky a serverové API endpointy. Každý podadresář ve složce `app/` odpovídá konkrétní URL cestě a používá se pro směrování v rámci aplikace.

Soubor `layout.js` definuje společné rozvržení pro všechny soubory `page.js`, soubor `globals.css` nastavuje pro jednotlivé elementy globální styly, které platí v celé aplikaci.

V kořenovém adresáři projektu se dále nachází následující složky:

- `Components/` – obsahuje znovupoužitelné komponenty uživatelského rozhraní.
- `Hooks/` – zahrnuje vlastní React hooky sloužící k opakovanému využití funkcí pro správu stavů.
- `Lib/` – obsahuje knihovní soubory pro připojení k databázi MongoDB a její obsluhu pomocí knihovny Mongoose.
- `Schemas/` – slouží k definici datových Mongoose schémat, která se využívají pro strukturování a validaci dat.

```
admin-panel/  
├─ app/  
│   ├─ page.js  
│   ├─ layout.js  
│   ├─ globals.css  
│   ├─ api/  
│   │   └─ [routy]/route.js  
│   ├─ products/  
│   │   ├─ createProduct/page.js  
│   │   └─ editProduct/[...id]/page.js  
│   ├─ categories/  
│   │   ├─ createCategory/page.js  
│   │   └─ editCategory/[...id]/page.js  
│   ├─ orders/  
│   │   └─ page.js  
│   └─ admins/  
│       ├─ createAdmin/page.js  
│       └─ editAdmin/[...id]/page.js  
├─ components/  
├─ hooks/  
├─ lib/  
├─ schemes/  
├─ public/  
├─ tailwind.config.js  
└─ další konfigurační soubory
```

Obrázek 6: zjednodušené schéma složkové struktury aplikace (zdroj: autor)

3.1.2 Připojení k databázi

Pro účely tohoto projektu byla zvolena databáze MongoDB. Pro snadnější práci s databází byly také použité dvě knihovny: mongodb a mongoose. MongoDB byla zvolená zejména z důvodu kompatibility s autentizační knihovnou NextAuth, použitou v aplikaci pro implementaci přihlašovací funkce. Tato funkce je podrobněji popsána v následující podkapitole. Knihovna Mongoose umožňuje vytvářet datová schémata a modely a v aplikaci se využívá pro ukládání dat o produktech, kategoriích nebo uživatelských účtech.

Prvním krokem při připojení databáze bylo vytvoření nového projektu ve cloudové službě MongoDB Atlas, poté byl odkaz na vytvořený projekt uložen do souboru .env.local.

Následně byl vytvořen konfigurační soubor pro MongoDB s názvem db.js. V souboru se používá tzv. singleton logika. Singleton je návrhový vzor, který dovoluje vytvořit pouze jednu instanci třídy a poskytuje celé aplikaci globální přístup k této instanci. (Tal, 2021) V kontextu daného projektu to znamená, že se vždy vytváří pouze jedno připojení k databázi, které může být opakovaně použito v celé aplikaci.

```
import { MongoClient, ServerApiVersion } from "mongodb";

if (!process.env.MONGODB_URI) {
  throw new Error('Invalid/Missing environment variable: "MONGODB_URI"');
}

const uri = process.env.MONGODB_URI;
const options = {
  serverApi: {
    version: ServerApiVersion.v1,
    strict: true,
    deprecationErrors: true,
  },
};

let client;
let clientPromise;

if (!global._mongoClientPromise) {
  client = new MongoClient(uri, options);
  global._mongoClientPromise = client.connect();
}
clientPromise = global._mongoClientPromise;

export default clientPromise;
```

Výpis 1: ukázka připojení k databázi

Pro připojení k databázi MongoDB pomocí knihovny mongoose.js byl vytvořen soubor mongoose.js. V souboru se také kontroluje, zda aktivní připojení již existuje. Tento přístup brání vytvoření nového připojení při každém volání serverové funkce.

```
const connectToMongoose = async () => {
  if (mongoose.connection.readyState >= 1) return;
  await mongoose.connect(process.env.MONGODB_URI, {
    useNewUrlParser: true,
    useUnifiedTopology: true,
  });
};
```

```
export default connectToMongoose;
```

Výpis 2: připojení k MongoDB pomocí knihovny mongoose

3.1.3 Funkce Autentizace

V zájmu zabezpečení firemních dat je přístup k panelu správce umožněn pouze úzkému okruhu uživatelů, kteří mají v databázi vytvořeny účty.

Pro implementaci autentizace v projektu byla použita knihovna NextAuth.js. Knihovna umožňuje přihlášení pomocí osobních údajů, jako je uživatelské jméno, e-mail a heslo, a také pomocí externích služeb, jako je Google nebo Github. Pro autentizaci v panelu správce však byla zvolena pouze funkce přihlášení pomocí hesla a e-mailu. Tím pádem správce může snadno samostatně přidávat a spravovat přístupové údaje.

Pro nainstalování knihovny byl použit příkaz `npm install next-auth`. Následně na cestě `app/api/auth` byl vytvořen soubor s názvem `[...nextauth]`. Soubor slouží jako dynamická trasa pro NextAuth.js a obsahuje jeho globální konfiguraci. (2024)

Jako adapter byl použit `MongoAdapter`, který umožňuje připojení k databázi MongoDB. Jako strategie byla zvolena strategie JWT (JSON Web Token), která má jednodušší implementaci oproti strategii „database“ a je postačující pro účely administrátorského panelu s menším počtem uživatelů. Vlastnost `maxAge` ukazuje, jak dlouho bude relace platná, a byla nastavena na 30 dnů.

V části `providers` byl uveden `CredentialsProvider`, který umožňuje přihlašování pomocí e-mailové adresy a hesla.

V souboru `[...nextauth]` je také definována funkce `authorize`, která se spouští po přihlášení a vyhledává uživatele v databázi na základě zadaných přihlašovacích údajů. Z bezpečnostních důvodů procházejí uživatelská hesla před uložením procesem hašování, které je v projektu vyřešeno pomocí populárního npm balíčku `bcrypt`. Pro vytvoření otisku vstupního hesla se používá funkce `bcrypt.hash`. K porovnání zadaného hesla s hashem uloženým v databázi v projektu slouží funkce `bcrypt.compare`.

```
export const authOptions = {
  adapter: MongoDBAdapter(clientPromise),
  session: {
    strategy: "jwt",
    maxAge: 30 * 24 * 60 * 60,
  },
  providers: [

    CredentialsProvider({
      name: "credentials",
      credentials: {
        email: { label: "Email", type: "text" },
        password: { label: "Heslo", type: "password" },
      },
    })
  ]
}
```

```

    async authorize(credentials) {
      await connectToMongoose();

      const user = await User.findOne({ email: credentials.email });
      if (!user) {
        throw new Error("User is not found");
      }

      const passwordMatch = await bcrypt.compare(credentials.password,
      user.password);
      if (!passwordMatch) {
        throw new Error("Invalid passpord");
      }

      return {
        id: user._id.toString(),
        name: user.name,
        email: user.email,
        image: user.image || null,
      };
    },
  }),
],
};

const handler = NextAuth(authOptions);

export { handler as GET, handler as POST };

```

Výpis 3: ukázka nastavení knihovny NextAuth.js

```
const hashedPassword = await bcrypt.hash(password, 10);
```

Výpis 4: ukázka funkce bcrypt.hash

Aby informace o aktuální relaci byly přístupné ve všech částech aplikace, layout aplikace byl obalen do komponenty `<SessionProvider>`. Na straně klienta byl použit React Hook `useSession`, pomocí kterého je možné zkontrolovat, zda je uživatel přihlášen, a zjistit data o relaci.

```
export default function RootLayout({ children }) {
```



```

return (
  <html lang="en">
    <body className={` ${geistSans.variable} ${geistMono.variable}
antialiased`} >
      <SessionProvider>
        {children}
        <Toaster position="top-right" reverseOrder={false} />
      </SessionProvider>
    </body>
  </html>
);
}

```

Výpis 5: použití komponenty <SessionProvider>

```
const {data: session, status} = useSession();
```

Výpis 6: získání dat relace pomocí funkce useSession

Pro zahájení samotného přihlášení uživatele se používá standardní metoda signIn(), která odesílá přihlašovací údaje, vytváří novou relaci a případně směřuje uživatele na jinou stránku. (2024)

```

async function handleSubmit(ev){
  ev.preventDefault();
  try{
    const responseLogin= await signIn("credentials", {email,password,redirect:
false,});
    if(responseLogin?.error){

      toast.error("Neplatné přihlšovací údaje")
      return;
    }

    setEmail("");
    setPassword("");
    router.refresh();

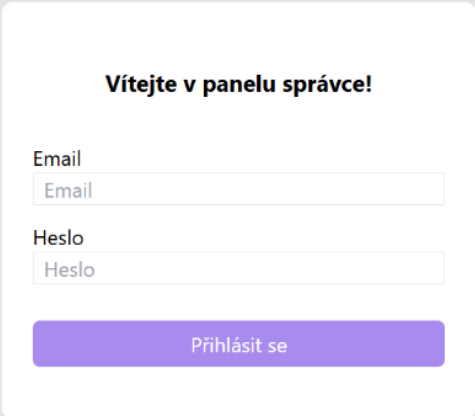
    console.log("SESSION", session);
    console.log("STATUS", status);

  }catch(error){
    console.log("error:",error)
  }
}

```

Výpis 7: ukázka použití metody signIn

Jako poslední krok pro implementaci funkce přihlášení byla vytvořena přihlašovací stránka, která obsahuje pole pro zadání e-mailové adresy a hesla.



Vítejte v panelu správce!

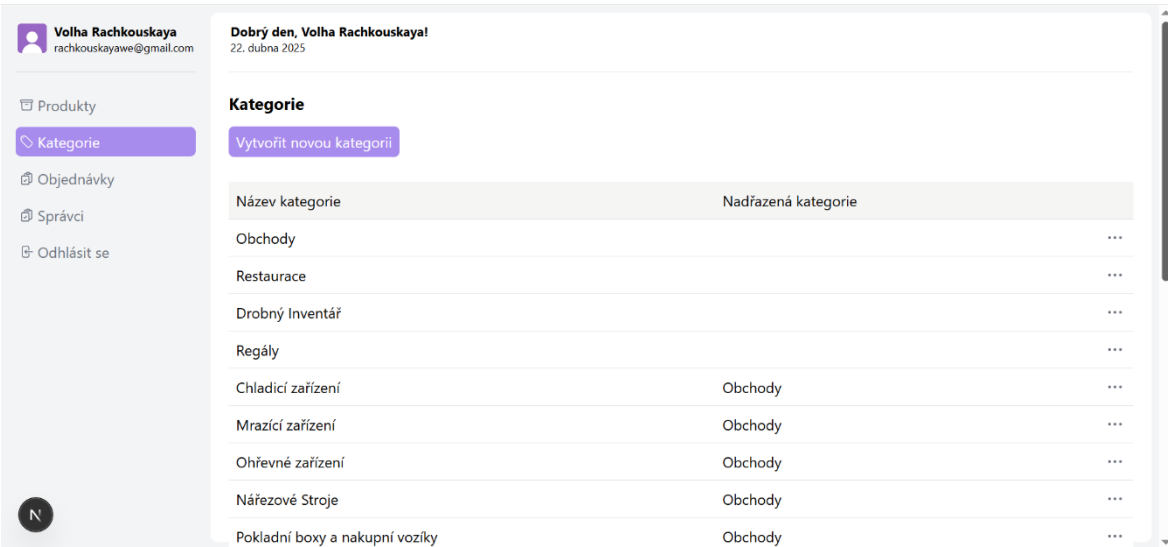
Email

Heslo

Obrázek 7: přihlašovací stránka (zdroj: autor)

3.1.4 Správa kategorií

Pro správu kategorií v panelu slouží stránka s názvem „Kategorie“. Zde může správce prohlížet stávající kategorie, upravovat je, mazat a přidávat nové.



Volha Rachkouskaya
rachkouskayawe@gmail.com

Dobrý den, Volha Rachkouskaya!
22. dubna 2025

Kategorie

[Vytvořit novou kategorii](#)

Název kategorie	Nadřazená kategorie	
Obchody		...
Restaurace		...
Drobný Inventář		...
Regály		...
Chladicí zařízení	Obchody	...
Mrazicí zařízení	Obchody	...
Ohřevné zařízení	Obchody	...
Nářezové Stroje	Obchody	...
Pokladní boxy a nákupní vozíky	Obchody	...

Obrázek 8: stránka "Kategorie" (zdroj: autor)

V tabulce jsou uvedeny všechny stávající kategorie. Po kliknutí na tlačítko menu, umístěné naproti každé položce, bude uživateli nabídnuta možnost upravit nebo odstranit kategorii.

Název kategorie	Nadřazená kategorie
Obchody	...
Restaurace	Upravit Odstranit
Drobný Inventář	...
Regály	...

Obrázek 9: tlačítko pro úpravu a odstranění kategorie (zdroj: autor)

Pro vytvoření nové kategorie lze použít tlačítko „Vytvořit novou kategorii“. Následně uživatel může zadat název a vybrat nadřazenou kategorii ze seznamu již existujících kategorií, nebo zvolit možnost „Žádná nadřazená kategorie“. Do kategorií lze také přidat fotografii, která se poté zobrazí na webových stránkách e-shopu.

Obrázek 10: přidání nové kategorie (zdroj: autor)

Obrázek 11: volba nadřazené kategorie (zdroj: autor)

Nahrávání fotografií bylo v projektu vyřešeno pomocí API endpointu umístěného na cestě `app/api/files/route.js`. V souboru byly definovány dvě metody: POST a DELETE. K uložení

fotografií byla použita služba Amazon Simple Storage Service (Amazon S3) a knihovna s názvem `@aws-sdk/client-s3`, vyvinutá společností Amazon Web Services speciálně pro práci s úložištěm Amazon S3.

Metoda POST se používá k uložení fotografií do databáze. Nejprve ze získaného požadavku extrahuje soubor, který následně převádí do formátu Buffer, což je standardní formát pro práci s binárními daty v prostředí Node.js.

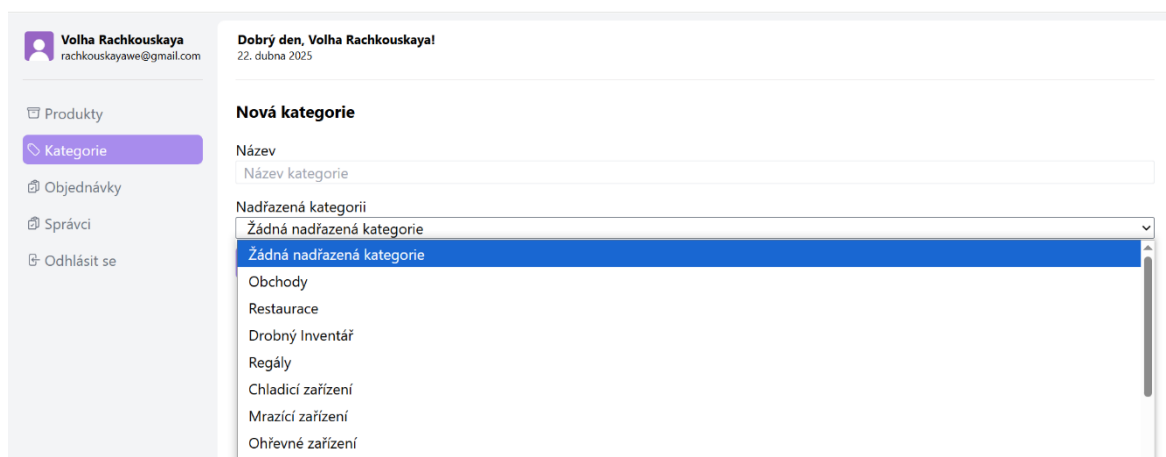
Každému souboru je vygenerován unikátní název a následně se soubor zasílá do úložiště pomocí funkce `PutObjectCommand`. Po úspěšném nahrání je vytvořen veřejně přístupný odkaz na uložený soubor. Tento odkaz se pak používá k zobrazení obrázku na stránkách webové aplikace.

Metoda DELETE slouží ke smazání nahraných obrázků z úložiště. Z požadavku získává název souboru, pod kterým je uložen, a poté jej odstraňuje pomocí funkce `DeleteObjectCommand`.

```
const client = new S3Client({
  region: "us-east-1",
  credentials: {
    accessKeyId:process.env.S3_ACCESS_KEY,
    secretAccessKey:process.env.S3_SECRET_ACCESS_KEY,
  },
});

await client.send(new PutObjectCommand({
  Bucket: bucketName,
  Key: newFileName,
  Body: fileBuffer,
  ContentType: file.type,
  ACL: "public-read",
}));
```

Výpis 8: Ukázka kódu pro připojení k službě Amazon S3 a odeslání souboru



Obrázek 12: volba nadřazené kategorie (zdroj: autor)

Pro implementaci funkcí přidání, úpravy, mazání kategorií a načtení jejich seznamu z databáze byl vytvořen API endpoint umístěný v souboru `app/api/categories/route.js`. V souboru byly definovány čtyři základní HTTP metody: POST, GET, PUT a DELETE.

Každá metoda nejprve používá dříve definovanou funkci `connectToMongoose` pro připojení k databázi. Po úspěšném připojení se s daty uloženými v databázi manipuluje pomocí mongoose modelu `Category` a základních funkcí knihovny Mongoose: `create`, `find`, `findOne`, `updateOne`, `deleteOne`.

Metoda POST slouží k vytvoření nové kategorie. Z těla požadavku získává název kategorie a vybranou podkategorii a vytváří nový záznam pomocí funkce `create`.

Metoda GET se používá k načtení již existujících kategorií z databáze. V případě, že tělo požadavku obsahuje unikátní identifikátor (ID) konkrétní kategorie, server vrací daný záznam této kategorie. Pokud odeslaný požadavek žádné id neobsahuje, server vrací seznam všech kategorií, který je následně použit pro zobrazení na stránce.

Metoda PUT je určena pro úpravu kategorií. Z těla požadavku získává id kategorie, nový název a novou nadřazenou kategorii a následně ukládá změny pomocí funkce `updateOne`.

Metoda DELETE získává ID kategorie a následně odstraňuje záznam pomocí funkce `deleteOne`.

Mongoose model definuje strukturu dokumentů, které se ukládají v databázi, a předchází ukládání neplatných nebo neúplných dat. V daném případě budou data úspěšně zapsána do kolekce pouze tehdy, pokud budou obsahovat povinný název ve formátu „text“ a volitelně zvolenou nadřazenou kategorii.

```
const CategorySchema = new Schema({
  name: { type: String, required: true },
  parent: { type: Schema.Types.ObjectId, ref: "Category" },
});
```

```
});

const Category = models.Category || model("Category", CategorySchema);

console.log("Loaded Category model:", Category);

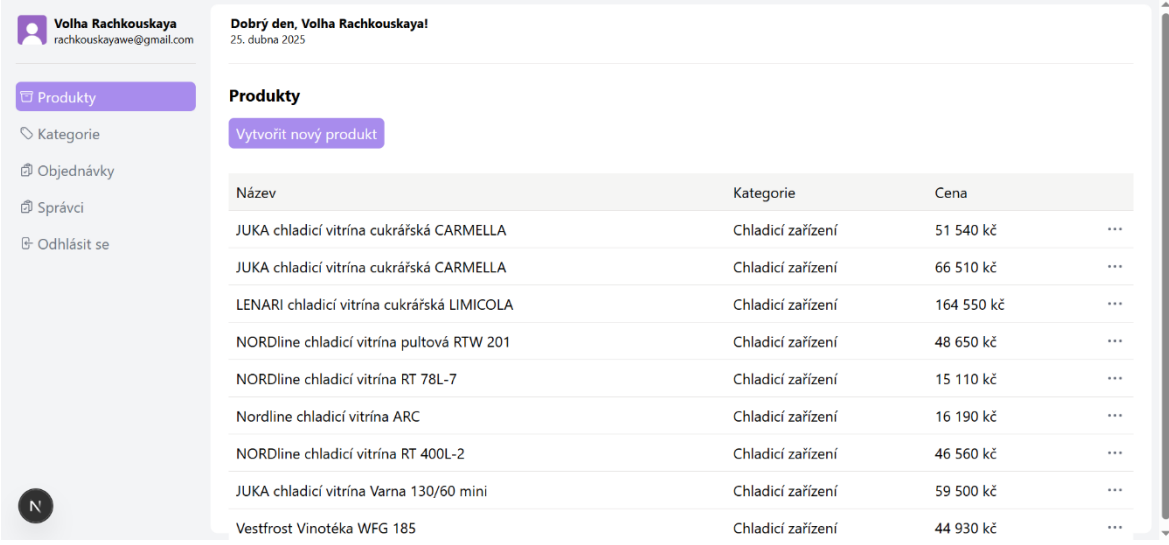
export default Category;
```

Výpis 9: model "Category"

3.1.5 Správa produktů

Pro správu produktů, které budou umístěny na webových stránkách firmy, může správce použít stránku s názvem „Produkty“. Zde je možné prohlížet existující produkty, upravovat je, mazat a přidávat nové položky.

Stránka má podobnou logiku jako stránka „Kategorie“. V tabulce jsou uvedeny všechny existující produkty, jejich kategorie a cena. Po rozkliknutí tlačítka se třemi tečkami může uživatel jednotlivý produkt upravit nebo odstranit. Tlačítko „Vytvořit nový produkt“ slouží k přidání produktů.



Název	Kategorie	Cena	
JUKA chladicí vitrina cukrářská CARMELLA	Chladicí zařízení	51 540 Kč	...
JUKA chladicí vitrina cukrářská CARMELLA	Chladicí zařízení	66 510 Kč	...
LENARI chladicí vitrina cukrářská LIMICOLA	Chladicí zařízení	164 550 Kč	...
NORDline chladicí vitrina pultová RTW 201	Chladicí zařízení	48 650 Kč	...
NORDline chladicí vitrina RT 78L-7	Chladicí zařízení	15 110 Kč	...
Nordline chladicí vitrina ARC	Chladicí zařízení	16 190 Kč	...
NORDline chladicí vitrina RT 400L-2	Chladicí zařízení	46 560 Kč	...
JUKA chladicí vitrina Varna 130/60 mini	Chladicí zařízení	59 500 Kč	...
Vestfrost Vinotéka WFG 185	Chladicí zařízení	44 930 Kč	...

Obrázek 13: stránka "Produkty", (zdroj: autor)

Obrázek 14: přidání nového produktu (zdroj: autor)

Při přidávání produktu lze zadat název, popis produktu, cenu, vybrat kategorii ze seznamu existujících kategorií a nahrát jednu nebo více fotografií produktu. Struktura těchto polí je definována v komponentě `ProductForm.js`.

Pro nahrávání fotografií se stejně jako v případě kategorií používá služba Amazon S3.

Pro uložení produktů a manipulaci s nimi byl definován API endpoint umístěný na cestě `app/api/products/route.js`. Soubor má podobnou logiku jako endpoint vytvořený za účelem práce s kategoriemi.

V souboru byly definovány metody POST, GET, PUT a DELETE, které slouží k uložení dat o produktu, jejich aktualizaci, vykreslení nebo mazání.

Připojení k databázi probíhá pomocí funkce `connectToMongoose`.

Pro manipulaci s daty byl také vytvořen Mongoose model `Product`. Model zajišťuje, že produkt je uložen pouze v případě, že splňuje určité požadavky – má název a popis typu „text“ a cenu uvedenou v číselném formátu. V případě, že správce zadá údaje, které nesplňují tato kritéria, na stránce se zobrazí chybové hlášení.

```
const productSchema = new Schema({
  name: {
    type: String,
    required: true,
    trim: true,
    minlength: 1,
  },
  description: {
    type: String,
    required: true,
    trim: true,
```

```

    },
    price: {
      type: Number,
      required: true,
      min: 0.01,
    },
    category: {type: Schema.Types.ObjectId, ref:"Category"},
    photos:[
      {type:String},
    ],
  },{ timestamps: true });

productSchema.index({ name: 'text' });
productSchema.index({ price: 1 });

const Product = models.Product || model('Product', productSchema);

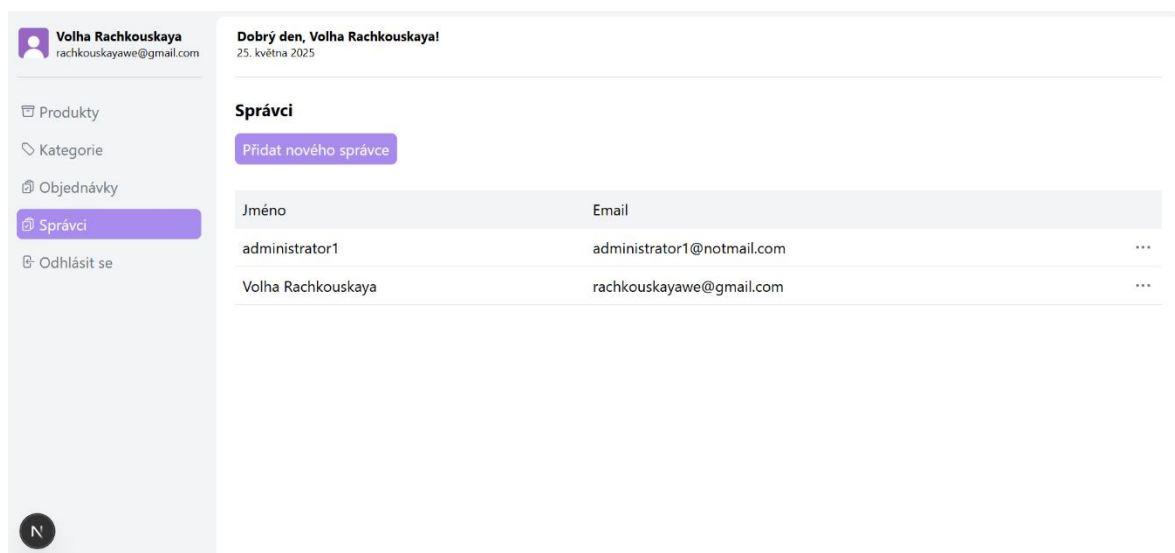
module.exports = Product;

```

Výpis 10: definování modelu "Product"

3.1.6 Správa uživatelů

Stránka „Správci“ slouží ke správě seznamu uživatelů, kteří mají přístup k funkcím a údajům v administrátorském panelu. Stránka obsahuje tabulku se všemi aktuálními uživateli a také tlačítka pro změnu údajů stávajících správců a pro jejich odstranění. Pomocí tlačítka „Přidat nového správce“ lze do seznamu přidat dalšího uživatele, který bude mít přístup do systému.



Obrázek 15: stránka "Správci" (zdroj: autor)

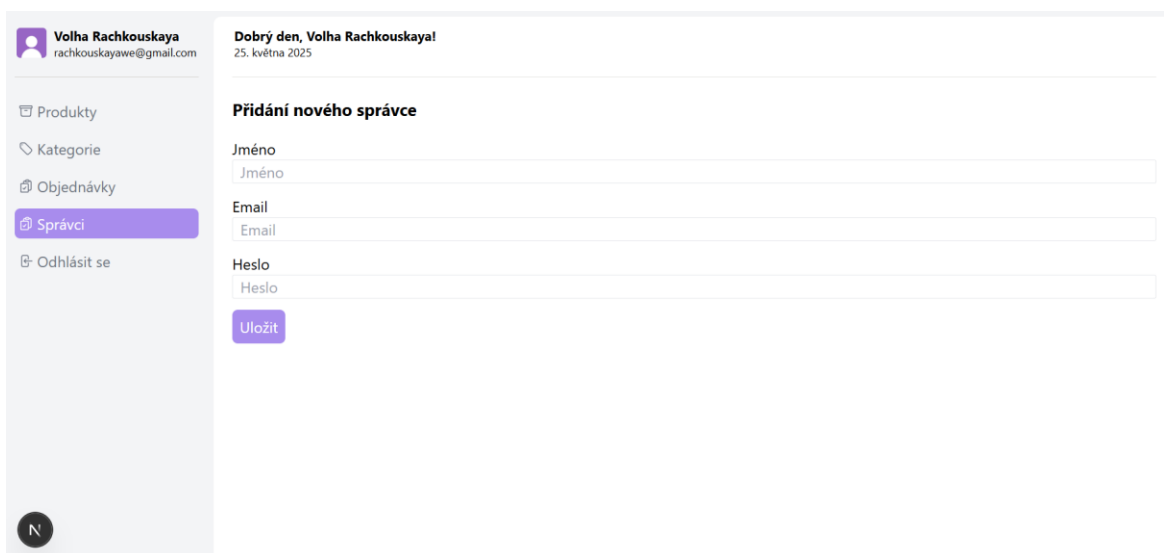
Pro kontrolu a uchování v databázi údajů o uživateli byl vytvořen Mongoose model User.

Při vytváření profilu nového správce je nutné zadat jméno, e-mail a heslo, které se budou používat pro autentizaci uživatele. Tento formulář a funkce pro zpracování zadaných údajů na server byly definovány v souboru UserForm.js.

Před uložením dat z formuláře se provádí kontrola, zda v databázi neexistuje uživatel s danou e-mailovou adresou. K tomu byl vytvořen API endpoint existingAdmin, ve kterém se pomocí funkce findOne a modelu User hledají uživatelé, jejichž e-mail se shoduje s novým e-mailem.

Součástí souboru UserForm.js je také ověření síly vytvořeného hesla – v kódu se kontroluje, zda je jeho délka delší než 8 znaků a zda je v něm přítomna aspoň jedna číslice a jedno velké písmeno.

Pro další manipulaci s údaji správců byl vytvořen API endpoint Admins. V souboru byly definovány asynchronní funkce POST, GET, PUT a DELETE, které se používají k ukládání nových dat, načítání nebo změně již existujících záznamů a k jejich odstranění.



Obrázek 16: formulář pro přidání nového správce (zdroj: autor)

```
export async function POST(request){
  try{
    await connectToMongoose();
    const {email} = await request.json();
    const user = await User.findOne({email}).select("_id");
    return NextResponse.json({ user });
  }catch(error){
    console.error('Error in POST /api/existingAdmin:', error);
    return NextResponse.json(
      { error: "Server error", details: error.message }
    )
  }
}
```

```
}
```

Výpis 11: funkce pro kontrolu unikátnosti zadaného hesla

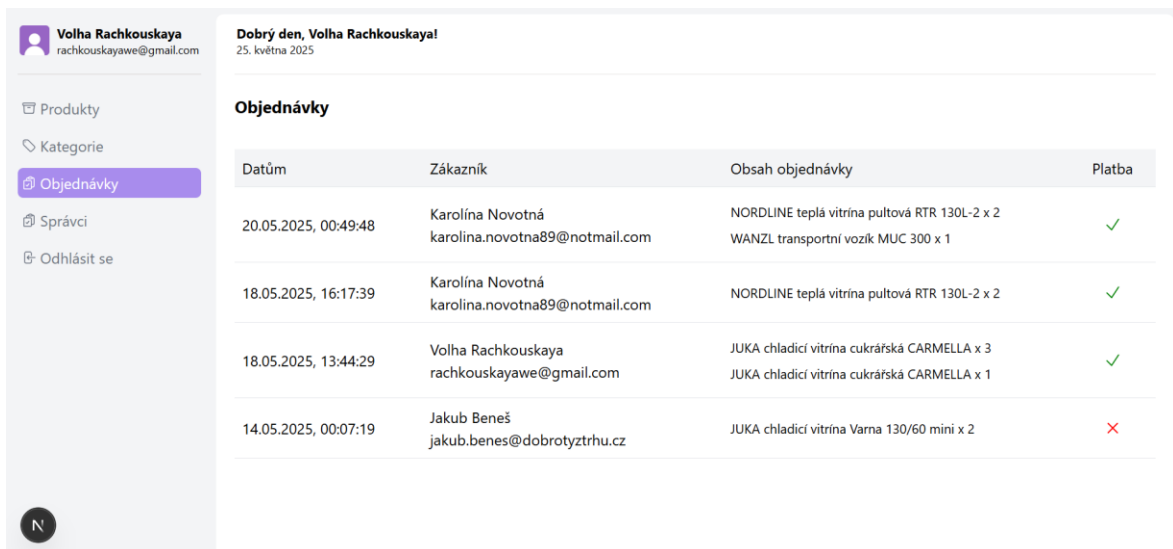
```
const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
  },
  email: {
    type: String,
    required: true,
  },
  password: {
    type: String,
    required: true
  },
}, { timestamps: true });

export const User = mongoose.models.User || mongoose.model("User",
userSchema);
```

Výpis 12: definování modelu User

3.1.7 Správa objednávek

Na stránce „Objednávky“ může správce zobrazit informace o všech objednávkách vytvořených v e-shopu. V tabulce se zobrazuje datum vytvoření objednávky, jméno zákazníka, jeho e-mailová adresa a zakoupené produkty. Sloupec Platba ukazuje, zda byla objednávka již zaplacená. Tato informace je získávána z aktuální relace vytvořené systémem pro online platby Stripe, který se používá v projektu v testovacím režimu. Podrobněji je implementace testovacího platebního systému popsána v kapitole „Objednání zboží“.



Obrázek 17: stránka "Objednávky" (zdroj: autor)

Pro získání informací o objednávce z databáze na adrese `app/api/orders/route.js` byl vytvořen API endpoint. Nejprve pomocí funkce `connectToMongoose` probíhá připojení k databázi, poté se pomocí funkcí `find` a datového modelu `Order` vyhledají všechny záznamy z kolekce v databázi. Nalezené objednávky se seřadí v sestupném pořadí – od nejnovějších po nejstarší.

```
export async function GET(){
  try{
    await connectToMongoose();
    const orders = await Order.find().sort({createdAt: -1});
    return NextResponse.json(orders);
  }catch(error){
    console.error('Error in GET /api/orders:', error);
    return NextResponse.json(
      { error: "Server error", details: error.message }
    );
  }
}
```

Výpis 13 : funkce GET v API endpointu Orders

3.2 Stránky E-shopu

3.2.1 Struktura projektu

Struktura webových stránek internetového obchodu je velmi podobná struktuře administrátorského panelu, protože oba projekty byly vytvořeny na základě frameworku Next.js 14, který pro přemístění uvnitř aplikace využívá princip App Router. Ve složce app/ se nacházejí následující jednotky:

- obecné stránky, jako jsou produkty, kategorie, účet apod.
- stránky obsahující v adrese dynamický parametr ID, jako jsou stránky konkrétní kategorie nebo konkrétního produktu
- API endpointy, sloužící pro komunikaci mezi klientskou a serverovou logikou
- standardní soubory page.js, layout.js a global.js, které jsou součástí architektury Next.js.

V kořenovém adresáři projektu se nacházejí mimo jiné složky components/ a hooks/, které byly vytvořené za účelem uložení kódu opakovaně používaného v různých částech aplikace.

Složka lib/ obsahuje logiku pro připojení k databázi.

Ve složce schemes/ jsou umístěna datová schémata, používaná pro výměnu dat s databází. Ve složce public/ se nachází statické soubory, jako jsou obrázky, které se využívají při vykreslování uživatelského rozhraní.

```
store-website/  
├─ app/  
│   ├─ page.js  
│   ├─ layout.js  
│   ├─ [routy]/page.js  
│   ├─ [routy]/[...id]/page.js  
│   └─ api/  
│       └─ [endpointy]/route.js  
├─ components/  
├─ hooks/  
├─ lib/  
├─ schemes/  
├─ public/  
└─ další konfigurační soubory
```

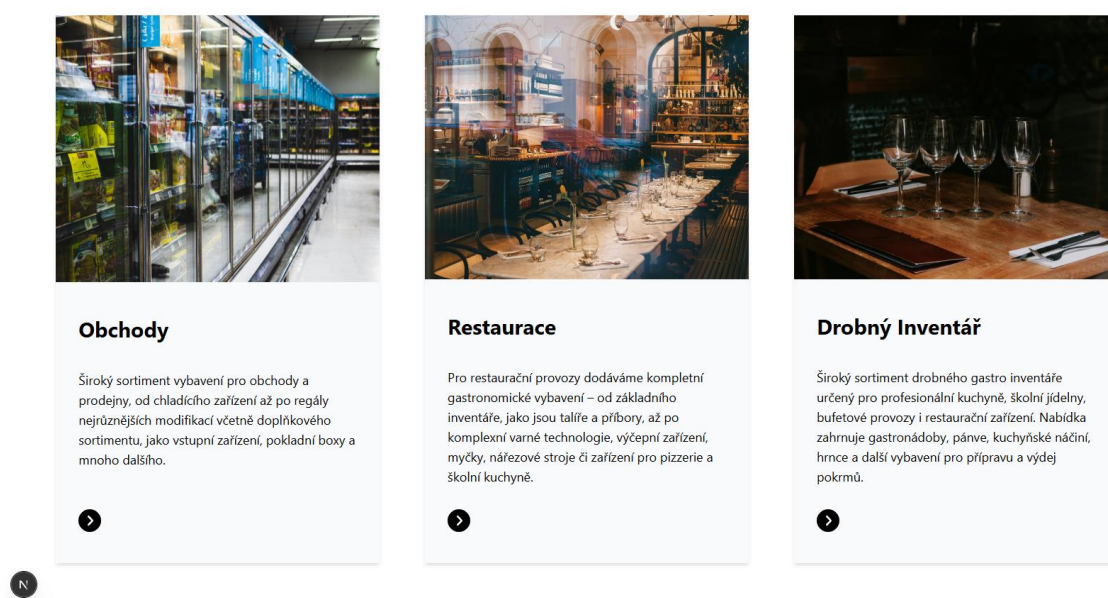
Obrázek 18: zjednodušené schéma složkové struktury webových stránek pro E-shop (zdroj: autor)

3.2.2 Domovská stránka

Při návrhu informační architektury hlavní stránky byl použit princip „hub-and-spoke“. Při tomto přístupu hlavní stránka funguje jako centrální rozcestník a odkazuje na další stránky s tematicky souvisejícím obsahem. Výběr této architektury měl dva hlavní důvody: usnadnění orientace zákazníků na webu a zlepšení optimalizace pro vyhledávače, jelikož stručný přehled jednotlivých sekcí webu na hlavní stránce pomáhá vyhledávacím algoritmům lépe porozumět webu a provádět efektivnější indexaci. (Brown, 2023)

3.2.3 Stránka „Produkty“

Vzhledem k širokému sortimentu společnosti byly produkty pro jednodušší orientaci rozčleněny do tří hlavních kategorií: vybavení pro restaurace a jídelny, vybavení pro obchody a drobný inventář. Stránka „Produkty“ obsahuje uživatelské rozhraní, které zobrazuje fotografie, název a podrobný popis každé kategorie. Po kliknutí na příslušné tlačítko je uživatel přesměrován na stránku dané kategorie.



Obrázek 19: uživatelské rozhraní zobrazující hlavní kategorie (zdroj: autor)

Na stránce konkrétní kategorie se nachází blok s odkazy na všechny dostupné podkategorie. Pod nimi je umístěn přehled jednotlivých produktů. Každá položka má svoji kartu s fotografií, názvem, popisem, cenou a tlačítky pro přidání zboží do nákupního košíku nebo seznamu oblíbených produktů.

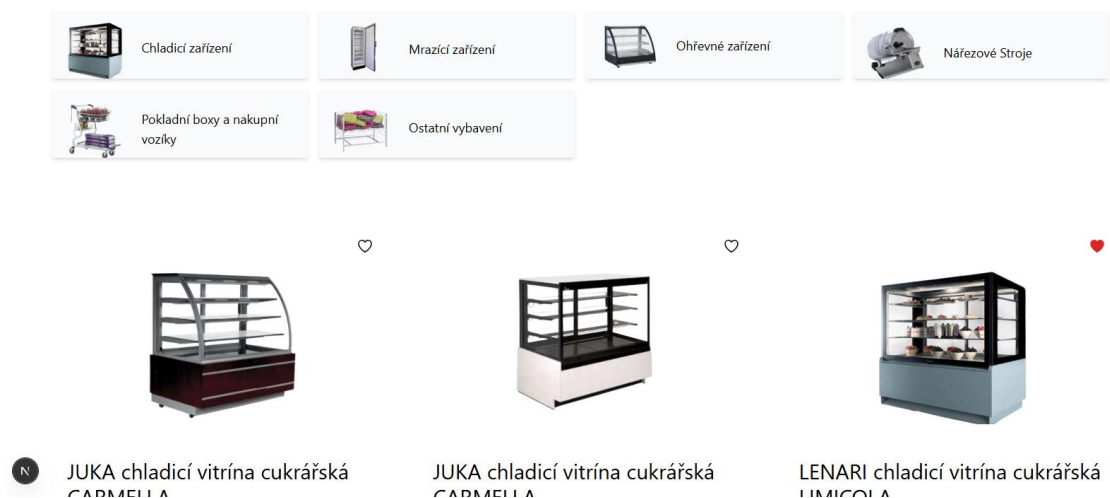
Stránka konkrétní kategorie obsahuje v adrese dynamický parametr – ID dané kategorie. K jeho získání se používá React hook *useParams* poskytovaný knihovnou *next/navigation*. Nalezený ID se následně používá k získání seznamu všech podkategorií a také produktů patřících do aktuální kategorie nebo jakékoli z jejích podkategorií.

Pro obdržení informací o podkategoriích z databázových sbírek na cestě `app/api/categories/route.js` byl vytvořen API endpoint. V souboru byl nadefinována asynchronní metoda GET, která pomocí funkce `find` a mongoose modelu `Category` hledá veškeré kategorie, jejichž pole `parent` odpovídá danému ID. Nalezený seznam se odešílá zpět na stranu klienta. Pro jeho zobrazení byl vytvořena samostatná komponenta „ChildCategoriesGrid“.

Podobný přístup se používá i k získání informací o zboží. Na adrese `app/api/products/route.js` byl vytvořen API endpoint zajišťující komunikaci s databází. V souboru byla definována asynchronní funkce GET, která slouží ke zpracování http GET požadavků. Nejprve se z URL požadavku získává ID příslušné kategorie pomocí metody `searchParams.get(„categoryId“)`. Následně se vytváří pole `categoryIds`, které obsahuje id aktuální kategorie a všech podkategorií. Pomocí funkce `find` a Mongoose modelu `Product` se provádí dotaz do databáze, který vrací odpovídající seznam produktů.

Výsledný seznam se vrací na stranu klienta a je vykreslen pomocí komponenty `ProductsGrid`, jež byla vytvořena za účelem zachování konzistenci v zobrazení produktů na různých stránkách aplikace.

Obchody



Obrázek 20: stránka s přehledem produktů vybrané kategorie (zdroj: autor)

3.2.4 Osobní účet

V části Osobní účet mají uživatelé možnost přihlásit se k již existujícímu účtu pomocí svých přihlašovacích údajů nebo si založit nový účet.

Přihlaste se nebo si vytvořte účet

Už máte účet? Přihlaste se

Email

Heslo

Přihlásit se

Ještě nemáte zákaznický účet?

Zaregistrujte se

Registrací můžete snadno spravovat své údaje, ukládat oblíbené produkty a sledovat objednávky.

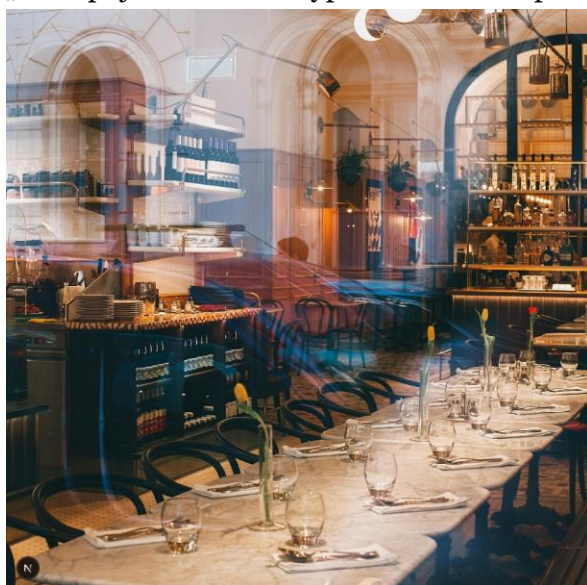
Registrovat



Výpis 14: stránka "Osobní účet" (zdroj: autor)

Při registraci musí uživatelé vyplnit formulář obsahující osobní údaje, jako je e-mail, jméno, příjmení a heslo. Z bezpečnostních důvodů musí heslo uživatelů mít alespoň 8 znaků. V případě vytvoření příliš krátkého hesla bude uživatel upozorněn chybovou zprávou.

Dále musí uživatel také zadat údaje o adrese, které mohou být použity pro zasílání faktur při vytváření objednávek. V případě, že zákazník zastupuje společnost, může zvolit možnost „Nakupuji na firmu“ a vyplnit dodatečná pole, jako je název společnosti, IČ a případně DIČ.



Vytvořte si účet

Osobní údaje

Email*

Heslo*

Fakturační adresa

☐ Nakupuji na firmu

Jméno a příjmení*

Telefon*

Ulice*

Číslo domu*

Město*

PSČ*

Vytvořit účet

Obrázek 21: registrační formulář (zdroj: autor)

Fakturační adresa

☒ Nakupuji na firmu

Jméno a příjmení*

Název Společnosti

IČ

DIČ (pouze plátcí DPH)

Obrázek 22: dodatečná pole pro registraci firmy (zdroj: autor)

Město*

Praha

PSC*

130 00

Heslo musí mít alespoň 8 znaků

Vytvořit účet

Obrázek 23: zpráva informující o nedostatečné délce hesla (zdroj: autor)

Pro implementaci registrační funkce byl vytvořen soubor s názvem ClientDataForm. Za účelem optimalizace kódu se stejný soubor používá i v dalších částech webu: při zobrazování údajů klienta v jeho osobním účtu a při zadávání objednávek. Proto byl v souboru použit parametr režimu (mode), který přizpůsobuje funkce a pole v souboru pro konkrétní stránku.

Na stránce s registrací se používá režim registration. V tomto režimu se zobrazí pole pro heslo a také se spustí funkce, která kontroluje, zda v databázi existuje účet se stejnou e-mailovou adresou. To se dělá, aby se zabránilo vytvoření dvou stejných účtů. Pokud je kontrola úspěšná, údaje nového uživatele se uloží do databáze.

Pro realizaci těchto funkcí byl vytvořen mongoose model Client.js a také soubory ExistingClient a Clients, které slouží jako API endpointy.

V souboru Clients se nové záznamy o klientech ukládají pomocí funkce POST. Funkce získává z těla příchozího požadavku údaje zadané zákazníkem. Následně probíhá připojení k databázi, hashování hesla a vytvoření nového záznamu v kolekci clients pomocí funkce create.


```

if(mode === "registration"){
  try{

    const responseClients = await
    axios.post("/api/existingClient",{email});

    if (responseClients.data.client) {
      setError("Účet s danou e-mailovou adresou již existuje");
      return;
    }

    const responseRegister = await axios.post("/api/clients", data);

    if (responseRegister.status === 200 || responseRegister.status ===
201){
      ev.target.reset();
      router.push("/account");
    }

  }catch(error){
    console.log("Error registering new admin: ", error);
  }

return;
}

```

Výpis 15: volání funkce pro uložení nového uživatele na stráně klienta

```

export async function POST(request){
  try{
    const { email, password, name, companyName, ic, dic, phone, street,
houseNumber, city, psc, isCompany} = await request.json();
    await connectToMongoose();
    const hashedPassword = await bcrypt.hash(password, 10);

    await Client.create({email,
      password: hashedPassword,
      name,
      companyName,
      ic,
      dic,
      phone,
      street,
      houseNumber,

```

```

        city,
        psc,
        isCompany});

    return NextResponse.json({message:"The registration information of
the new client was successfully saved"});

}catch(error){
    console.error('Error in POST /api/clients:', error);
    return NextResponse.json(
        { error: "Server error", details: error.message }
    )}
}

```

Výpis 16: ukázka funkce POST v souboru Clients

Po vytvoření účtu může uživatel prohlížet své osobní údaje a předchozí objednávky. Pro zobrazení údajů zákazníka byl použit výše zmíněný soubor ClientDataForm s parametrem režimu account a také API endpoint Clients, ve kterém byla definována asynchronní funkce PUT. Tato funkce umožňuje upravovat již existující záznam o zákazníkovi v databázi.

Obrázek 24: zobrazení uložených osobních údajů (zdroj: autor)

```

if(mode === "account"){
    try{
        const responseUpdate = await axios.put("/api/clients",
            {...data,_id}
        );

    }catch(error){
        console.log("Error updating personal data: ", error);
    }
}
}

```

Výpis 17: volání funkce pro úpravu uživatelských údajů

```

export async function PUT(request){
  try{
    await connectToMongoose();

    const {
      _id,
      email, name, companyName, ic, dic,
      phone, street, houseNumber, city,
      psc
    } = await request.json();

    const updatedClient = await Client.updateOne({_id: new
mongoose.Types.ObjectId(_id)},
    {email, name, companyName, ic, dic, phone, street,
    houseNumber, city, psc,
    });

    return NextResponse.json({success:true, client: updatedClient});

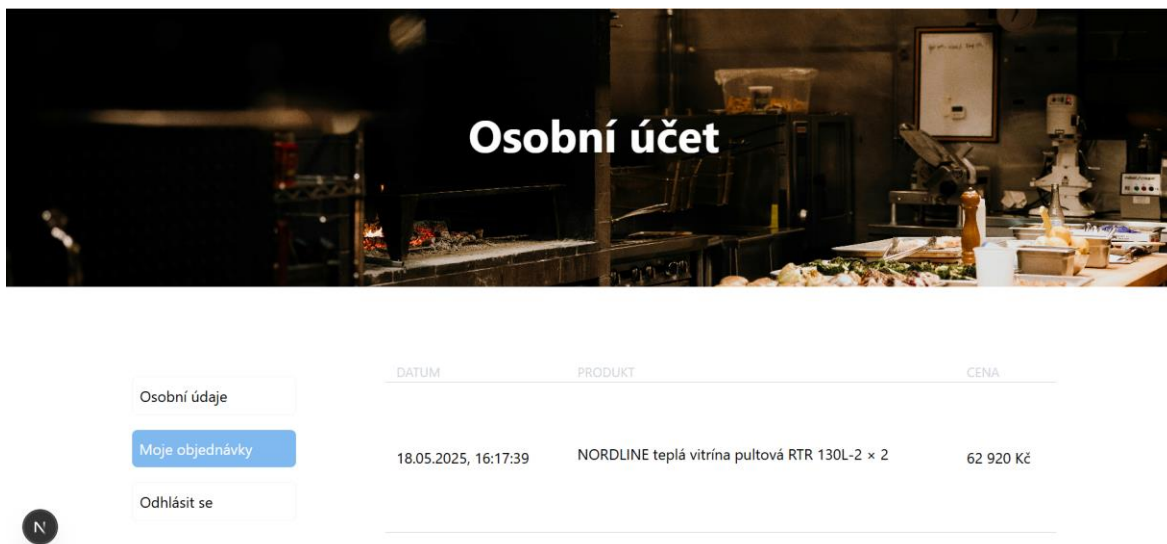
  }catch(error){
    console.error('Error in PUT /api/clients:', error);
    return NextResponse.json(
      { error: "Server error", details: error.message }
    );
  }
}

```

Výpis 18: ukázka funkce PUT v souboru Clients

Aby uživatel mohl zobrazit informace o předchozích objednávkách, z aktuální relace se bere identifikační číslo (ID) zákazníka. Následně se v databázi vyhledávají záznamy o objednávkách obsahující dané ID. K tomu se používá API endpoint Order a mongoose schéma Order, které byly vytvořeny pro správu objednávek.

V souboru Order byla použita asynchronní funkce GET. Funkce nejdříve vytváří připojení k databázi a následně ze vstupního požadavku získává parametr clientId. Pokud je clientId úspěšně nalezen, provádí se dotaz do databáze pomocí funkce find a vracejí se pouze záznamy objednávek obsahujících dané zákaznické ID.



Obrázek 25: zobrazení předchozích objednávek v osobním účtu (zdroj: autor)

```
export async function GET(request){
  try{
    await connectToMongoose();
    const url = new URL(request.url,"http://localhost:3003");
    const clientId = url.searchParams.get("clientId");

    if(clientId){
      const orders = await Order.find({clientId:clientId});
      if(!orders){
        return NextResponse.json({error:"The customer does not have
any orders yet"});
      }
      return NextResponse.json(orders);
    }
  }
}
```

Výpis 19: ukázka funkce GET v souboru Order

3.2.5 Seznam oblíbených produktů

Uživatelé webu mají možnost uložit si produkty, které se jim líbí, aby se k nim mohli později vrátit a rozhodnout se, zda je chtějí koupit. Funkce ukládání oblíbených produktů v tomto projektu byla vyřešena pomocí webové technologie Local Storage. Tato funkce umožňuje ukládat informace v prohlížeči uživatele a zajišťuje, že data o zvolených produktech zůstanou uložena i po opakovaném načtení stránky nebo zavření prohlížeče. Tento způsob byl zvolen také proto, aby se uživatel nemusel pro uložení produktů registrovat.

Local Storage se používá i v jiné části webové aplikace – pro ukládání produktů přidávaných do košíku. Aby se zabránilo opakování kódu, funkce pro práci s lokální pamětí byly přidány do vlastního React Hooku useLocalStorage.

Oblíbené



LENARI chladicí vitrina cukrářská LIMICOLA

Ventilované chlazení s odvlhčováním, vestavěný agregát, digitální termostat, automatické odtávání, automatické odpařování kondenzátů, 3 pevné chlazené police, nerezová...

164 550 Kč

Přidat do košíku

NORDline chladicí vitrina RT 400L-2

Jednodveřová vitrina, ventilované chlazení, automatické odtávání, digitální termostat, 4 nastavitelné skleněné police, trojitá izolační skla...

46 560 Kč

Přidat do košíku

LENARI distributor zmrzliny LIMOSA

Bez opláštění, ventilované chlazení, vestavěný agregát, digitální termostat, automatické odtávání pomocí horkých par, automatické odpařování kondenzátu, čelní izolační zvedací...

245 500 Kč

Přidat do košíku

Obrázek 26: stránka pro přidání oblíbených produktů (zdroj: autor)

```
export function useLocalStorage(key, initialValue){
  const [storedValue, setStoredValue] = useState(() => {

    if (typeof window === "undefined") return initialValue;

    const item = window.localStorage.getItem(key);
    return item? JSON.parse(item) : initialValue;
  });

  useEffect(() => {
    if (typeof window !== "undefined") {
      try {
        window.localStorage.setItem(key,
          JSON.stringify(storedValue));
      } catch (error) {
        console.error(" Failed to set new item in useLocalStorage:",
error);
      }
    }
  }, [key, storedValue]);
```

```

    return [storedValue, setStoredValue];
}

```

Výpis 20: ukázka kódu vlastního React hooku useLocalStorage

Seznam oblíbených produktů se ukládá v lokální paměti prohlížeče pod klíčem „wishlist“. Aby komponenty v celé aplikaci měly přístup k informacím o oblíbených produktech a k funkcím pro manipulaci s tímto seznamem, byl vytvořen Context API WishlistContext, který umožňuje sdílení globálního stavu. Daný soubor obsahuje stavovou proměnnou s ID jednotlivých produktů, funkci pro přidání produktů do seznamu oblíbených a funkci pro jejich odstranění.

```

export function WishlistContextProvider({children}){
  const [wishlistProducts, setWishlistProducts] =
    useLocalStorage("wishlist", []);

  function addToWishlist(productId){
    if(!productId) return;
    setWishlistProducts((prev) => {
      if (prev.includes(productId)) {
        return prev;
      }
      return [...prev, productId]
    });
  }

  function removeFromWishlist(productId){
    setWishlistProducts(prev =>{
      return prev.filter(id => id !== productId);
    });
  }

  return(
    <WishlistContext.Provider value={{wishlistProducts, setWishlistProducts,
    addToWishlist, removeFromWishlist, clearWishlist}}>
      {children}
    </WishlistContext.Provider>
  );
}

```

Výpis 21: ukázka kódu ze souboru wishlistContext

3.2.6 Nákupní košík

V této části webu mohou zákazníci přidávat produkty, které chtějí koupit. Přidání produktů do košíku a následné zpracování objednávky nevyžaduje registraci.

Pro uložení seznamu produktů v košíku byla v projektu použita již zmíněná technologie Local Storage a dříve vytvořený vlastní React hook `useLocalStorage`. Aby byly údaje o produktech v košíku dostupné všem komponentám v celé webové aplikaci, bylo vytvořeno Context API `CartContext`. K získání aktuální hodnoty z kontextu se v projektu používá React hook `useContext`. Soubor `CartContext` obsahuje globální proměnnou `cartProducts` a také funkce pro přidání produktu do košíku, funkci pro snížení množství vybraného produktu o jeden kus, funkci pro úplné odstranění konkrétní položky z košíku a funkci pro vyprázdnění celého košíku. Funkce přidávání a odebírání produktů může provádět zákazník, funkce úplného vyprázdnění košíku je prováděna aplikací v okamžiku úspěšného dokončení objednávky.

Aby se na stránce zobrazily všechny informace o produktech, včetně jejich názvů, cen a fotografií, z globální proměnné `cartProducts` se extrahují ID produktů. Po obdržení těchto údajů klientská strana odesílá na server požadavek na získání zbývajících informací o produktech. Pro větší efektivitu kódu byly tyto funkce přidány do vlastního React hooku `useCartProducts`.

Tento hook také obsahuje stavovou proměnnou `isClient`, která kontroluje, zda již byla komponenta vykreslena na straně prohlížeče. Proměnná se dále používá jako podmínka pro zobrazení informací o produktech. Tímto je v aplikaci vyřešen problém tzv. hydration error, který by mohl nastat, pokud by se komponenta generovala odlišně na straně klienta a na straně serveru a v důsledku toho by server vrátil jiný počet produktů.

Poslední proměnná, kterou hook `useCartProducts` vrací, je celková cena produktů, která se počítá jako součet cen všech produktů vynásobených počtem kupovaných kusů.

Nákupní košík

PRODUCT	CENA	MNOŽSTVÍ	CELKEM
 NORDLINE teplá vitrina pultová RTR 130L-2	15 730 Kč	- 2 +	31 460 Kč ×
 WANZL transportní vozík MUC 300	1 450 Kč	- 1 +	1 450 Kč ×

Shrnutí

3 produktů	32 910 Kč
CELKEM	32 910 Kč

Pokračovat

Obrázek 27: zobrazení produktů na stránce "Nákupní košík"

```

export function useCartProducts(cartProducts) {
  const [isClient, setIsClient] = useState(false);
  const [products, setProducts] = useState([]);

  useEffect(() => {
    setIsClient(true);
  }, []);

  useEffect(() => {
    if (cartProducts?.length > 0) {
      const validIds = cartProducts.filter((id) => id);
      axios.post("/api/products", { ids: validIds }).then((res) => {
        setProducts(res.data);
      });
    } else {
      setProducts([]);
    }
  }, [cartProducts]);

  const totalPrice = cartProducts.reduce((sum, productId) => {
    const product = products.find((p) => p._id === productId);
    return sum + (product?.price || 0);
  }, 0);

  return { isClient, products, totalPrice };
}

```

Výpis 22: ukázka kódu vlastního React hooku useCartProducts

3.2.7 Objednání zboží

Pro úspěšné provedení objednávky v internetovém obchodě je nutné, aby zákazník zadal potřebné údaje, jako je e-mail a adresa. K tomu se v aplikaci používá dříve vytvořený formulář ClientDataForm s parametrem mode="order".

Pokud má zákazník svůj účet a před nákupem se přihlásí, údaje ve formuláři se vyplní automaticky. K tomu se z aktuální relace získává ID zákazníka, které se poté odešle v dotazu do databáze. Databáze následně vrací záznam o daném zákazníkovi včetně všech uložených údajů.

Detaily objednávky

Osobní údaje

Email*

karolina.novotna89@hotmail.com

Fakturační adresa

☐ Nakupuji na firmu

Jméno a příjmení*

Karolína Novotná

Telefon*

+420 737 654 321

Ulice*

Kučerova

Číslo domu*

1276

Město*

Praha

PSČ*

190 00

Pokračovat

Shrnutí



NORDLINE teplá vitrína pultová RTR 130L-2

2 ks

15 730 Kč



WANZL transportní vozík MUC 300

1 ks

1 450 Kč

3 produktů

32 910 Kč

CELKEM

32 910 Kč

Obrázek 28: stránka pro zadání podrobností objednávky (zdroj: autor)

Poté, co zákazník zadá všechny potřebné informace, jsou tyto údaje spolu se seznamem kupovaných produktů odeslány na server. K vytvoření nové objednávky a následné inicializaci platební relace byl na cestě `app/api/order/route.js` vytvořen API endpoint.

Pro realizaci funkce platby v aplikaci bylo použito rozhraní Stripe API, vyvinuté společností Stripe. Služba Stripe umožňuje přijímání plateb, provádění fakturací, správu objednávek a další funkce. Jelikož se jedná o prototyp internetového obchodu, byl použit testovací režim platebního systému, který vytváří připojení pomocí testovacích API a poté umožňuje generovat fiktivní platby pomocí speciálních testovacích čísel platebních karet. V budoucnu bude pro přechod do produkčního režimu nutné ověřit účet ve Stripe a nahradit testovací API klíče produkčními.

Stripe vyžaduje, aby produkty v objednávce měly určitý formát, proto se v API endpointu Order pro každý produkt vytváří záznam v poli `line_items`, které obsahuje počet nakupovaných kusů, měnu, ve které bude probíhat platba, název produktu a jeho cenu uvedenou v nejmenší měnové jednotce.

Výsledné pole `line_items` se poté spolu s údaji o zákazníkovi ukládá do databáze pomocí mongoose schématu Order. Schéma zajišťuje, že do databáze budou uloženy pouze záznamy obsahující všechna povinná pole. Ve schématu je také stanoveno, že ke každému záznamu v dané kolekci musí být automaticky vygenerováno časové razítko.

Posledním krokem při implementaci platebního systému je vytvoření nové relace (checkout session) pomocí příkazu `stripe.checkout.sessions.create`. Součástí příkazu jsou následující parametry: režim relace (payment mode), zákaznický e-mail, URL adresa stránky, na kterou bude zákazník přesměrován v případě úspěšného dokončení platby, a URL adresa stránky pro přesměrování v případě zrušení platby.

```
const session = await stripe.checkout.sessions.create({
  line_items: lineItems,
```

```

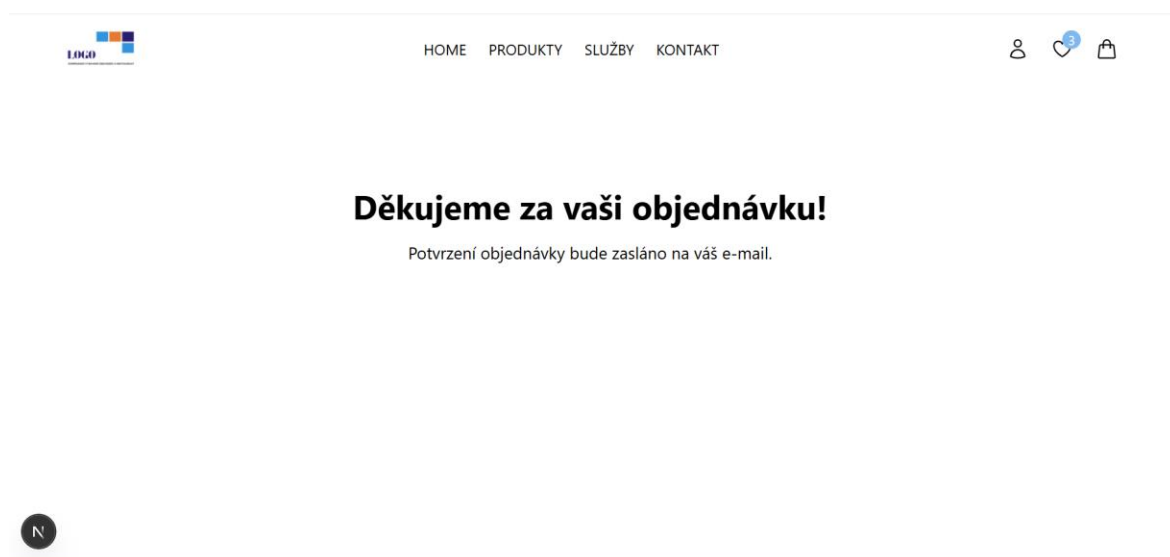
mode: "payment",
customer_email: email,
success_url: process.env.PUBLIC_URL + "/order?success=true",
cancel_url: process.env.PUBLIC_URL + "/order?canceled=true",
metadata: {orderId: orderDoc._id.toString()},
});

```

Výpis 23 : funkce pro vytvoření platební relace pomocí služby Stripe

Po úspěšném vytvoření platební relace je vrácena URL adresa stránky, na kterou má být zákazník přesměrován pro provedení platby.

V případě, že platba proběhla bez problémů, bude klient přesměrován na stránku potvrzující úspěšné dokončení nákupu.



Obrázek 29: stránka potvrzující úspěšné dokončení nákupu (zdroj: autor)

3.2.8 Rezervace termínu pro konzultaci

Jednou ze služeb, které tato firma nabízí, je konzultace klientských projektů. Za účelem optimalizace procesu rezervace konzultačního termínu byla na webových stránkách implementována speciální sekce obsahující rezervační formulář.

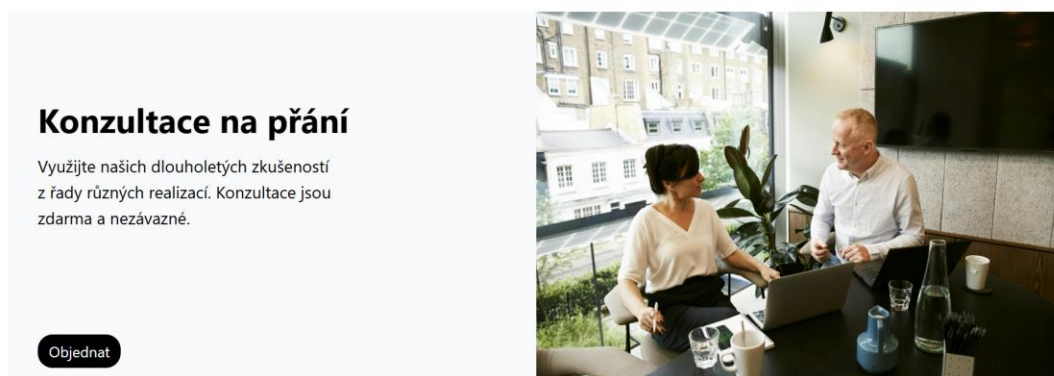
Klient si může zvolit preferovaný způsob konzultace – buď osobní setkání v kanceláři společnosti, nebo online schůzku prostřednictvím videohovoru. Formulář také nabízí možnost výběru času. Pro implementaci této funkce v projektu byla použita komponenta DatePicker z knihovny react-datepicker. Vybrané datum se uchovává ve stavové proměnné `selectedDate`. Uvnitř komponenty byly použity filtry, které omezují výběr termínů pouze na

pracovní dobu společnosti a znemožňují zákazníkům rezervovat termíny o víkendech nebo ve večerních hodinách.

Poslední částí formuláře jsou pole pro zadání informací potřebných k rezervaci konzultace: e-mailu, telefonního čísla a jména klienta. Tyto údaje se také ukládají do stavové proměnné. V případě, že některé pole nebude vyplněno, nebo nebude zvolen preferovaný způsob a čas konzultace, bude zákazník upozorněn chybovou zprávou.

Ke zpracování nových žádostí o konzultaci byl na cestě `app/api/consultation/route.js` vytvořen API endpoint. V daném souboru se pomocí asynchronní funkce `POST` údaje o nové naplánované konzultaci ukládají do databáze. V kódu bylo také předem nastaveno automatické odesílání e-mailů správci a klientovi pro potvrzení schůzky. Tato funkce byla implementována pomocí externí e-mailové služby Resend.

Momentálně se v aplikaci používá testovací režim služby, který umožňuje odesílat e-maily pouze jednomu příjemci. Před zveřejněním webové aplikace bude nutné provést ověření firemního účtu v systému Resend a změnit v kódu testovací e-mail na skutečné e-maily správce internetového obchodu a zákazníků.



Obrázek 30: UI komponenta sloužící k prezentaci konzultačních služeb (zdroj: autor)

Rezervace bezplatné konzultace

Typ schůzky



Setkání v naší kanceláři

Navštivte nás v kanceláři na
adrese: Tesařská 127/2, Kosmonosy 293
06, (okr. Mladá Boleslav)



Online schůzka

Zavoláme vám a probereme váš
projekt, ať už se nacházíte kdekoli.



Obrázek 31: uživatelské rozhraní pro výběr preferovaného způsobu konzultace (zdroj: autor)

Termín schůzky

21/05/2025 12:00

May 2025							Time
Su	Mo	Tu	We	Th	Fr	Sa	
27	28	29	30	1	2	3	09:00
4	5	6	7	8	9	10	10:00
11	12	13	14	15	16	17	11:00
18	19	20	21	22	23	24	12:00
25	26	27	28	29	30	31	13:00
							14:00
							15:00

is mohli kontaktovat a potvrdit vaši schůzku. Upozorňujeme, že všechna pole jsou povinná.

Email

Obrázek 32: uživatelské rozhraní pro výběr preferovaného času konzultace

Kontaktní údaje

Žádáme vás o kontaktní údaje, abychom vás mohli kontaktovat a potvrdit vaši schůzku. Upozorňujeme, že všechna pole jsou povinná.

Jméno

Email

Phone

☐ Zastupuji firmu

Odeslat

Obrázek 33: formulář pro zadání klientských údajů (zdroj: autor)

3.2.9 Stránka „Služby“

Stránka obsahuje stručný popis všech doplňkových služeb poskytovaných společností. Součástí stránky je také blok nabízející rezervaci konzultačních služeb. Vzhledem k tomu, že je tato sekce opakovaně využívána na různých stránkách webu, pro ni byla vytvořena samostatná komponenta na cestě `app/components/ConsultationHome.js`.

3.2.10 Stránka „Kontakt“

Tato stránka byla vytvořena, aby zákazníci mohli snadno kontaktovat společnost v případě dotazů nebo zájmu o spolupráci. Součástí stránky je kontaktní formulář pro odesílání zpráv, mapa zobrazující přesné umístění provozovny a sekce obsahující veškeré důležité kontaktní údaje, jako je název společnosti, adresa, telefonní číslo a e-mailová adresa.

Pro realizaci kontaktního formuláře byl vytvořen samostatný soubor s názvem `ContactForm`, který obsahuje rozhraní pro zadávání klientského jména, e-mailové adresy a textu zprávy.

Pro zpracování vstupů z formuláře byl také vytvořen API endpoint `Contact`, umístěný na adrese `app/api/contact/route.js`. Soubor přijímá data odeslaná z kontaktního formuláře a následně posílá e-mail s obsahem zákaznické zprávy na firemní e-mailovou adresu. K automatickému odesílání e-mailů byla použita již zmiňovaná e-mailová platforma `Resend`, která je integrována ve více částech webové aplikace.

4 Implementace řešení pomocí AI

V této části popsán proces vytvoření projektu s aktivním využitím umělé inteligence pro asistované programování. Autor použil systémy založené na velkých jazykových modelech k automatizovanému generování návrhů kódu v reálném čase, k tvorbě jednotlivých funkcí, komponent uživatelského rozhraní a konfiguračních souborů, k detekci a odstraňování chyb a k návrhům na vylepšení stávajícího kódu.

V kapitolách není popsána podrobná funkčnost a účel jednotlivých stránek webu, jelikož se tomu již podrobně věnovala kapitola o implementaci řešení bez použití umělé inteligence.

4.1 Postup řešení

V průběhu implementace řešení pomocí umělé inteligence autor zadával nástrojům pokyny ve formě promptů formulovaných v anglickém jazyce. Použití anglického jazyka v promptech bylo zvoleno s cílem dosažení větší efektivity a přesnosti při zpracování modelem vstupů ve formě přirozeného jazyka.

Následně byly vygenerované úryvky kódu integrovány do řešení. V některých případech navrhovaný kód neodpovídal požadovanému chování systému nebo způsobil v řešení výskyt chyb. V takových situacích se autor snažil opravit řešení pomocí iterativního zadání dodatečných podrobnějších promptů nebo opravoval kód ručně na základě vlastních zkušeností a znalostí.

4.2 Výběr nástrojů

Pro realizaci řešení pomocí nástrojů umělé inteligence byly vybrány platformy ChatGPT a GitHub Copilot. Hlavním faktorem při výběru nástrojů byla jejich široké rozšíření mezi vývojáři. Podle zprávy “State of Developer Ecosystem Report“ (JetBrains, 2024) na konci roku 2024 nástroj ChatGPT pravidelně využívalo 49 % respondentů a GitHub Copilot – 26% respondentů, což je činí nejčastěji využívanými systémy pro asistované programování.

Dalším faktorem byla cenová dostupnost nástrojů. Oba nástroje nabízejí v základní verzi omezený přístup zdarma. Měsíční předplatné rozšířené verze chatGPT k datu zpracování této práce je dostupné za 23 euro a měsíčné předplatné rozšířené verze GitHub Copilot činí 8,68 euro měsíčně. GitHub také nabízí speciální bezplatnou verzi pro studenty, učitele a správce populárních open-source projektů.

4.3 Panel administrátora

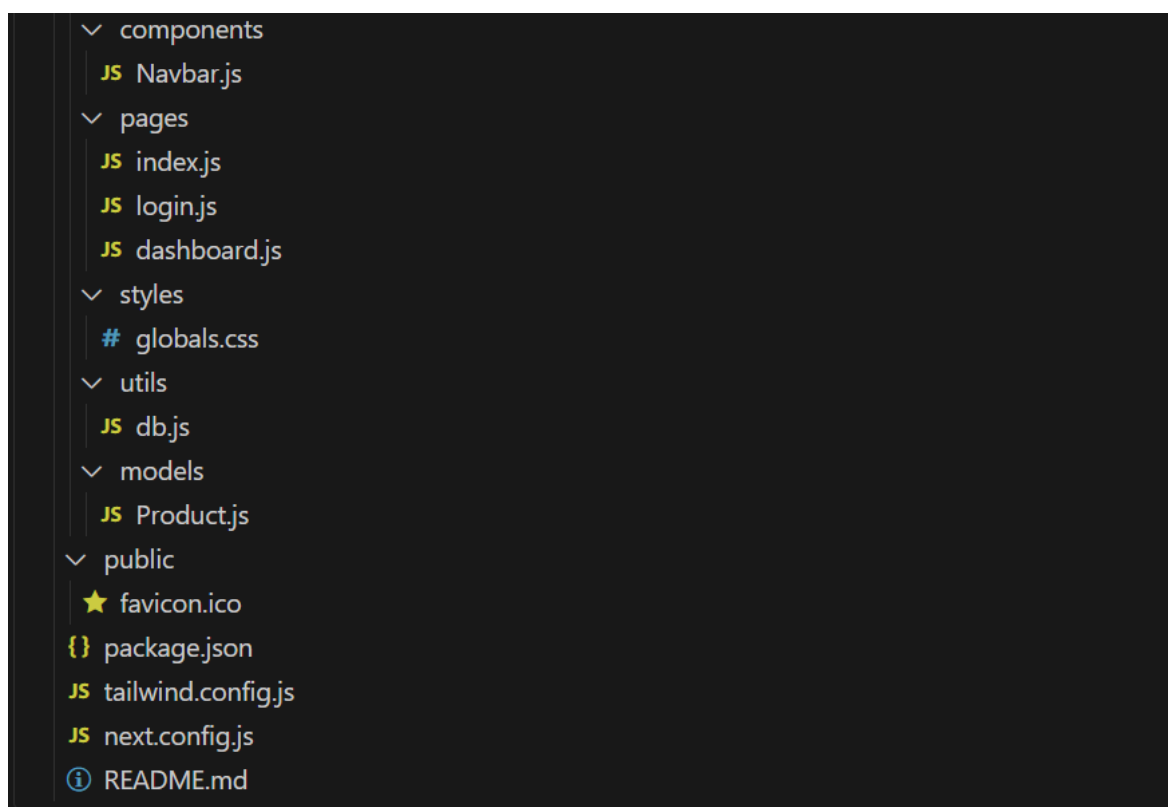
K vytvoření panelu administrátora byl využit nástroj GitHub Copilot. Pro jeho použití ve vývojovém prostředí, bylo nejprve nutné nainstalovat rozšíření „GitHub Copilot“ do prostředí Visual Studio Code a následně provést autentizaci pomocí uživatelského účtu na platformě GitHub.

4.3.1 Struktura souboru

Nástroj Copilot byl využit autorem ke generování výchozí struktury projektu a doporučení k instalaci potřebných závislostí. Za tímto účelem byl definován následující prompt:

I'm building an admin panel for an e-commerce web application from scratch using Node.js, Next.js, React.js, Tailwind CSS, MongoDB and Mongoose. Provide step-by-step guide on how to set up the project structure.

Níže je uvedena struktura souborů, která byla automaticky vytvořena nástrojem v kořenové složce /src.



Obrázek 34: struktura souborů navržená nástrojem GitHub Copilot (zdroj: autor)

Vygenerovaná struktura byla sama o sobě správná a zohledňovala použití uvedených technologií, ale odpovídá struktuře využívané ve starších verzích frameworku Next.js, kde se směrování realizovalo prostřednictvím tzv. Pages Router. Od verze Next.js 13.4, uvedené v květnu 2023, se doporučuje přístup s použitím tzv. App Router, takže navrhovanou strukturu lze považovat za neaktuální.

Prompt byl následně upraven tak, aby implicitně odkazoval na použití novější verze s App Routerem, ale vygenerovaná struktura stále neodpovídala požadovanému přístupu a obsahovala nevhodné složky a soubory.

Struktura byla poté manuálně přepracována autorem.

4.3.2 Připojení k databázi

Pro správu dat na webu byla použita databáze MongoDB a knihovna Mongoose. Níže je uveden text promptu, na základě kterého nástrojem byl vygenerován modul pro připojení k databázi, který obsahuje logiku pro inicializaci připojení a jeho opakované použití.

Create a JavaScript file within the /lib directory that establishes a connection to a MongoDB database using the mongoose library. The connection string should be loaded from the environment variable MONGO_URI defined in the .env.local file. The file should export an asynchronous function connectToMongoose, which handles the database connection logic.

Dále je uveden výstupní kód včetně automaticky generovaných komentářů, který byl následně využit v souboru mongoose.js.

```
import mongoose from 'mongoose';

const connectToMongoose = async () => {
  if (mongoose.connection.readyState === 1) {
    // Already connected
    return mongoose.connection;
  }

  try {
    const MONGO_URI = process.env.MONGODB_URI; // Load MONGO_URI from .env.local
    if (!MONGO_URI) {
      throw new Error('MONGODB_URI is not defined in .env.local');
    }

    await mongoose.connect(MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });

    console.log('Connected to MongoDB');
    return mongoose.connection;
  } catch (error) {
    console.error('Error connecting to MongoDB:', error);
    throw error;
  }
}
```



```

    }
  };

export { connectToMongoose };

```

Výpis 24: ukázka kódu pro připojení k databázi vygenerovaného nástrojem GitHub Copilot

4.3.3 Funkce Autentizace

Pro implementaci funkce autentizace byla použita knihovna Next.js. Pomocí GitHub Copilot byl vygenerován konfigurační soubor. Níže jsou uvedeny příklady vybraných použitých promptů.

Create a NextAuth configuration object using CredentialsProvider with email and password fields. Inside the provider, implement an asynchronous authorize function that connects to a MongoDB database and searches for a user in the admins collection by email.

In the authorize function, use bcrypt library to compare the entered password with the hashed password from the database.

Vygenerovaná řešení byla využita ve výsledném řešení. Níže je uveden kód konfiguračního souboru pro Next.js

```

import NextAuth from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";
import { MongoDBAdapter } from "@next-auth/mongodb-adapter";
import clientPromise from "@/lib/mongodb";
import bcrypt from "bcrypt";

const authOptions = {
  adapter: MongoDBAdapter(clientPromise),
  secret: process.env.NEXTAUTH_SECRET,
  session: {
    strategy: "jwt",
    maxAge: 30 * 24 * 60 * 60,
    updateAge: 24 * 60 * 60,
  },

  providers: [
    CredentialsProvider({
      name: "Credentials",
      credentials: {
        email: { label: "Email", type: "text", placeholder:
"admin@example.com" },
        password: { label: "Password", type: "password" },

```

```

    },
    async authorize(credentials) {
      const { email, password } = credentials;

      const client = await clientPromise;
      const db = client.db();

      const admin = await db.collection("admins").findOne({ email });
      if (!admin) throw new Error("Invalid email or password");

      const isValidPassword = await bcrypt.compare(password,
admin.password);
      if (!isValidPassword) throw new Error("Invalid email or password");

      return { id: admin._id, email: admin.email, name: admin.name };
    },
  })),
],
};

```

Výpis 25: ukázka kódu pro konfiguraci autentizace pomocí NextAuth.js vygenerovaného nástrojem GitHub Copilot

4.3.4 Správa dat pomocí CRUD operací

V dané webové aplikaci se používají 4 základní datové entity: produkty, kategorie, správci a objednávky. Pro manipulaci se těmito daty byly použity funkce CRUD (Create, Read, Update, Delete).

Níže je uveden příklad zadání použitého pro generování metody GET pracující s daty kategorií. Funkce slouží k získání z databáze dat, které se následně používají pro úpravu konkrétní kategorie nebo zobrazení seznamu všech kategorií uložených v databázi. Z tohoto důvodu funkčnost zahrnuje dva scénáře použití a liší se v závislosti na vstupních parametrech v požadavku.

Create an async GET API handler in Next.js that connects to a MongoDB database using a custom function connectToMongoose. If the request contains a query parameter id, fetch a single product by ID using the Product Mongoose model and populate its category field with the category name. If no ID is provided, fetch all products and populate their categories. Return JSON responses and HTTP status codes.

Níže je uveden příklad použití vygenerované funkce v projektu.

```

export async function GET(request) {
  try {
    await connectToMongoose();

    const { searchParams } = new URL(request.url);

```

```

const id = searchParams.get("id");

if (id) {
  // Fetch a specific product by ID
  const product = await Product.findById(id).populate("category",
"name");
  if (!product) {
    return NextResponse.json(
      { error: "Produkt nebyl nalezen." },
      { status: 404 }
    );
  }
}

```

Nástroj byl použit v projektu pro generování všech funkcí typu CRUD. Ve většině případů vygenerovaný kód vyžadoval drobné úpravy, protože nástroj ne vždy správně rozuměl kontextu z jiných souborů a funkce nemusela zcela odpovídat použitým v projektu datovým atributům nebo očekávanému formátu požadavku ze strany klienta.

Kromě generování nových částí kódu byl nástroj GitHub Copilot při vytváření řešení použit také k asistenci při revizi kódu, k vygenerování testovacích scénářů a k navržení možných způsobů vylepšení stávajícího kódu.

Níže je uveden vybraný prompt použitý pro realizaci ladění konkrétní funkce.

An uncaught error occurs in the /api/products endpoint during the handling of POST requests. Implement structured debugging in this file to identify the cause of the error .

```

export async function POST(req) {
  try {
    await connectToMongoose(); // Ensure the database is connected

    const body = await req.json();
    console.log("Incoming Request Body:", body); // Debugging: Log the
request body

    const { name, description, price, category, photos } = body;

    // Validate required fields
    if (!name || !price || !category) {
      console.error("Validation Error: Missing required fields"); //
Debugging
      return NextResponse.json(
        { error: "Missing required fields: name, price, or category." },
        { status: 400 }
      );
    }
  }
  // Create the product

```

```

    const newProduct = await Product.create({ name, description, price,
category, photos });
    console.log("Created Product:", newProduct); // Debugging: Log the
created product

    return NextResponse.json(newProduct, { status: 201 });
  } catch (error) {
    console.error("Error creating product:", error); // Debugging: Log the
error
    return NextResponse.json(
      { error: "Failed to create product." },
      { status: 500 }
    );
  }
}

```

Výpis 26: rozšíření kódu o diagnostické funkce pomocí nástroje GitHub Copilot

Po implementaci jednotlivých částí aplikace byla využita umělá inteligence k navržení testovacích scénářů. Níže je uveden příklad zadání zaměřeného na ověření funkcí souvisejících s přidáním nových produktů do databáze.

Create a set of test cases for adding a new product through the admin panel. The form is implemented in component ProductForm, and the server-side logic is handled in file /api/products.

Nástroj GitHub Copilot vygeneroval 10 testovacích scénářů. Každý scénář obsahoval popis případu, seznam kroků, které je třeba provést a definici očekávaného výsledku. S ohledem na rozsah práce níže jsou uvedeny pouze jejich stručné popisy.

Testovací případy pro soubor ProductForm:

- úspěšné odeslání přidání produktů
- chybějící povinná pole
- zadání neplatné ceny
- nahrání fotografie
- odstranění fotografie.

Testovací případy pro endpoint /api/products:

- úspěšné vytvoření produktu
- chybějící povinná pole
- zadání neplatné ceny
- nahrání fotografie
- duplicitný název produktu.

Nástroj byl také použit ke generování návrhů na vylepšení již napsaného kódu.

What improvements can be made to the code responsible for adding new product data and storing it in the MongoDB database?

Níže je uveden seznam stručně popsanych vygenerovaných návrhů týkající se komponenty ProductForm a API endpointu /api/products. K některým z nich nástroj nabídl také ukázky toho, jak by mohl vylepšený kód vypadat.

Nabídnutá vylepšení:

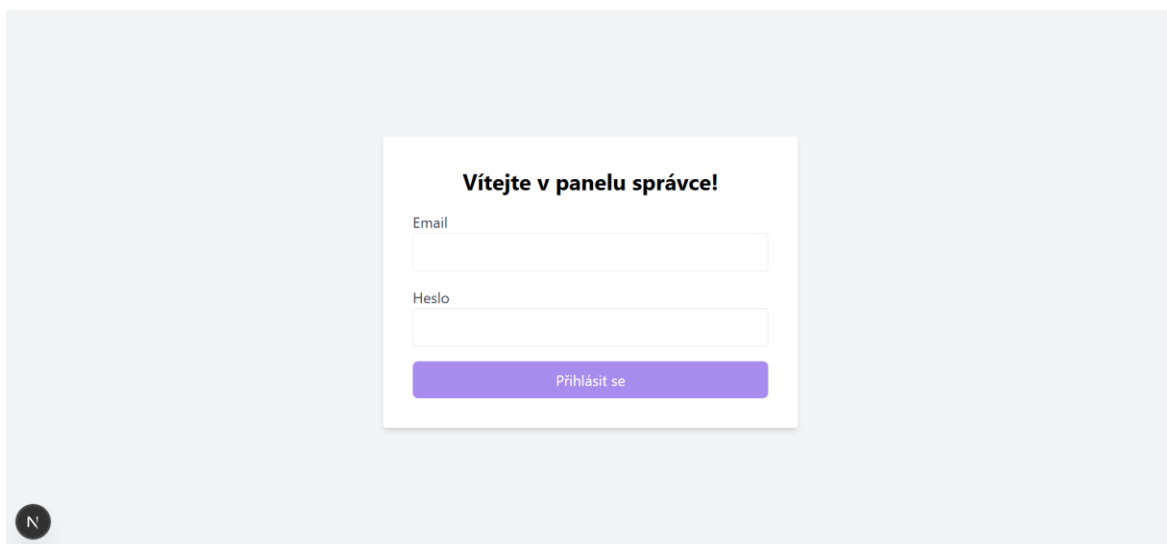
- Přidání validace vstupních dat do API Route souboru.
- Doplnění validace vstupních dat na stráně klienta před jejich odesláním na server.
- Přidání ošetření chyb na stráně klienta při nahrávání fotografií.
- Přesunutí proměnných používaných pro připojení k externím službám do seznamu globálních proměnných.
- Přidání komentářů pro vysvětlení účelu každé sekce.
- Použití výstižných názvů proměnných.

Všechny návrhy byly relevantní a vztahovaly se k obecně přijímaným požadavkům na kvalitu realizace softwarového projektu. Navržené části kódu však nebyly vždy v souladu se strukturou celého souboru a logikou již naprogramovaných funkcí, proto jejich přímá implementace do kódu by mohla narušit fungování aplikace. Z tohoto důvodu dané kódové ukázky byly použity autorem pouze za účelem inspirace.

4.3.5 Vytváření komponent uživatelského rozhraní

Při vytváření vizuálních prvků webové aplikace bylo cílem dodržet již vybrané designové řešení, které bylo použito při tradičním způsobu vývoji. Ve výsledném řešení byly ponechány malé změny v návrhu v případech, kdy se autor posoudil, že vygenerované řešení je vhodnější z praktických nebo estetických důvodů.

Níže jsou uvedeny ukázky vizuálního vzhledu vybraných stránek výsledné webové aplikace.



Obrázek 35: stránka pro přihlášení vytvořená pomocí GitHub Copilot (zdroj: autor)

 **testAdmin**
testadmin@example.com

[Produkty](#)

[Kategorie](#)

[Objednávky](#)

[Správci](#)

[Odhlásit se](#)

Dobrý den, testAdmin!
neděle 22. června 2025

Objednávky

DATUM	ZÁKAZNÍK	OBSAH OBJEDNÁVKY	PLATBA
22. 06. 2025	Martin Král martin.kral@example.cz	LENARI distributor zmrzliny LIMOSA x 2	✓
22. 06. 2025	Anna Procházková anna.prochazkova87@example.cz	WANZL prodejní stůl PS x 1	✓

Obrázek 36: stránka "Objednávky" vytvořená pomocí GitHub Copilot (zdroj: autor)

The image shows a web application interface for adding a new product. On the left is a sidebar with a user profile for 'testAdmin' and navigation links: 'Produkty' (highlighted), 'Kategorie', 'Objednávky', 'Správci', and 'Odhlásit se'. The main area is titled 'Vytvořit nový produkt' and contains several form fields: 'Název produktu' (with placeholder 'Zadejte název'), 'Popis' (with placeholder 'Popis produktu'), 'Cena' (with placeholder 'Cena v Kč'), and a 'Kategorie' dropdown menu (with placeholder 'Vyberte kategorii'). Below these is a 'Fotografie' section with a plus icon in a square box. At the bottom of the form is a purple button labeled 'Přidat produkt'. The top of the page shows a user profile for 'testAdmin' and a greeting 'Dobrý den, testAdmin!' with the date 'pondělí 23. června 2025'.

Obrázek 37: stránka pro přidání nového produktu vytvořená pomocí GitHub Copilot (zdroj: autor)

Vytváření vizuálních prvků pomocí nástroje GitHub Copilot bylo pro autora z hlediska vývojářské zkušenosti poměrně složitým procesem, protože nástroj nepodporuje generování kódu na základě poskytnutých uživatelem obrázků. Z tohoto důvodu pro dosažení konkrétního vizuálního návrhu musel autor velmi podrobně popisovat v zadání vnější charakteristiky každého prvku, jako jsou barva, velikost, odsazení a interaktivní chování. V takovém případě může formulace přesného promptu trvat stejně dlouho jako ruční stylizace prvků.

4.4 Stránky e-shopu

K realizaci webových stránek e-shopu byl využit nástroj ChatGPT, dostupný prostřednictvím webového rozhraní.

Struktura a funkčnost jednotlivých stránek webu jsou popsány v kapitole věnované tradičnímu přístupu k vývoji e-shopu. Tato kapitola se zaměřuje pouze na vybrané části aplikace vytvořené s využitím nástroje ChatGPT.

4.4.1 Použití LocalStorage

Webová aplikace využívá lokální paměť k ukládání seznamů produktů přidáných uživateli do nákupního košíku a seznamu oblíbených produktů. Níže je uvedeno zadání použité pro vygenerování kódu pro custom hook sloužící k práci s localStorage v rámci Reactu.

Implement a custom React hook named useLocalStorage for storing and retrieving data from localStorage. The hook will be used to manage a list of products in both the cart and wishlist. Ensure that the implementation includes a check for typeof window !== "undefined" before accessing or modifying localStorage.

Níže je uveden kód vygenerovaný nástrojem ChatGPT podle zadaného promptu.

```
import { useState, useEffect } from 'react';

export function useLocalStorage(key, initialValue) {
  const isClient = typeof window !== 'undefined';

  const [storedValue, setStoredValue] = useState(() => {
    if (!isClient) return initialValue;
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.error('Error reading localStorage key "' + key + '": ',
error);
      return initialValue;
    }
  });

  useEffect(() => {
    if (!isClient) return;
    try {
      window.localStorage.setItem(key, JSON.stringify(storedValue));
    } catch (error) {
      console.error('Error setting localStorage key "' + key + '": ',
error);
    }
  }, [key, storedValue, isClient]);

  return [storedValue, setStoredValue];
}
```

Obrázek 38: ukázka kódu vlastního React hooku useLocalStorage vygenerovaného nástrojem ChatGPT

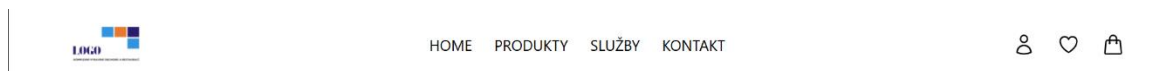
Před zařazením kódu autor provedl jednu úpravu: podmínka, která je ve vygenerovaném kódu umístěna mimo samotný hook, byla přesunuta dovnitř hooku useEffect s cílem předejít možným problémům typu „hydration mismatch“.

4.4.2 Vytvoření prvků uživatelského rozhraní

Nástroj ChatGPT se ukázal jako užitečný při generování vizuálních prvků, jelikož chatbot je schopen rozpoznat obrázky nahrané uživatelem do konverzace a může vygenerovat kód pro prvek s odpovídajícím designem.

Create a file containing the Header component. The design of the component should match the layout and style shown in the provided image.

K zadání byl přidán obrázek znázorňující požadovaný design.



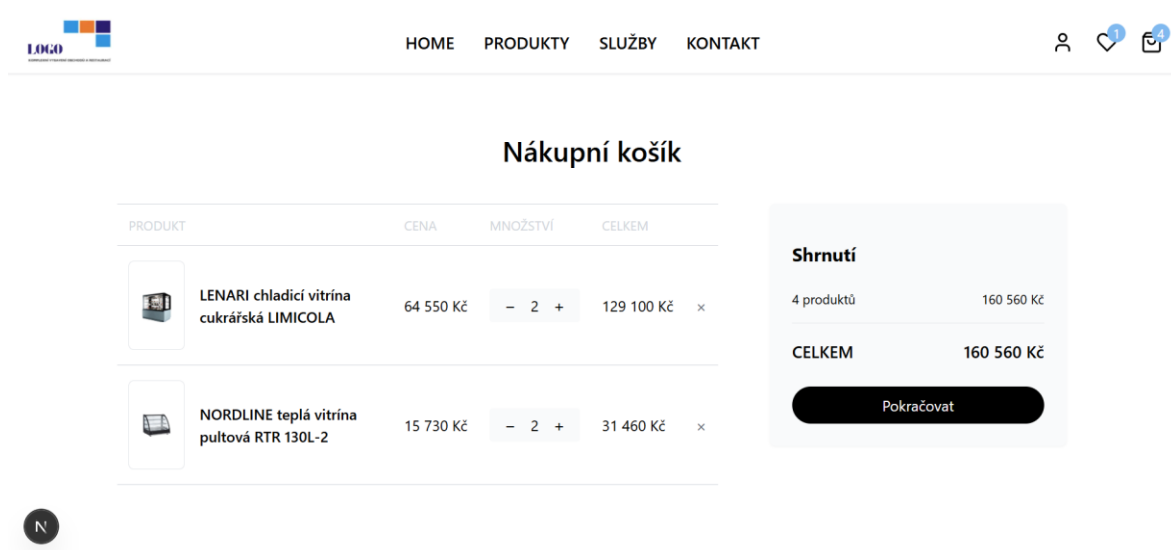
Obrázek 39: obrázek poskytnutý nástroji ChatGPT jako vzor (zdroj: autor)

Vygenerovaný prvek byl prakticky identický s prvkem, který byl zadán nástroji jako vzor. Níže je zobrazen vzhled vygenerovaného prvku.

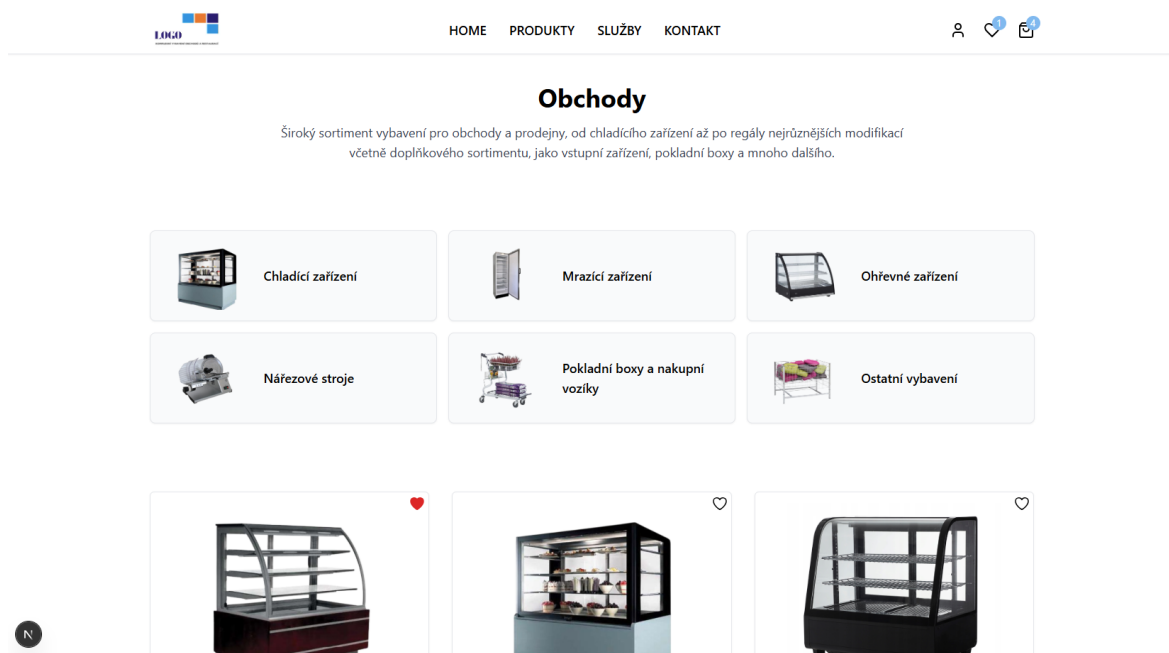


Obrázek 40: vzhled prvku Header vytvořeného pomocí ChatGPT (zdroj: autor)

Při vytváření vizuálního vzhledu stránek se autor snažil dodržet dříve zvolený návrh jednotlivých stránek. Výsledný design generovaných prvků se na některých stránkách mírně lišil. Stejně jako u panelu správce byly návrhy vygenerované AI ponechány v případech, kdy jej autor považoval za vhodnější.



Obrázek 41: stránka "Nákupní košík" vytvořená pomocí ChatGPT (zdroj: autor)



Obrázek 42: stránka s přehledem produktů vybrané kategorie vytvořená pomocí ChatGPT (zdroj: autor)

5 Porovnání odlišných metod vývoje konkrétního webu

5.1 Definice kritérií

Na základě literární rešerše a konzultací s vlastníkem společnosti byla pro účely porovnání pomocí AHP metody vybrána následující kritéria:

Kvalita kódu

Dané kritérium se skládá ze tří ukazatelů:

- **Spolehlivost:** Ukazatel měří počet nalezených chyb, které mohou potenciálně způsobit selhání systému.
- **Udržitelnost:** Metrika ukazuje míru technického dluhu a tzv. code smells neboli indikátorů špatného návrhu kódu.
- **Bezpečnost:** Metrika vyjadřuje míru identifikovaných zranitelností.

Vyhodnocení přístupu na základě kritéria bude vyhodnoceno pomocí nástroje SonarQube, určeného k automatizované kontrole kvality.

Uživatelská přívětivost

Kritérium se skládá ze dvou podkritérií:

- **Výkonnost:** Metrika vyjadřuje rychlost načítání a interakce webu.
- **Přístupnost:** Kritérium sleduje kontrasty barev, přítomnost alternativních textů u multimediálního obsahu nebo správné použití HTML značek.

Kritérium bylo hodnocen na základě analýzy realizované prostřednictvím nástroje Google Lighthouse, který je zaměřen na měření kvality interakcí uživatelů s webovými stránkami.

Časová náročnost

Kritérium ukazuje, kolik času trvala realizace celého projektu. V dané práci měření času bylo provedeno pomocí analytického nástroje WakaTime, určeného k monitorování produktivity vývojářů.

Míra potřebné odbornosti

Kritérium reprezentuje úroveň dovedností a znalostí v oblasti vývoje webových aplikací, jimiž musí vývojář disponovat pro aplikaci konkrétního přístupu. Hodnocení bude přiděleno subjektivně na základě dokumentovaných zkušeností autora s použitím obou přístupů.

Míra potřebné odbornosti

Kritérium reprezentuje úroveň dovedností a znalostí v oblasti vývoje webových aplikací, jimiž musí vývojář disponovat pro aplikaci konkrétního přístupu. Hodnocení bude přiděleno subjektivně na základě dokumentovaných zkušeností autora s použitím obou přístupů.

Vývojářská zkušenost

Kritérium představuje subjektivní hodnocení spokojenosti vývojáře s jednotlivými přístupy. Budou zohledněny faktory jako pracovní komfort, plynulost vývojového procesu a mentální náročnost spojená s realizací projektu. Hodnocení bude vycházet z osobních zkušeností autora s implementací řešení pomocí obou přístupů.

5.2 Výpočet vah kritérií

Určení vah jednotlivých kritérií bylo provedeno pomocí metody AHP Saatyho. V rámci párového porovnání každému kritériu byla přiřazena číselná hodnota ze stupnice od 1 do 9, která ukazuje, do jaké míry je jedno kritérium důležitější než druhé. Při srovnání důležitosti kritérií se autor opíral především o poznatky získané v průběhu literární rešerše..

Kritéria	Kvalita	Uživat. přívětivost	Čas. náročnost	Potřeb. odbornost	Vývojář. zkušenost	Geom. průměr	Váha
Kvalita kódu	1	2	3	5	3	2.460	0.40
Uživatelská přívětivost	1/2	1	3	3	3	1.683	0.28
Časová náročnost	1/3	1/3	1	3	2	0.922	0.15
Míra potřebné odbornosti	1/5	1/3	1/3	1	1/2	0.407	0.07
Vývojářská zkušenost	1/3	1/3	1/2	2	1	0.644	0.11

Tabulka 1: Výpočet vah jednotlivých kritérií (zdroj: autor)

Na základě provedeného párového porovnání kritérií metodou AHP Saatyho bylo jako nejvýznamnější identifikována kvalita kódu, jako druhé nejdůležitější kritérium se ukázala uživatelská přívětivost. Nejmenší váhu v důsledku porovnání získala míra potřebné odbornosti.

Ověření konzistence párového porovnání

Vypočtený poměr konzistence matice činí přibližně 1.3 %. Taková hodnota indexu konzistence je považována za přijatelnou a znamená, že přiřazování hodnocení v rámci jednotlivých párů bylo provedeno logickým a systematickým způsobem.

5.3 Hodnocení přístupů v rámci vybraných kritérií

Tato kapitola popisuje proces hodnocení dvou vybraných přístupů k tvorbě webových aplikací – tradiční vývoj a vývoj pomocí nástrojů založených na umělé inteligenci podle předem definovaných kritérií: kvality kódu, uživatelské přívětivosti, časové náročnosti, míry potřebné odbornosti a vývojářské zkušenosti.

5.3.1 Kvalita kódu

Pro hodnocení kritéria kvality kódu byla provedena kontrola dvou výsledných webů – panel správce a stránek pro e-shop vyvinutých v rámci obou přístupů pomocí nástroje SonarQube.

V rámci analýzy spolehlivosti, bezpečnosti a udržitelnosti kódu nástroj poskytuje počet nalezených problémových míst. Nástroj také přiděluje souborům hodnocení na stupnici od A do E v závislosti na závažnosti nalezených problémů podle následujícího systému:

- A – žádný problém nebo pouze problémy informativního charakteru.
- B – alespoň jeden problém s nízkou prioritou.
- C – alespoň jeden problém se střední prioritou.
- D – alespoň jeden problém s vysokou prioritou.
- E – alespoň jeden problém blokující další postup.

Pro posouzení udržitelnosti webu nástroj navíc měří úroveň technického dluhu nalezeného v kódu a vystavuje hodnocení podle následující stupnice:

- A – technický dluh tvoří méně než 5 % vývoje.
- B – technický dluh tvoří od 6 do 10 %.
- C – technický dluh tvoří 11–20 %.
- D – technický dluh tvoří od 21 do 50 %.
- E – technický dluh tvoří více než 50 %.

Níže je uveden přehled hlavních výsledků analýzy.

Obecné hodnocení

Panel správce	Bezpečnost	Spolehlivost	Udržitelnost
Tradiční vývoj	A	C	A
Pomocí AI nástrojů	A	C	A

Tabulka 2: Skóre kvality stránek administračního rozhraní stanovené nástrojem SonarQube (zdroj: autor)

E-shop	Bezpečnost	Spolehlivost	Udržitelnost
Tradiční vývoj	A	C	A
Pomocí AI nástrojů	A	C	A

Tabulka 3: Skóre kvality stránek e-shopu stanovené nástrojem SonarQube (zdroj: autor)

Počet nalezených problémů

Panel správce	Bezpečnost	Spolehlivost	Udržitelnost	Technický dluh
Tradiční vývoj	0	39	87	0.7%
pomocí AI nástrojů	0	17	28	0.2%

Tabulka 4: Zjištěné problémy v kódu administračních panelů podle analýzy nástroje SonarQube (zdroj: autor)

E-shop	Bezpečnost	Spolehlivost	Udržitelnost	Technický dluh
Tradiční vývoj	0	90	107	0.6%
pomocí AI nástrojů	0	67	68	0.4%

Tabulka 5: Zjištěné problémy v kódu e-shopů podle analýzy nástroje SonarQube (zdroj: autor)

Při analýze bezpečnosti dosáhly oba přístupy stejných výsledků, proto nebyl toto dílčí kritérium při srovnání zohledněn.

Technický dluh se u obou přístupů pohybuje na velmi nízké úrovni, kterou lze v kontextu analýzy kvality kódu nástrojem SonarQube považovat za statisticky nevýznamnou.

Kód obou webových stránek vytvořených pomocí AI nástrojů obsahoval o něco méně problémů než řešení realizovaná tradičním způsobem. Avšak tento rozdíl v kombinaci s nízkým dopadem problémů na technický dluh je nevýrazný. Na základě výsledků analýzy autor přidělil následující hodnocení:

- **Tradiční vývoj:** 5.
- **Vývoj pomocí AI nástrojů:** 6.

5.3.2 Uživatelská přívětivost

Analýza uživatelské zkušenosti byla provedena pomocí nástroje Google LightHouse. Výsledkem analýzy je přiřazení nástrojem jednotlivým kritériím číselného skóre od 1 do 100. Výsledné hodnocení lze interpretovat pomocí následující stupnice:

- 0-49 – nízká kvalita.
- 50-89 – průměrná kvalita.
- 90-100 – vysoká kvalita.

Pro porovnání uživatelské přívětivosti e-shopu byla vybrána hlavní stránka webu, protože se jedná o první stránku, se kterou uživatelé interagují po příchodu na web. Pro porovnání dvou implementací panelu správce byla vybrána stránka pro správu produktů.

Níže jsou prezentovány výsledky analýzy.

Panel správce	Výkonnost	Přístupnost
Tradiční vývoj	63	86
Pomocí AI nástrojů	72	92

Tabulka 6: Skóre uživatelské přívětivosti stránek administračního rozhraní stanovené nástrojem Google Lighthouse (zdroj: autor)

E-shop	Výkonnost	Přístupnost
Tradiční vývoj	49	84
Pomocí AI nástrojů	88	92

Tabulka 7: Skóre uživatelské přívětivosti stránek e-shopu stanovené nástrojem Google Lighthouse (zdroj: autor)

Výsledek analýzy ukázal velký rozdíl ve výkonnosti hlavní stránky e-shopu. Skóre dosažené v případě tradičního způsobu vývoje je velmi nízké a spadá do kategorie nízké kvality. Ukazatel výkonnosti stránky administrátorského panelu je v případě vývoje s AI také o něco vyšší.

Z hlediska přístupnosti mají weby vytvořené bez použití AI o něco nižší skóre a spadají do kategorie průměrné kvality.

Na základě výsledků analýzy autor přidělil následující hodnocení:

- **Tradiční vývoj: 4.**
- **Vývoj pomocí AI nástrojů: 7.**

5.3.3 Časová náročnost

Pro měření času stráveného realizací celého projektu byl použit nástroj Waketime. Tabulka níže ukazuje přibližný čas strávený vývojem obou webů při použití tradičního přístupu a při programování s pomocí AI nástrojů..

	Panel správce	E-shop
Tradiční vývoj	28 hodin	39 hodin
Pomocí AI nástrojů	14 hodin	12 hodin

Tabulka 8: Časová náročnost obou přístupů měřený nástrojem WakaTime (zdroj: autor)

Vývoj s podporou AI se ukázal jako mnohem efektivnější z hlediska časové náročnosti. V případě realizace administrátorského panelu vývoj pomocí AI byl přibližně dvakrát rychlejší a v případě e-shopu přibližně třikrát rychlejší než tradiční přístup.

Výsledné hodnocení dvou přístupů:

- **Tradiční vývoj: 4.**
- **Vývoj pomocí AI nástrojů: 9.**

5.3.4 Míra potřebné odbornosti

Z vlastní zkušenosti s realizací řešení pomocí obou přístupů autor dospěl k závěru, že vývoj tradiční metodou vyžaduje mírně větší úroveň technických znalostí a dovedností. Vývojář

musí být schopen samostatně identifikovat a řešit vznikající chyby a také detailně znát API používaných frameworků a knihoven.

Zkušenosti autora ukazují, že vývoj pomocí AI rovněž vyžaduje vysokou úroveň technických znalostí. Ačkoli AI nástroj může programátorovi pomoci se zorientovat v nové technologii, výstupy generované AI vyžadují neustálou průběžnou kontrolu a opravy. Z tohoto důvodu je nutné, aby vývojář dobře rozuměl všem aspektům realizace projektů a mohl posoudit správnost a vhodnost navrhovaného řešení.

Autorem přiřazené hodnocení:

- **Tradiční přístup:** 3.
- **Vývoj pomocí AI nástrojů:** 4.

5.3.5 Vývojářská zkušenost

Na základě vlastních zkušeností autora subjektivně ohodnotil proces tradičního vývoje jako plynulejší a příjemnější. Za hlavní výhodu daného přístupu autor považuje lepší orientaci ve struktuře projektu, porozumění implementované logice a způsobům propojení jednotlivých částí aplikace.

Ačkoli nástroje založené na umělé inteligenci umožňují rychlejší generování výchozího kódu, podle zkušenosti autora vyžadoval vývoj pomocí AI mnohem více času na následné odstraňování chyb a integraci jednotlivých částí aplikace. Využití nástrojů pro generování kódu také vedlo k tomu, že autor měl větší potíže s orientací v kódu, což zvyšovalo mentální zátěž a dodatečně komplikovalo proces řešení vznikajících problémů.

Porovnáváním přístupům byly přiřazeny následující hodnocení:

- **Tradiční přístup:** 7.
- **Vývoj pomocí AI nástrojů:** 3.

5.4 Výpočet výsledného hodnocení

Tabulka níže obsahuje přehled všech hodnocení přidělených dvěma uchazečům v rámci kapitoly „Hodnocení přístupu na základě vybraných kritérií“.

Kritéria	Kvalita	Uživat. Přívětivost	Čas. náročnost	Potřeb. odbornosti	Vývojář. zkušenost
Tradiční vývoj	5	4	4	3	7
Pomocí AI nástrojů	6	7	9	4	3

Tabulka 9: Výsledné hodnocení obou přístupů (zdroj: autor)

K určení celkového hodnocení jednotlivých variant byl použit výpočet váženého součtu.

Výsledné scóre jednotlivých přístupů k vývoji webových aplikací:

- **Tradiční vývoj:** 4,65.
- **Vývoj pomocí AI nástrojů:** 6,28.

Závěr

Hlavním cílem práce bylo porovnání dvou přístupů k vývoji webových aplikací – tradičního přístupu a programování s využitím systémů založených na umělé inteligenci.

První kapitola práce byla věnována literární rešerši. Autor nejprve definoval pojem webová aplikace a popsal její základní typy. Následně se autor zaměřil na problematiku spojenou s vývojem webových aplikací, včetně specifikace požadavků, modelování softwarových procesů a přehledu existujících vývojových metodik. Druhá část teoretické části se zabývala oblastí velkých jazykových modelů. Byl popsán jejich historický vývoj, základní principy fungování a představení vybraných nástrojů, postavených na těchto modelech, které lze využít za účelem asistence při programování.

Druhá kapitola se zabývala návrhem softwarového řešení pro malou českou společnost, která se specializuje na vybavení obchodů a restaurací. Navržený systém se skládal ze dvou částí – webových stránek pro e-shop a administračního panelu pro správu obsahu.

Ve třetí kapitole autor zdokumentoval proces vývoje navrhovaného řešení bez použití umělé inteligence. Implementace byla realizována v jazyce JavaScript s využitím frameworku Next.js, knihovny React.js a databázového systému MongoDB. Následně bylo totožné řešení s využitím dvou nástrojů založených na umělé inteligenci – ChatGPT a GitHub Copilot.

Závěrečná kapitola se zabývala porovnáním dvou použitých vícekritériální metody AHP. Autor hodnotil oba přístupy podle pěti předem stanovených kritérií: kvalita kódu, uživatelská přívětivost, časová náročnost, míra potřebné odbornosti a zkušenost vývojáře. Pomocí párového porovnání autor přiřadil jednotlivým kritériím váhy a následně vyhodnotil oba přístupy na základě měřitelných metrik a vlastních zkušeností s vývojem v rámci experimentů.

Výsledky analýzy ukázaly, že využití systémů založených na umělé inteligenci při vývoji webových aplikací může přinést do procesu vývoje řadu výhod, zejména v oblasti časové efektivity. V rámci analýzy však byly identifikována určitá omezení přístupu, která by měli vývojáři zohlednit při rozhodování o integraci těchto nástrojů do vlastního vývojového procesu.

Diskuse

Pro výběr nejlepšího přístupu k vývoji webové aplikace autor použil metodu AHP a pět předem definovaných kritérií: kvalitu kódu, uživatelskou přívětivost výsledných řešení, časovou náročnost, míru potřebné odbornosti a zkušenost vývojáře. Na základě provedené komparativní analýzy byl jako efektivnější vyhodnocen přístup programování s využitím nástrojů umělé inteligence.

Analýza výsledných řešení prokázala, že využití nástrojů umělé inteligence v procesu vývoje může výrazně přispět ke zkrácení celkové doby potřebné pro dodání softwarového produktu. Webové stránky vytvořené pomocí AI také dosahovaly vyššího hodnocení v oblasti uživatelské přívětivosti, zejména při měření jejich výkonnosti. Daný výsledek ukazuje, že AI nástroje automaticky uplatňují tzv. best practices neboli osvědčené vývojové postupy, jako je komprese přenášených souborů, minimalizace velikosti DOM a efektivní práce s CSS třídami. Z tohoto důvodu mohou být tyto nástroje vhodné také pro optimalizaci stávajícího kódu, zaměřenou na zvýšení výkonnosti webových stránek.

Z hlediska kvality kódu dosáhl přístup využívající AI mírně lepších výsledků ve srovnání s tradičním vývojem. Rozdíl však nebyl zásadní, což může naznačovat, že měřené metriky jako je bezpečnost, udržitelnost a spolehlivost softwarového řešení nejsou ovlivněny samotným přístupem k psaní kódu, nýbrž vhodně navrženou strukturou projektu a promyšlenou systematickou organizací jednotlivých částí aplikace.

Při provedení praktického experimentu se nástroje umělé inteligence ukázaly se jako efektivní zejména při generování standardizovaných částí webových aplikací jako jsou konfigurační soubory pro připojení k databázi, operace typu CRUD nebo integrace externích služeb. Využití AI nástroje se také osvědčily jako přínosné při generování při generování testovacích případů nebo tvorbě návrhů na vylepšení stávajícího kódu. Přesto je nutné zmínit určitá omezení spojená s využitím umělé inteligence při vývoji webových aplikací:

- AI modely mohou mít omezený přístup k aktuálním datům v důsledku tzv. cut-off date.
- AI Nástroje ukázaly omezenou schopnost porozumět globálním kontextu projektů a vztahům mezi jednotlivými částmi.
- Efektivní využití umělé inteligence vyžaduje od vývojáře srovnatelnou úroveň technických znalostí s tradičním přístupem.
- Použití AI při komplexních úkolech může mít negativní dopad na vývojářskou zkušenost.
- S využitím AI jsou spojena i další specifická rizika, jako jsou například otázky týkající se ochrany osobních údajů a duševního vlastnictví.

Na závěr lze konstatovat, že využití nástrojů založených na umělé inteligenci může významně přispět ke zvýšení efektivity vývoje, avšak jejich integrace do vývojového procesu konkrétního projektu vyžaduje důkladnou analýzu přínosů a potenciálních omezení a rizik.

Omezení práce

Tato práce je spojena s určitými omezeními. Za prvé, autor se zaměřil pouze na dva možné přístupy k vývoji webových řešení: tradiční přístup k vývoji a programování asistované pomocí nástrojů umělé inteligence. V praxi se však implementace webových stránek, zejména typu e-shop, často realizuje pomocí jiných přístupů, které nebyly do analýzy zahrnuty. Příklady zahrnují low-code a no-code platformy, speciální redakční systémy (CMS) nebo specializované e-commerce platformy.

Za druhé, autor použil k implementaci řešení pomocí AI pouze dva konkrétní nástroje. Existuje však celá řada AI systémů určených pro podporu programování, z nichž každý má své silné strany a může být svým způsobem přínosná při racionálním zařazení do vývojového procesu.

Omezením práce je také míra technických zkušeností a znalostí autora v oblasti softwarového vývoje. Zkušenost s různými způsoby vývoje by se mohla výrazně lišit pro vývojáře s různou kompetenční úrovní.

Použitá literatura

- Ahmed, N. A. (2024, leden 24). *The 10 Best Custom GPTs on the GPT Store*.
<https://www.datacamp.com/blog/best-custom-gpts-on-the-gpt-store>
- AI vs. Machine Learning vs. Deep Learning vs. Neural Networks* | IBM. (2023, červenec 6). <https://www.ibm.com/think/topics/ai-vs-machine-learning-vs-deep-learning-vs-neural-networks>
- Alammar, J., & Grootendorst, M. (2024). *Hands-On Large Language Models*. O'Reilly Media, Inc. <https://learning.oreilly.com/library/view/hands-on-large-language/9781098150952/>
- Aleksendrić, M., Borucki, A., Domingues, L., Abu Hammad, M., Hannouch, E., Nair, R., & Palmer, R. (2024). *Mastering MongoDB 7.0—Fourth Edition*. Packt Publishing. <https://learning.oreilly.com/library/view/mastering-mongodb-7-0/9781835460474/>
- Aytemiz, T. (2023, leden 14). How does the SDLC play a role in the success of product development and launch? *Agile Insider*. <https://medium.com/agileinsider/how-does-the-sdlc-play-a-role-in-the-success-of-product-development-and-launch-a17baaac1054>
- Belcic, I., & Stryker, C. (2024, září 18). *What is GPT (generative pre-trained transformer)?* | IBM. <https://www.ibm.com/think/topics/gpt>
- Belcic, I., & Stryker, C. (2025, duben 19). *What is ChatGPT?* | IBM. <https://www.ibm.com/think/topics/chatgpt>
- Berryman, W. J., & Ziegler, A. (2024). *Prompt Engineering for LLMs*. O'Reilly Media, Inc. <https://learning.oreilly.com/library/view/prompt-engineering-for/9781098156145/>

- Brown, T. (2023, srpen 29). Hub and Spoke Content Model: Improve Your Search Visibility. *[Https://Victorious.Com/](https://Victorious.Com/)*. <https://victorious.com/blog/hub-and-spoke-content-model/>
- Bruckner, T., Buchalceková, A., & Voříšek, J. (2012). *Tvorba informačních systémů*. Grada Publishing, a.s. <https://www.bookport.cz/e-kniha/tvorba-informacnich-systemu-2383133/>
- Buna, S. (2025). *Efficient Node.js: A Beyond-the-Basics Guide*. O'Reilly Media, Inc. <https://learning.oreilly.com/library/view/efficient-node-js/9781098145187/>
- Caelen, O., & Blete, M.-A. (2024). *Developing Apps with GPT-4 and ChatGPT, 2nd Edition*. O'Reilly Media, Inc. <https://learning.oreilly.com/library/view/developing-apps-with/9781098168094/>
- Campeato, O. (2024). *Large Language Models*. Stylus Publishing, LLC.
- Casteleyn, S., Dolog, P., Daniel, F., & Matera, M. (2009). *Engineering Web Applications*. Springer Science & Business Media. https://www.google.cz/books/edition/Engineering_Web_Applications/uOJJC6bH9EC?hl=cs&gbpv=1&dq=functional+requirements+for+web+applications&pg=PA82&printsec=frontcover
- Ceci, L. (2025, leden 28). *Global mobile traffic 2024*. Statista. <https://www.statista.com/statistics/277125/share-of-website-traffic-coming-from-mobile-devices/>
- Černý, D., Mařík, V., & Trčka, M. (2024). *Proč se nebát umělé inteligence?: AI pohledem nejen českých odborníků*. Jota.
- Dowswell, K. (2024). *Programming with GitHub Copilot*. Wiley. <https://learning.oreilly.com/library/view/programming-with-github/9781394263370/>
- Earle, Castledine, Eftos, Myles, & Wheeler, Max. (2013). *Vytváříme mobilní web a aplikace pro chytré telefony a tablety* (1. vyd). Computer Press.

- Enhancing Software Development with AI Tools: A practical Look at Cursor IDE. (2025, březn 12). *Eleks*. <https://eleks.com/research/cursor-ide/>
- Finio, M., & Downie, A. (2024, říjen 7). *AI in Software Development | IBM*. <https://www.ibm.com/think/topics/ai-in-software-development>
- Frisbie, M. (2023). *Professional JavaScript for Web Developers, 5th Edition*. Wrox. <https://learning.oreilly.com/library/view/professional-javascript-for/9781394193219/>
- Geetha, T. V., & Sendhilkumar, S. (2023). *Machine Learning: Concepts, Techniques and Applications*. CRC Press.
- Gurnov, A. (2025). *What is Iterative and Incremental Development? | Wrike*. <https://www.wrike.com/project-management-guide/faq/what-is-iterative-incremental-development/>
- Hoffman, A. (2020). *Web Application Security: Exploitation and Countermeasures for Modern Web Applications*. O'Reilly Media, Inc.
- Hutsulyak, O. (2024, srpen 1). *Why Use React for Web Development: 10 Reasons To Apply*. Blog | TechMagic. <https://www.techmagic.co/blog/why-we-use-react-js-in-the-development/>
- Cherukuri, B. R. (2024). (PDF) AI-powered personalization: How machine learning is shaping the future of user experience. *ResearchGate*. <https://doi.org/10.30574/ijjsra.2024.12.1.0961>
- Jain, S., & Chittezath, M. D. (2023). *Modern Web Applications with Next.JS: Learn Advanced Techniques to Build and Deploy Modern, Scalable and Production Ready React Applications with Next.JS*. Orange Education Pvt Ltd.
- Kluza, T., & Nowak, M. (2024, listopad 18). *AI in Code Analysis: Benefits and Challenges*. <https://vmsoftwarehouse.com/ai-in-code-analysis-benefits-and-challenges>
- Kumar, T. (2024). *Fluent React*. O'Reilly Media, Inc. <https://learning.oreilly.com/library/view/fluent-react/9781098138707/>

- Kush, J. (2025, duben 11). Vibe Coding with AI: The Security Risks We're Ignoring. *Medium*.
<https://medium.com/@janani.kush/vibe-coding-with-ai-the-security-risks-were-ignoring-d5dbd30a9fe9>
- Laplante, P. A. (2007). *What Every Engineer Should Know about Software Engineering*. CRC Press.
- Lee, K. (b.r.). *Exploring Reinforcement Learning with Human Feedback*. Kili Technology. Získáno 4. červen 2025, z <https://kili-technology.com/large-language-models-llms/exploring-reinforcement-learning-from-human-feedback-rlhf-a-comprehensive-guide>
- Minjire, V. (2024, leden 24). Understanding SSR in Next.js and Its Benefits. *Medium*.
<https://medium.com/@markminj/understanding-ssr-in-next-js-and-its-benefits-e54ffed48294>
- O'Grady, B. (2020, březn 5). What is JavaScript and What is it Used for? *Code Institute Global*. <https://codeinstitute.net/global/blog/what-is-javascript-and-why-should-i-learn-it/>
- O'Regan, G. (2008). *A Brief History of Computing*. Springer Science & Business Media.
- Osmani, A. (2025). *Beyond Vibe Coding*. O'Reilly Media, Inc.
<https://learning.oreilly.com/library/view/beyond-vibe-coding/9798341634749/>
- Osyenko, A. (b.r.). *Go, AWS & React Cloud-Native Development*. MadAppGang. Získáno 26. červen 2025, z <https://madappgang.com/blog/single-page-applications-vs-multi-page-applications/>
- Parody, L. (2025, leden 28). *How Node.js Works: A Comprehensive Guide in 2025*. The NodeSource Blog - Node.js Tutorials, Guides, and Updates.
<http://nodesource.com/blog/how-nodejs-works/>
- Prabhu, T. (2025). *Generative AI: From Algorithms to Art in the Age of Machine Creativity*. NestFame Creations Pvt Ltd.
- Rafalski, K. (2025a). *10 Web Application Types and How They Benefit Businesses*.
<https://www.netguru.com/blog/web-application-types>

- Rafalski, K. (2025b, květen 20). *16 Top Web Development Trends in 2025*.
<https://www.netguru.com/blog/web-development-trends>
- Rascia, T. (2020). *Understanding the DOM — Document Object Model*. DigitalOcean.
- Robertson, J., & Robertson, S. (1995). *Šablona pro specifikaci požadavků*. The Atlantic Systems Guild Limited. www.systemsguild.com
- Rodrigues, D. (2025). *LEARN NEXT.JS: From Fundamentals to Practical Applications*. StudioD21 Smart Tech Content & Intell Systems.
- Shelby, A. (2024, červen 11). *What is Next.js and why developers love it*. Austin Shelby.
<https://www.austinshelby.com/blog/what-is-next-js-and-why-developers-love-it>
- Shrivastava, A. (2025, únor 28). *AI Use Cases for Developers: How AI is Transforming Software Development*. <https://www.moveworks.com/us/en/resources/blog/ai-use-cases-for-developers>
- Single-page applications (SPAs)—What they are and how they work*. (2023).
<https://business.adobe.com/blog/basics/learn-the-benefits-of-single-page-apps-spa>
- Smalley, I., & Susnjara, S. (2024). *What Is Serverless Computing? | IBM*.
<https://www.ibm.com/think/topics/serverless>
- Software Developers Statistics 2024—State of Developer Ecosystem Report*. (2024). JetBrains: Developer Tools for Professionals and Teams.
<https://www.jetbrains.com/lp/devecosystem-2024>
- Stevens, E. (2024, únor 16). Will AI (Artificial Intelligence) replace UX Designers? *UX Design Institute*. <https://www.uxdesigninstitute.com/blog/will-ai-replace-ux-designers/>
- Stöffelbauer, A. (2023, říjen 24). How Large Language Models Work. *Data Science at Microsoft*. <https://medium.com/data-science-at-microsoft/how-large-language-models-work-91c362f5b78f>

- Supervised vs Unsupervised vs Reinforcement Learning*. (2025). GeeksforGeeks.
<https://www.geeksforgeeks.org/machine-learning/supervised-vs-reinforcement-vs-unsupervised/>
- Šilhavý, R., Šilhavý, P., Prokopová, Z., Pokorný, P., Sysel, M., Matýsek, M., Vlček, K., & Svobodová, L. (2013). *Vybrané aspekty návrhu webových informačních systémů*. Scientific press. www.scientificpress.eu
- Tam, A. (2023, červen 1). A Gentle Introduction to Hallucinations in Large Language Models. *MachineLearningMastery.Com*.
<https://www.machinelearningmastery.com/a-gentle-introduction-to-hallucinations-in-large-language-models/>
- Tarek, S. (2023, září 19). *Web app development: Types of web apps you can create*.
<https://decode.agency/article/web-app-development-app-types/>
- Taulli, T. (2024). *AI-Assisted Programming*. <https://learning.oreilly.com/library/view/ai-assisted-programming/9781098164553/>
- Teselko, O. (2025, leden 13). *Headless CMS vs. Traditional CMS: A Guide for Growing Businesses* | Storyblok. <https://www.storyblok.com/mp/headless-cms-vs-traditional-cms>
- Tomasis, R. (2025, leden 12). *Mobile-first vs responsive design: All you need to know*. Wix Blog. <https://www.wix.com/blog/mobile-first-vs-responsive-design-all-you-need-to-know>
- Understand progressive web apps (PWAs) and how to set them up*. (2022).
<https://business.adobe.com/blog/basics/progressive-web-apps-overview>
- What is a progressive web app? - Progressive web apps* | MDN. (2025, květen 5).
https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/What_is_a_progressive_web_app
- What Is MongoDB?* | IBM. (2021, říjen 14). <https://www.ibm.com/think/topics/mongodb>
- What is React.js? Overview of the JavaScript Library for UI Development - Glossary*. (2024, srpen 23). Sanity.Io. <https://www.sanity.io/glossary/react-js>

What is the Waterfall Model in Software Engineering? Waterfall Software Development Model Phases and Examples. (2014, únor 5). <https://www.oxagile.com/article/the-waterfall-model/>

Whelan, L. (2025, květen 7). *Real-world Use Cases of AI Code Generation.* <https://zencoder.ai/blog/ai-code-generation-use-cases>

